

HYBRID MODELLING

Combining Machine Learning
With
Knowledge-based Models For
Biological Data

BioSB Summer School on Advance Machine
Learning and AI for Life Science

August 26th Lunteren



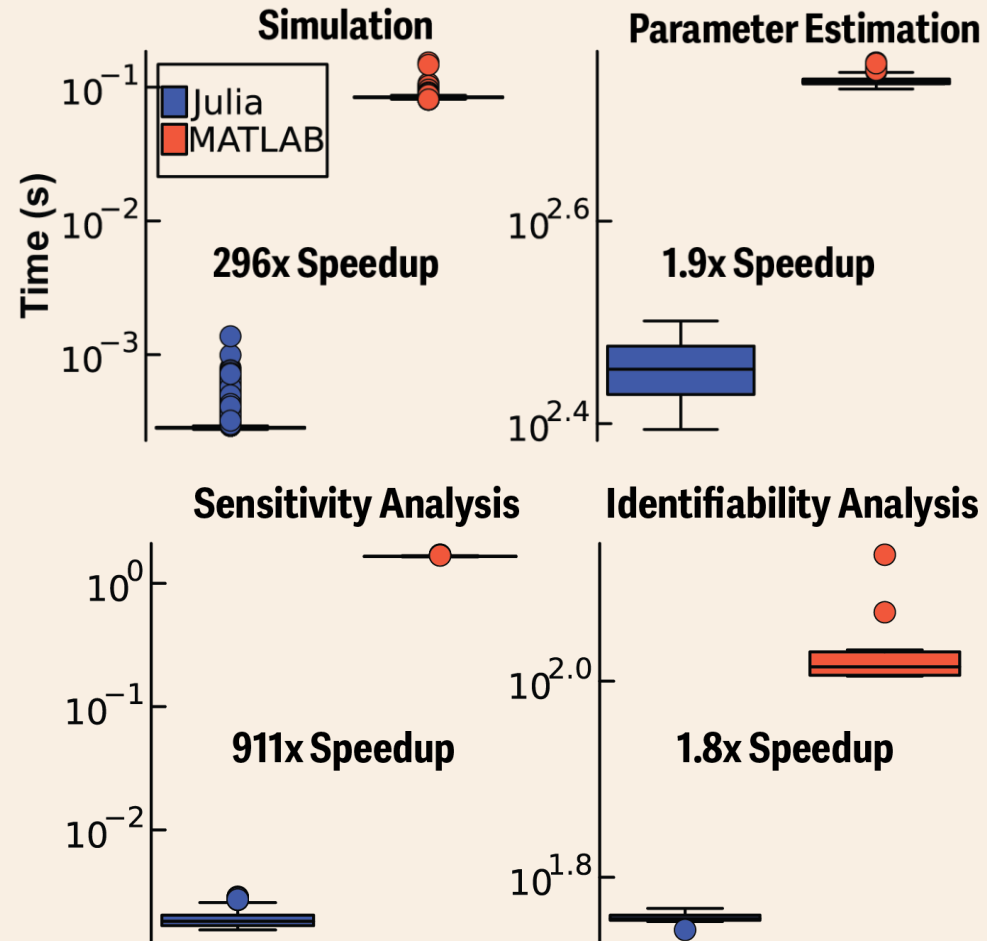
AGENDA

time	Topic	activity
09:00-09:15	Introduction - bottom-up v top-down: what is hybrid modelling	Lecture
09:15 - 09:45	Intro to ODE-based models in systems biology and in Julia	Lecture and practical
09:45 - 10:15	Hybrid models in series	Lecture and practical
10:15 - 13:30	Systems Biology-Informed Neural Networks -	Lecture
10:30-10:45	break	
10:45-11:00	Universal Differential Equations	Lecture
11:00 - 12:00	Hands-on UDEs in Julia - regularization and model training	practical
12:00-12:30	Symbolic regression	Lecture and practical

MATERIALS



Systems Biology



nature methods

Perspective

<https://doi.org/10.1038/s41592-023-01832-z>

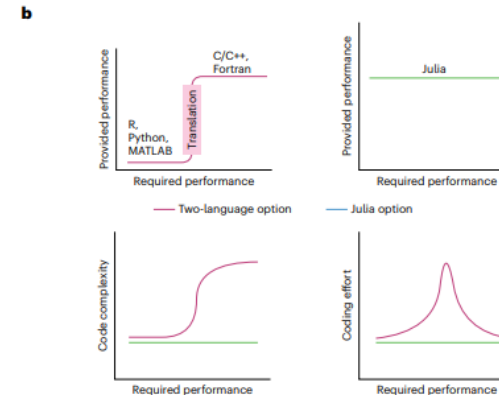
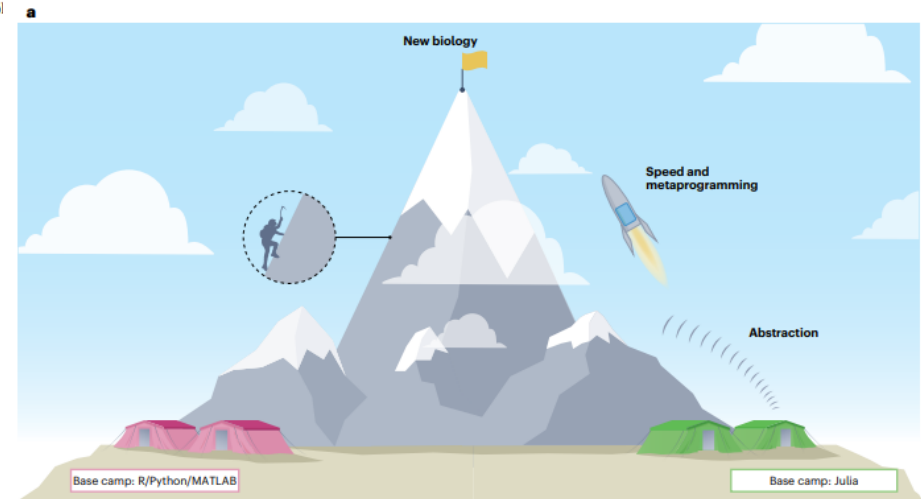
Julia for biologists

Received: 21 September 2021

Accepted: 27 February 2023

Elisabeth Roesch^{1,2,3}, Joe G. Greener⁴, Adam L. MacLean⁵,
Huda Nassar⁶, Christopher Rackauckas^{3,7,8}, Timothy E. Holy⁹
& Michael P. H. Stumpf^{1,2,10,11}✉

^{Publ}



Current project	Julia	R, Python, MATLAB	C/C++, Fortran
Add new data types?	✓	✗	✓
Add new functions?	✓	✓	✗

MATERIALS

 Platform ▾ Solutions ▾ Resources ▾ Open Source ▾ Enterprise ▾ Pricing



Sign in

Sign up



 BioSB-courses / BioSB-SummerSchool-2026

Public

 Notifications

 Fork 0

 Star 3

<> Code Issues Pull requests Actions Projects Security and quality Insights

 Files

main



Go to file

>  Lecture-slides

>  Practical-1-Explainable-AI

>  Practical-2-Generative-models

>  Practical-3-Scientific-machine-lea...

>  cat

 1-ode-models.ipynb

 2-serial-hybrid.ipynb

 3-pinns.ipynb

BioSB-SummerSchool-2026 / Practical-3-Scientific-machine-learning / 

 pdmoerland Updates to the scientific machine learning session ✓ e84d15b · 4 hours ago  History

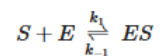
Name	Last commit message	Last commit date
 ..		
 cat	Add Julia hands-on	4 days ago
 1-ode-models.ipynb	Updates to the scientific machine learning...	4 hours ago
 2-serial-hybrid.ipynb	Add Julia hands-on	4 days ago
 3-pinns.ipynb	Add Julia hands-on	4 days ago
 4-udes.ipynb	Updates to the scientific machine learning...	4 hours ago

MATERIALS

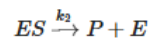
Example model: an Enzyme-catalyzed reaction

In this example, we will model a simple enzyme-catalyzed reaction using ordinary differential equations (ODEs). We have a substrate (S), an enzyme (E), and a product (P). The reactions are as follows:

1. The substrate binds to the enzyme to form a complex (ES).



2. The complex then converts to the product and releases the enzyme.



Using mass-action kinetics, we can represent the dynamics of the system with the following ODEs:

$$\begin{aligned}\frac{d[S]}{dt} &= -k_1[S][E] + k_{-1}[ES] \\ \frac{d[E]}{dt} &= -k_1[S][E] + (k_{-1} + k_2)[ES] \\ \frac{d[ES]}{dt} &= k_1[S][E] - (k_{-1} + k_2)[ES] \\ \frac{d[P]}{dt} &= k_2[ES]\end{aligned}$$

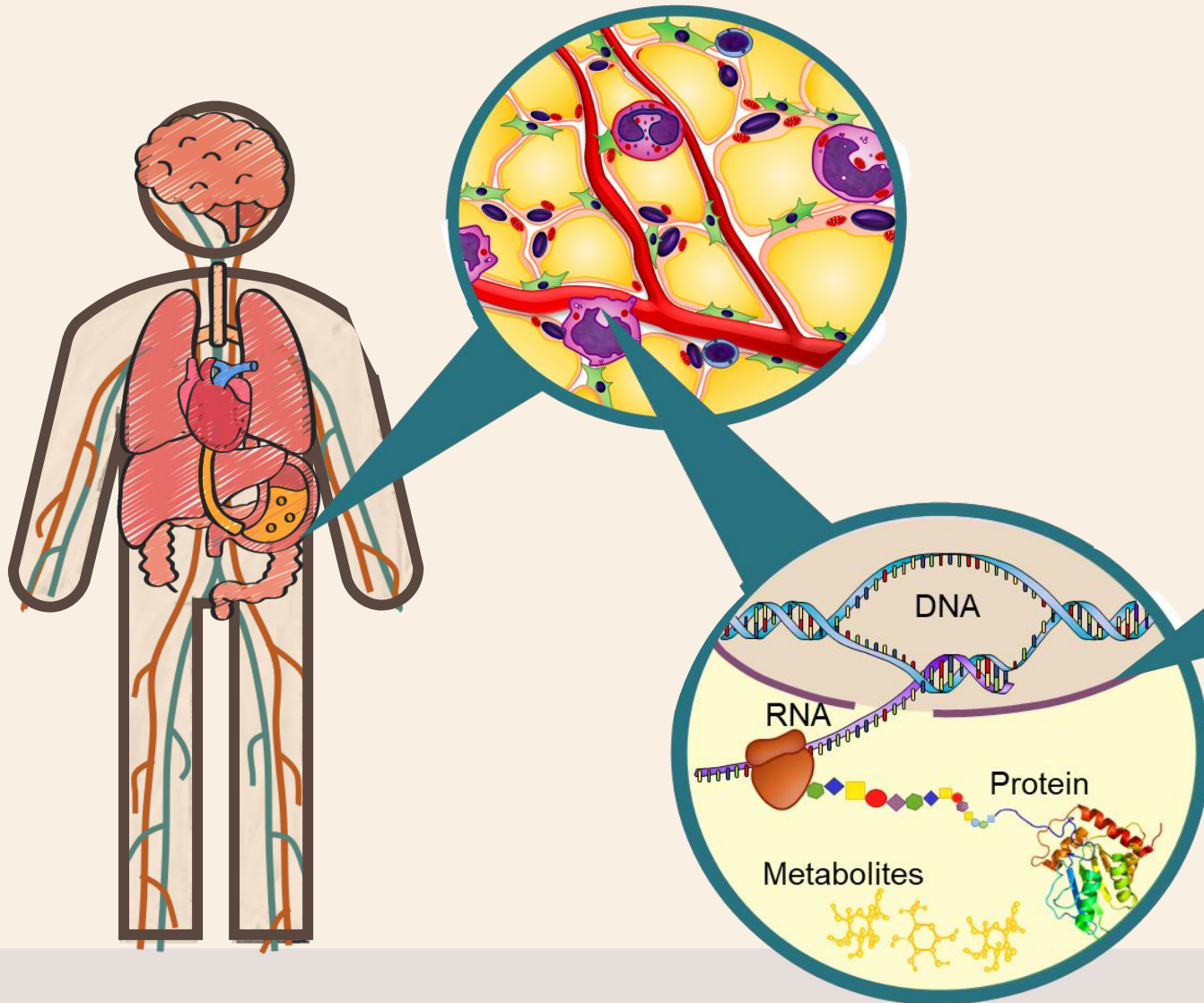
We can implement this model in Julia using the `OrdinaryDiffEq` package

We first implement the set of equations as a function with the signature `f(du, u, p, t)`, where `u` is the vector of state variables, `p` is the vector of parameters, and `t` is time. The function will compute the derivatives of the state variables and store them in `du`.

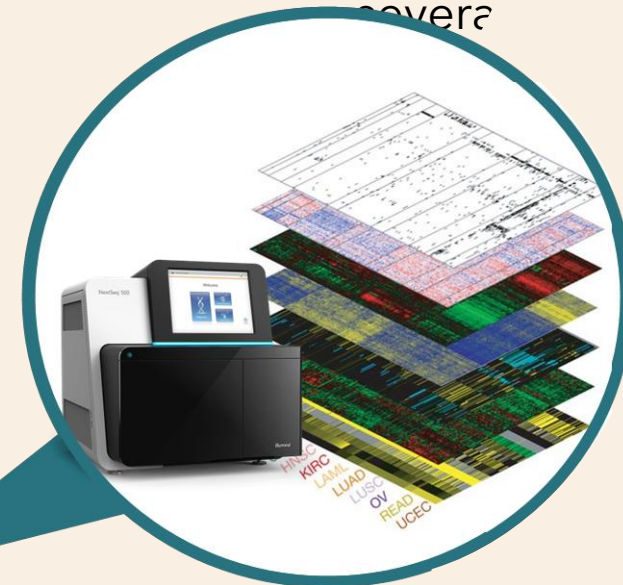
```
In [2]: function enzyme_reaction!(du, u, p, t)
        k1, kn1, k2 = p
        S, E, ES, P = u

        du[1] = -k1 * S * E + kn1 * ES
        du[2] = -k1 * S * E + (kn1 + k2) * ES
        du[3] = k1 * S * E - (kn1 + k2) * ES
        du[4] = k2 * ES
    end
```


WHY SYSTEMS BIOLOGY?



The human body consists of many different **interconnected systems** operating over temporal and spatial



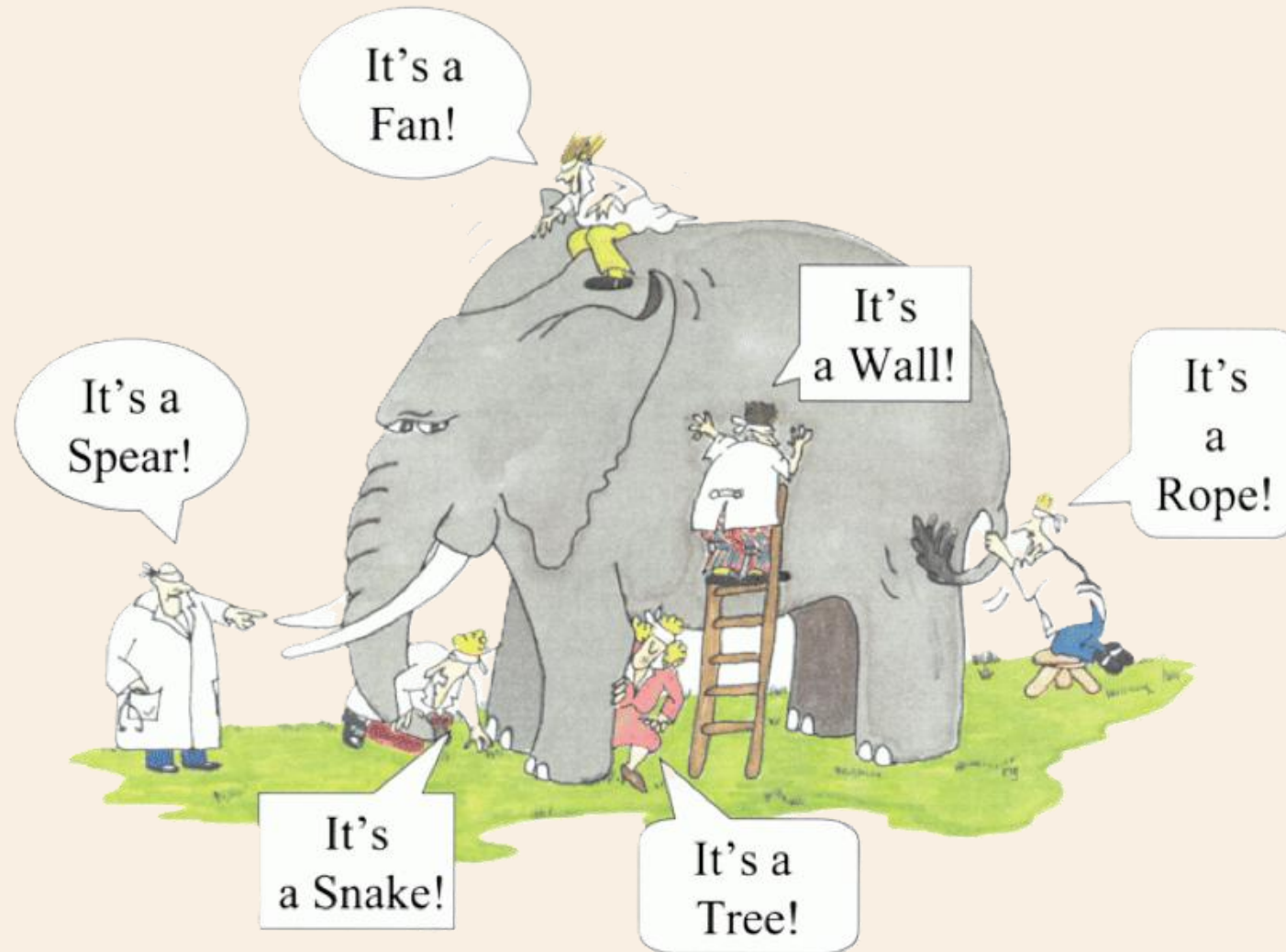
With advances in **high-throughput omics** technologies we can now observe and measure these systems.

BIGGER challenges

- High dimensional data.
- Different properties and structure.
- Small sample sizes.



WHY SYSTEMS BIOLOGY?

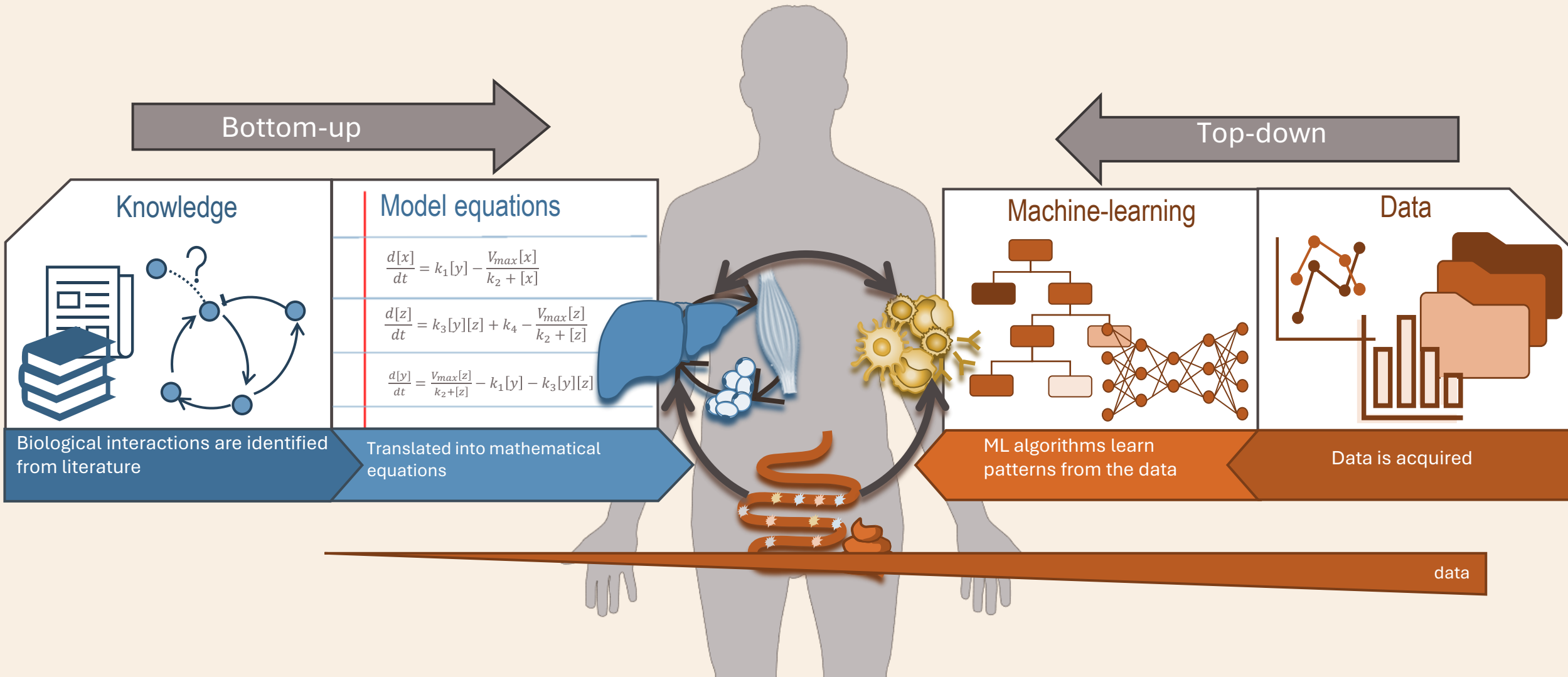


Why Systems Biology?

Systems biology focuses on the **mathematical analysis and modelling of complex biological systems.**



TOP-DOWN V BOTTOM-UP



BOTTOM-UP MODELS

Physical system

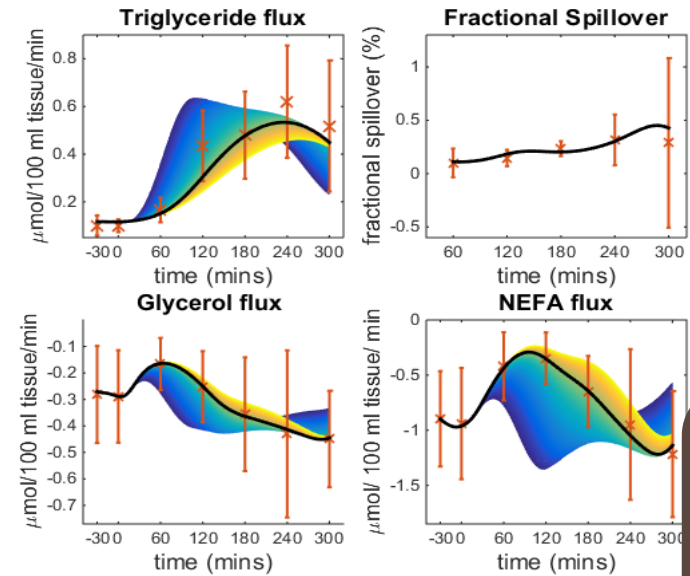
Virtual system

Model equations

$$\frac{d[x]}{dt} = k_1[y] - \frac{V_{max}[x]}{k_2 + [x]}$$

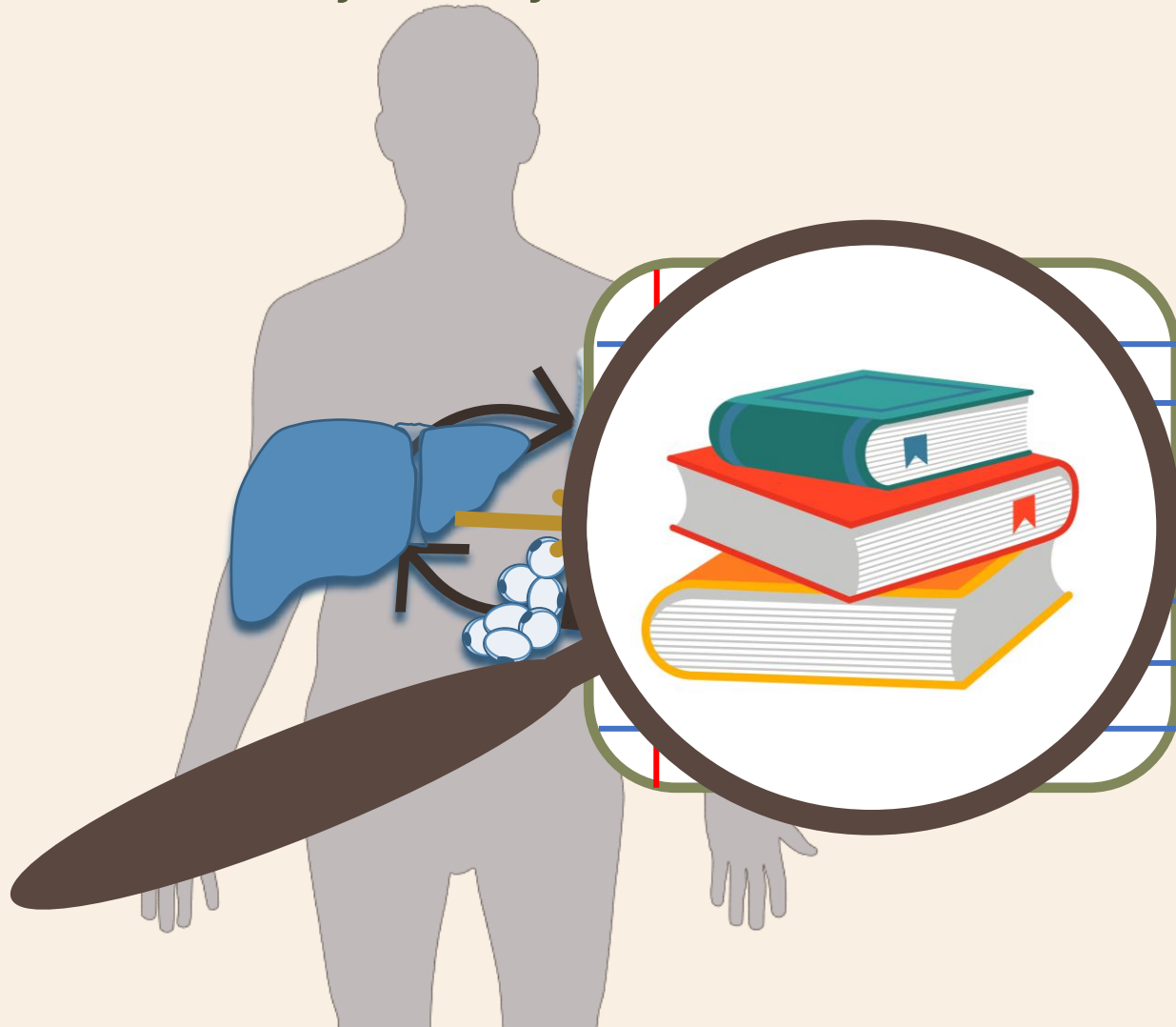
$$\frac{d[y]}{dt} = \frac{V_{max}[z]}{k_2 + [z]} - k_1[y] - k_3[y][z]$$

$$\frac{d[z]}{dt} = k_3[y][z] + k_4 - \frac{V_{max}[z]}{k_2 + [z]}$$

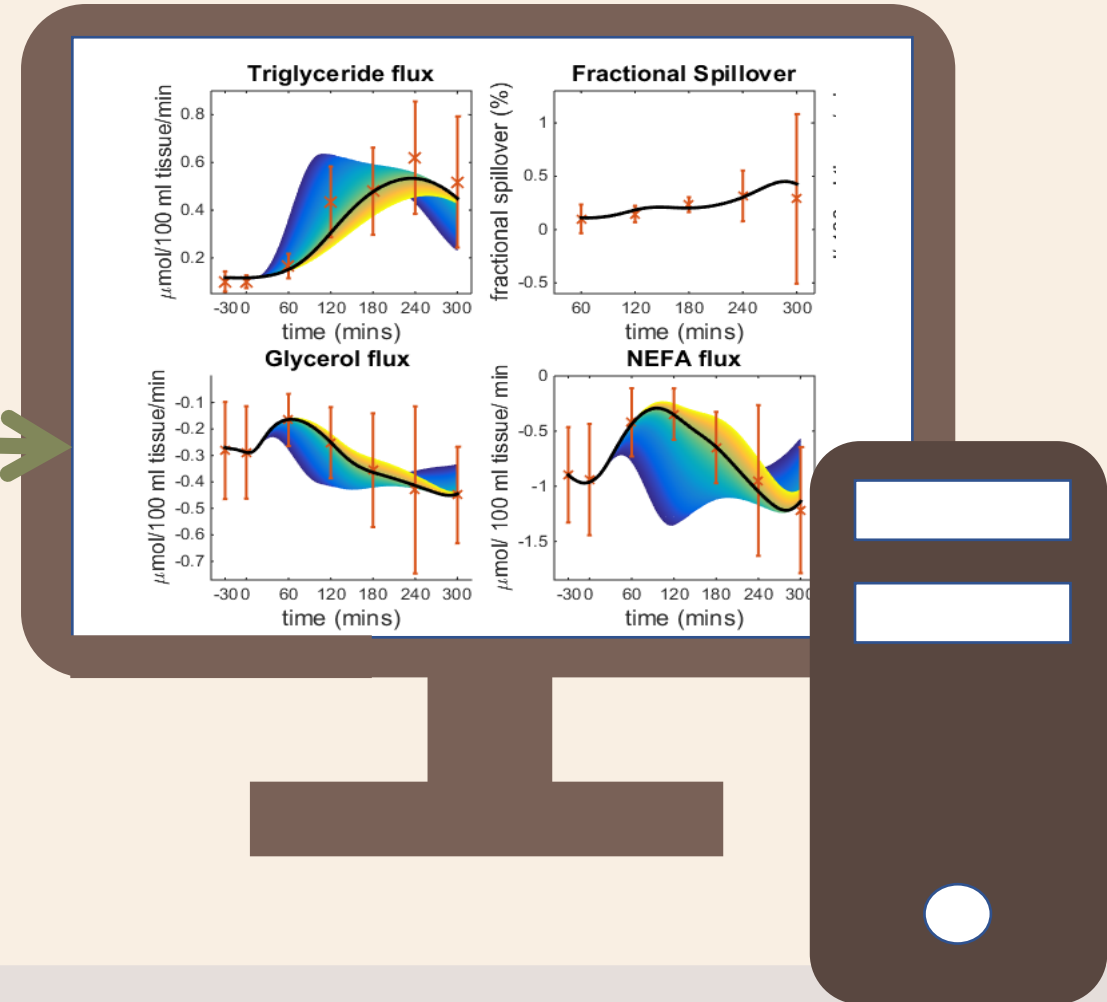


BOTTOM-UP MODELS

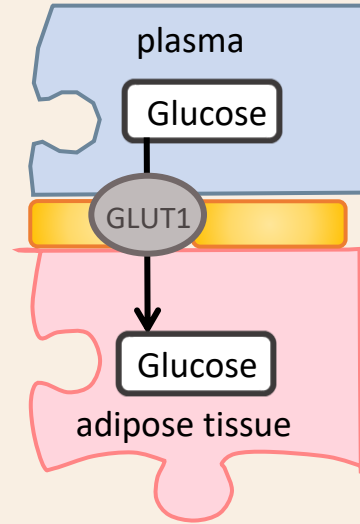
Physical system



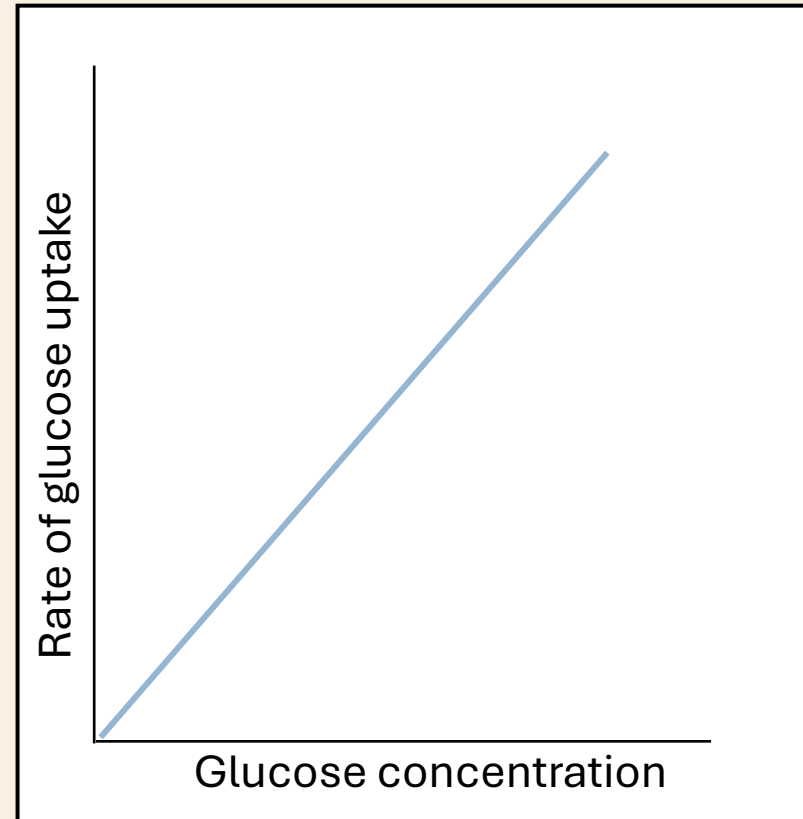
Virtual system



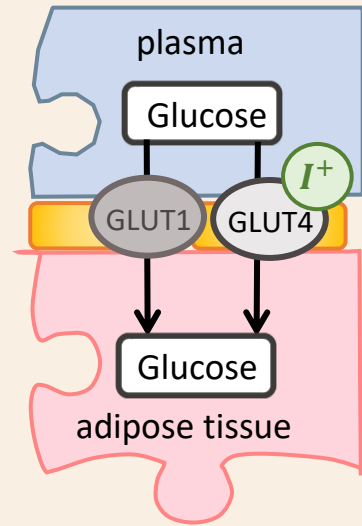
MODEL CONSTRUCTION



$$\frac{d[\text{glucose}_{AT}]}{dt} = k_{GLUT1}[\text{glucose}_{PL}]$$

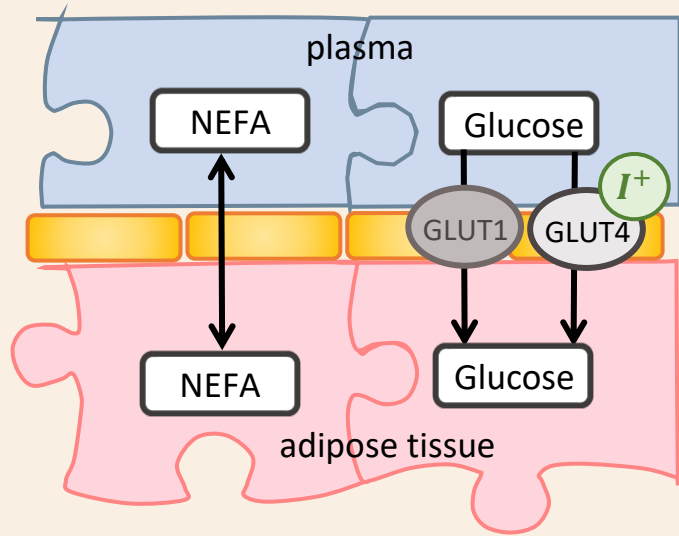


MODEL CONSTRUCTION



$$\frac{d[\text{glucose}_{AT}]}{dt} = k_{GLUT1}[\text{glucose}_{PL}] + k_{GLUT4}[\text{insulin}][\text{glucose}_{PL}]$$

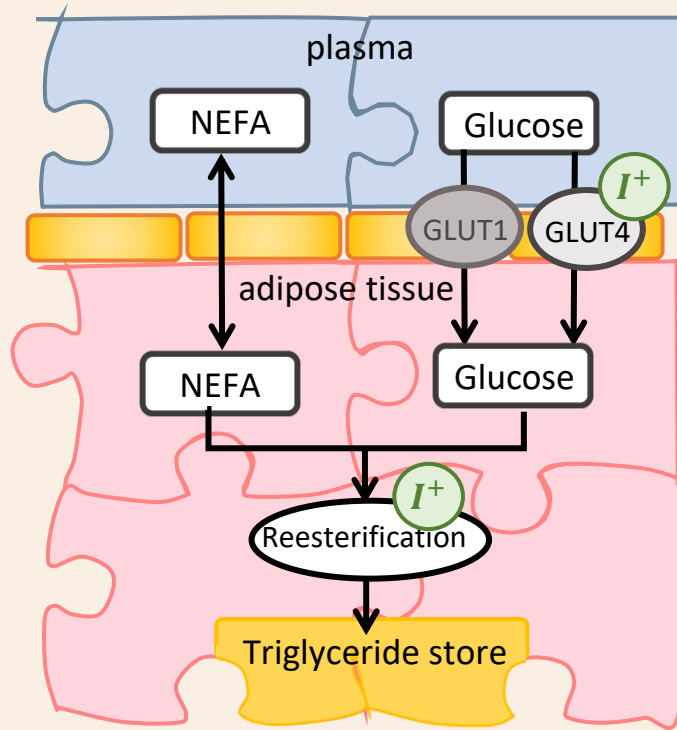
MODEL CONSTRUCTION



$$\frac{d[\text{glucose}_{AT}]}{dt} = k_{GLUT1}[\text{glucose}_{PL}] + k_{GLUT4}[\text{insulin}][\text{glucose}_{PL}]$$

$$\frac{d[NEFA_{AT}]}{dt} = k_{NEFA}([NEFA_{PL}] - [NEFA_{AT}])$$

MODEL CONSTRUCTION

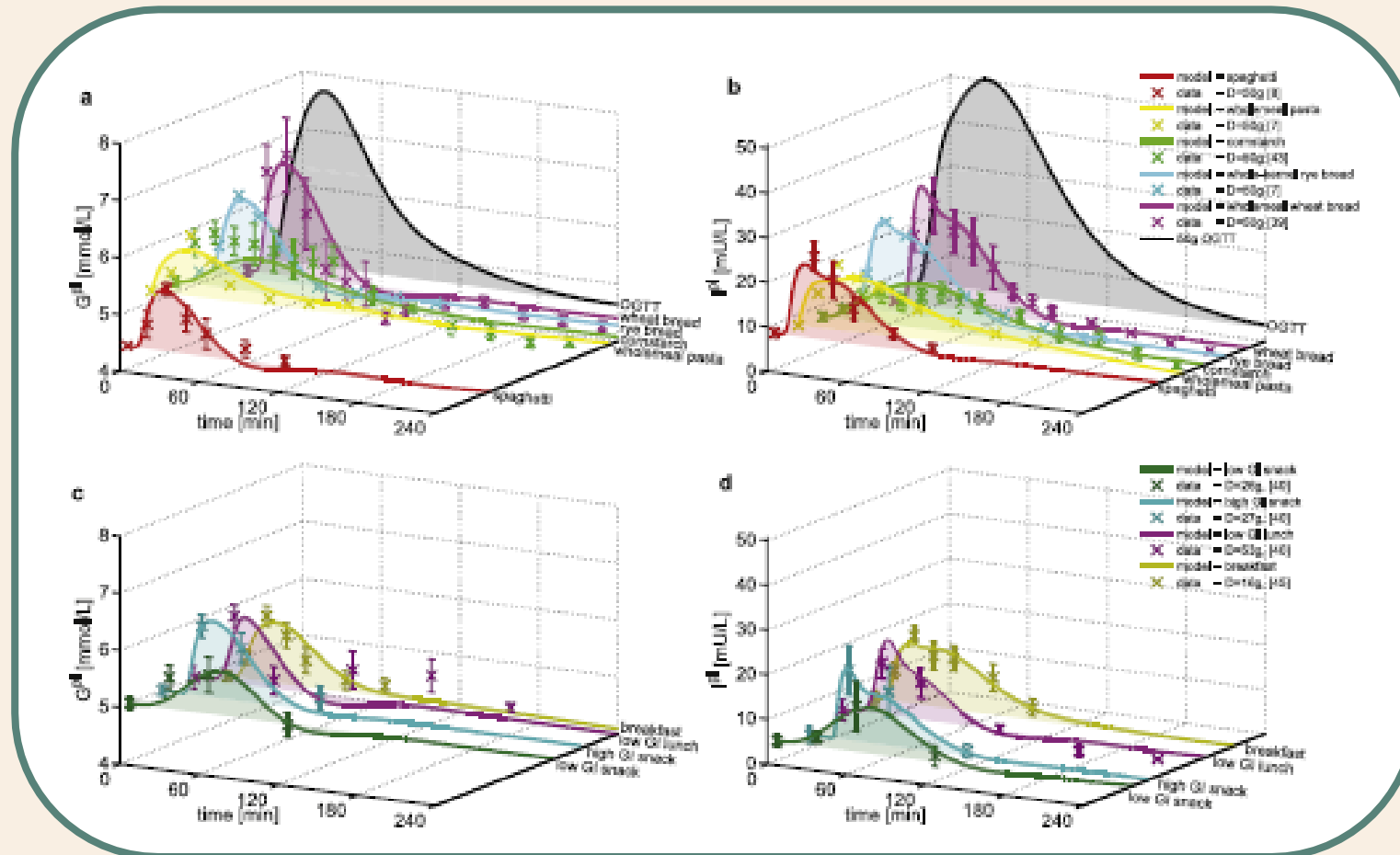


$$\frac{d[\text{glucose}_{AT}]}{dt} = k_{GLUT1}[\text{glucose}_{PL}] + k_{GLUT4}[\text{insulin}][\text{glucose}_{PL}] - k_{reest}[\text{NEFA}_{AT}][\text{glucose}_{AT}][\text{insulin}]$$

$$\frac{d[\text{NEFA}_{AT}]}{dt} = k_{NEFA}([\text{NEFA}_{PL}] - [\text{NEFA}_{AT}]) - k_{reest}[\text{NEFA}_{AT}][\text{glucose}_{AT}][\text{insulin}]$$

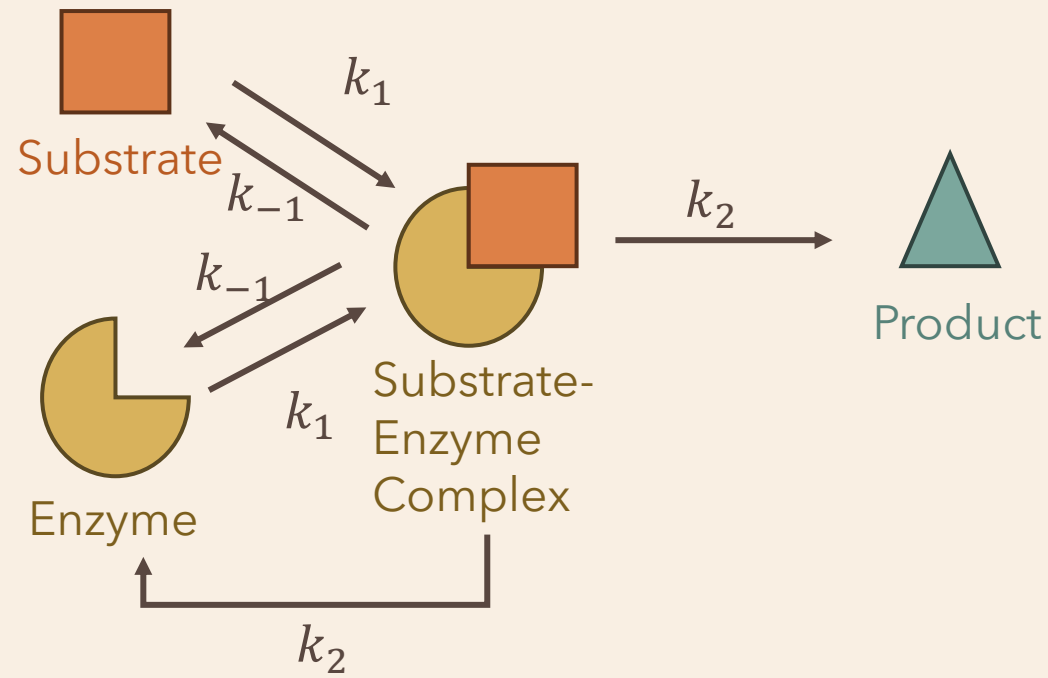
$$\frac{d[\text{triglyceride}_{AT}]}{dt} = k_{reest}[\text{NEFA}_{AT}][\text{glucose}_{AT}][\text{insulin}]$$

MODEL SIMULATION



MODEL SIMULATION

- Model of enzymatic catalysis of a reaction



Model Enzyme Catalysis

$$\frac{d[S]}{dt} = -k_1[S][E] + k_{-1}[ES]$$

$$\frac{d[E]}{dt} = -k_1[S][E] + (k_{-1} + k_2)[ES]$$

$$\frac{d[ES]}{dt} = k_1[S][E] - (k_{-1} + k_2)[ES]$$

$$\frac{d[P]}{dt} = k_2[ES]$$

MODEL SIMULATION

Step 1: Implement model equations as a function in the language of your choice

Model Enzyme Catalysis

$$\begin{aligned}\frac{d[S]}{dt} &= -k_1[S][E] + k_{-1}[ES] \\ \frac{d[E]}{dt} &= -k_1[S][E] + (k_{-1} + k_2)[ES] \\ \frac{d[ES]}{dt} &= k_1[S][E] - (k_{-1} + k_2)[ES] \\ \frac{d[P]}{dt} &= k_2[ES]\end{aligned}$$

Function inputs:

- Current state (du)
- Initial values (u)
- Parameters (p)
- Time (t)

```
function enzyme_reaction!(du, u, p, t)

    # unpack parameters (p)
    k1, kn1, k2 = p
    # unpack initial values (u)
    S, E, ES, P = u

    # define equations
    du[1] = -k1 * S * E + kn1 * ES
    du[2] = -k1 * S * E + (kn1 + k2) * ES
    du[3] = k1 * S * E - (kn1 + k2) * ES
    du[4] = k2 * ES

end
```

Define differential equations as a function of state variables and parameters

End function output is the derivatives of the state variables du

MODEL SIMULATION

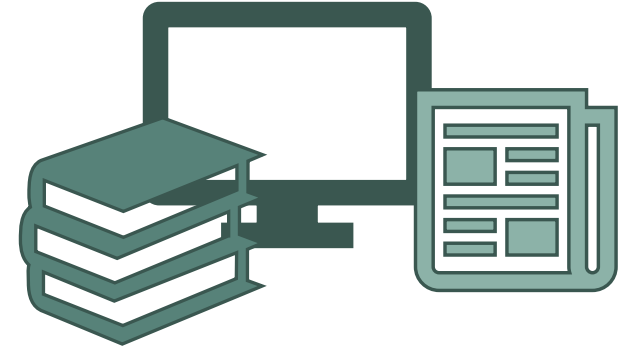
Step 2: Numerically approximate the solution of the system of ODEs using an ODE solver

```
# define necessary
p = [0.7, 0.5, 0.1]
u0 = [1.0, 2.0, 0.0]
tspan = (0.0, 10.0)

#define ODE problem
prob = ODEProblem(enzyme_, u0, tspan)

#solve ODE problem
solution = solve(prob, Tsit5(), saveat=0.1),
```

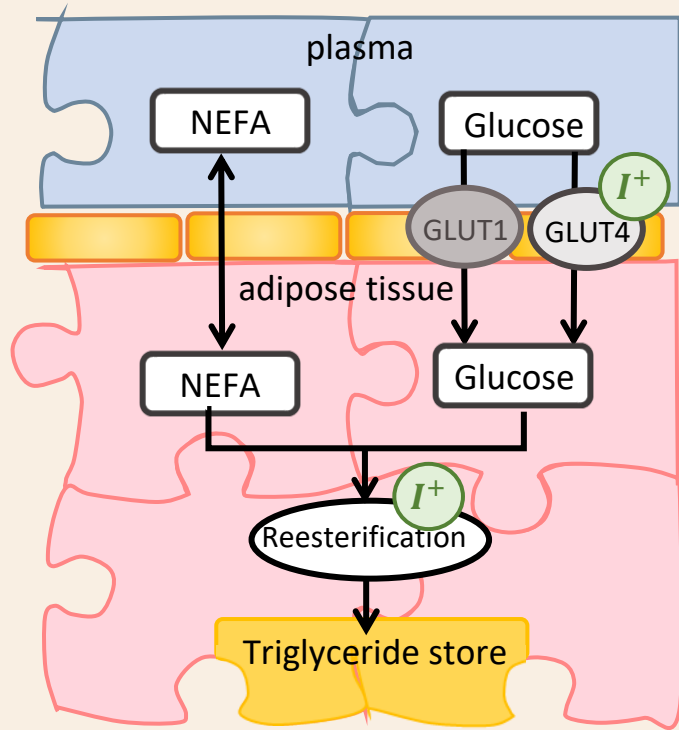
To the literature



ODE
system directing
to inputs

Solve the ODE problem,
here we also specify the
ODE solver (Tsit5)

MODEL SIMULATION



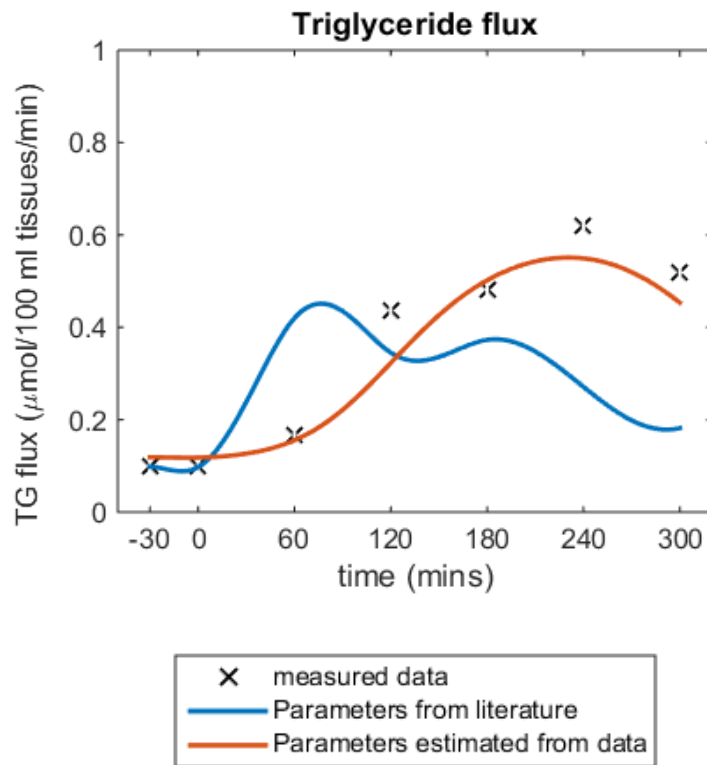
$$\frac{d[\text{glucose}_{AT}]}{dt} = k_{GLUT1}[\text{glucose}_{PL}] + k_{GLUT4}[\text{insulin}][\text{glucose}_{PL}] - k_{reest}[\text{NEFA}_{AT}][\text{glucose}_{AT}][\text{insulin}]$$

$$\frac{d[\text{NEFA}_{AT}]}{dt} = k_{NEFA}([\text{NEFA}_{PL}] - [\text{NEFA}_{AT}]) - k_{reest}[\text{NEFA}_{AT}][\text{glucose}_{AT}][\text{insulin}]$$

$$\frac{d[\text{triglyceride}_{AT}]}{dt} = k_{reest}[\text{NEFA}_{AT}][\text{glucose}_{AT}][\text{insulin}]$$

MAXIMUM LIKELIHOOD ESTIMATION

$$\frac{d[TG_{PL}]}{dt} = \theta_1 [I_{RLPL}] [TG_{PL}]$$



Model can be described as

$$y(t) = f(u(t), \theta) + \xi(t)$$

Probability density function

$$p(\xi(t)) = \frac{1}{\sqrt{2\pi\sigma}} e^{-\xi(t)^2 / (2\sigma^2)}$$

Maximise the likelihood

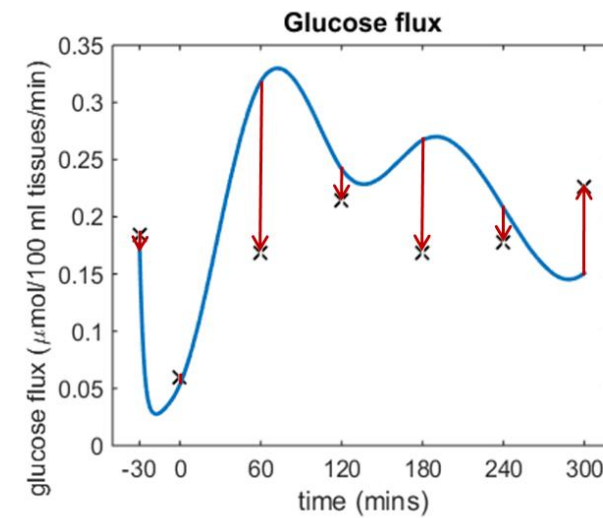
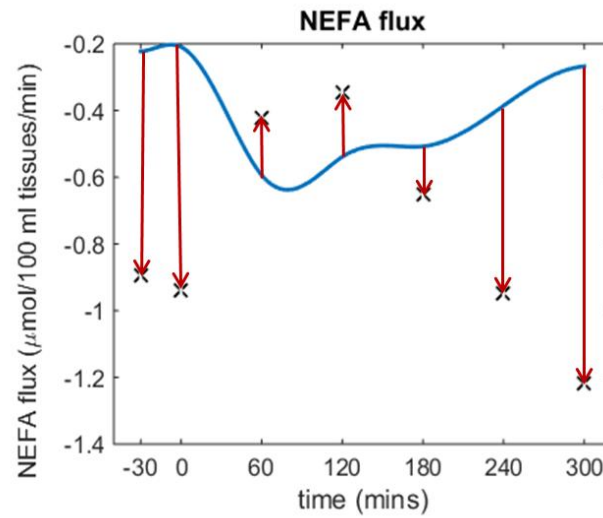
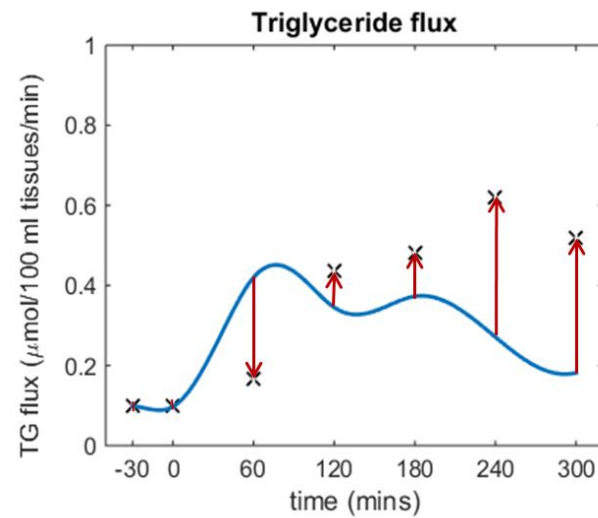
$$\hat{\theta} = \widehat{\max_{\theta} \ell(\theta)}$$

DATA ASSIMILATION

$$\frac{d[TG_{PL}]}{dt} = k_{ai}[I_{LPL}][TG_{PL}]$$

$$\frac{d[NEFA_{PL}]}{dt} = k_{spill}(k_{ai}[I_{LPL}][TG_{PL}]) + P_{NEFA}([NEFA_{AT}] - [NEFA_{PL}])$$

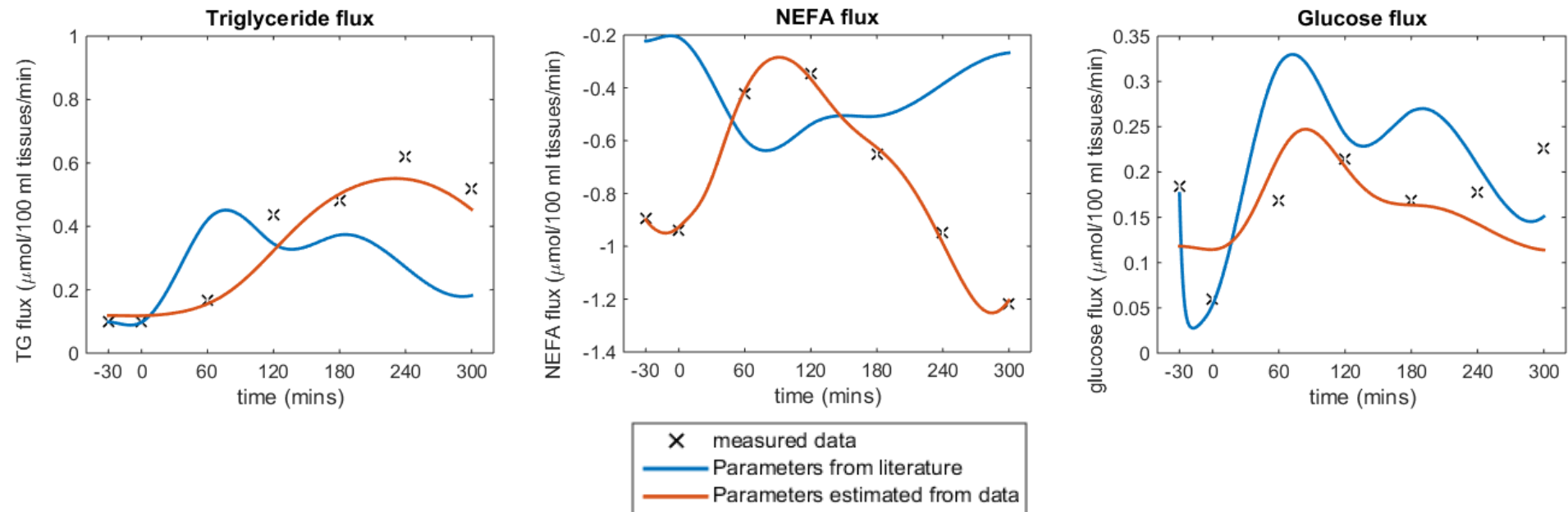
$$\frac{d[G_{PL}]}{dt} = -GLUT1[G_{PL}] - GLUT4[I_{AT}][G_{PL}]$$



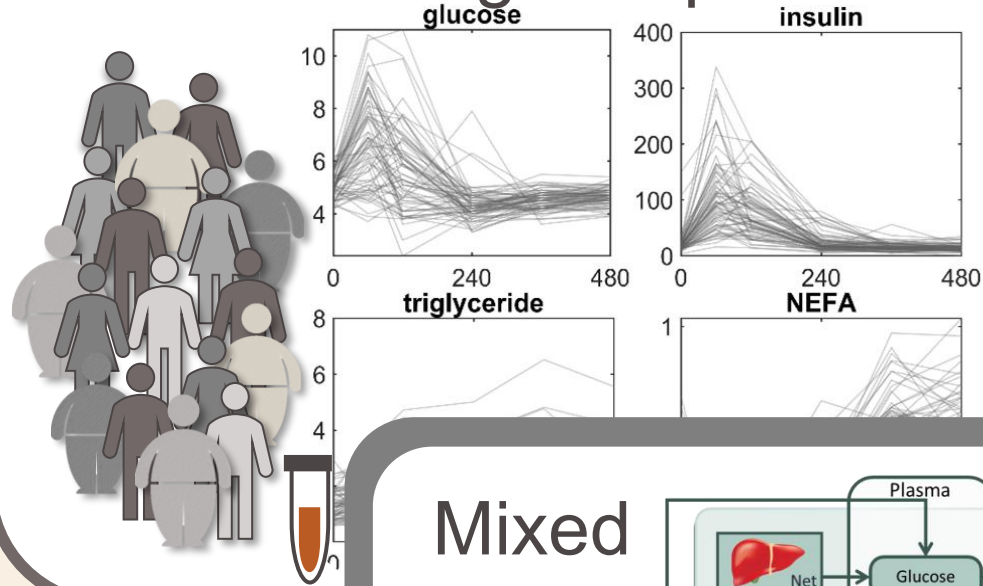
× measured data
— Parameters from literature

DATA ASSIMILATION

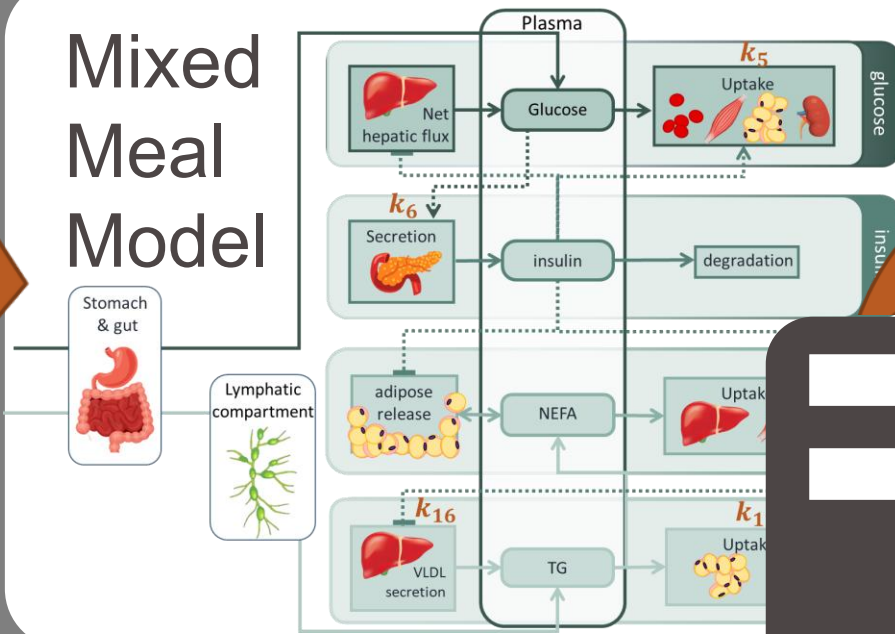
$$\frac{d[TG_{PL}]}{dt} = 0.0096[I_{LPL}][TG_{PL}] \quad \frac{d[NEFA_{PL}]}{dt} = 0.75(0.0096[I_{LPL}][TG_{PL}]) + 0.044([NEFA_{AT}] - [NEFA_{PL}]) \quad \frac{d[G_{PL}]}{dt} = -0.017[G_{PL}] - 0.0006[I_{AT}][G_{PL}]$$



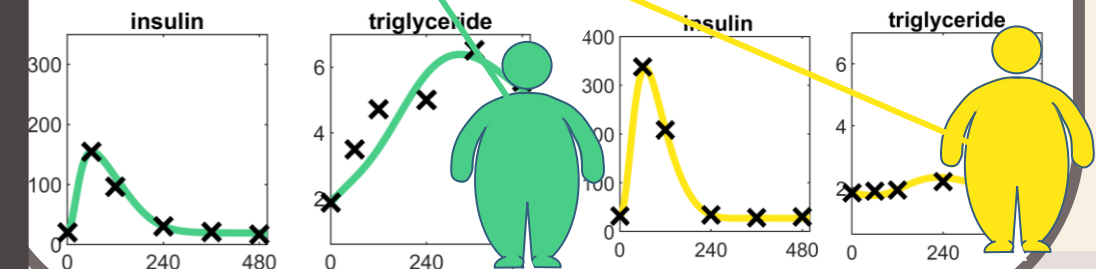
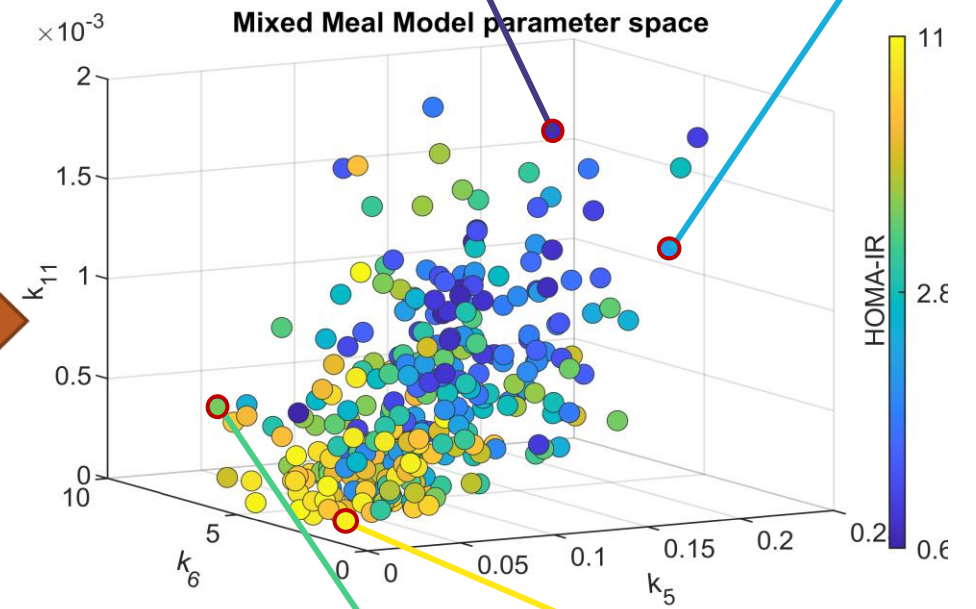
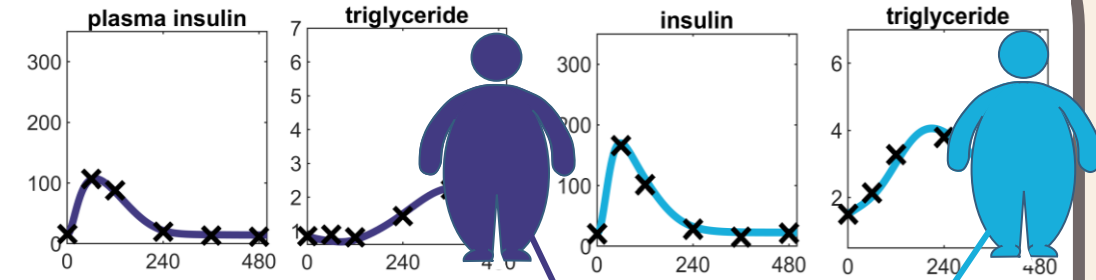
Meal challenge responses



Mixed Meal Model



Metabolic resilience phenotype



DATA ASSIMILATION

Step 1: Define your error function/ log likelihood

```
function negative_log_likelihood(p_est, (prob, substrate, product, timepoints))  
  
    # estimate parameters in log-scale to prevent negative values  
    p_est_scaled = exp.(p_est)  
  
    u0 = [substrate[1], 8e-1, 0.0, product[1]]  
  
    solution = Array(solve(prob, Rodas5P(), p=p_est_scaled[1:3], saveat=timepoints, save_idxs=[1,4]))  
  
    sigma_substrate = p_est_scaled[4]  
    sigma_product = p_est_scaled[5]  
    nll_substrate = (1/(2 * sigma_substrate^2)) * sum(abs2, substrate - solution[1,:]) + 2*(length(timepoints)/2) * log(sigma_substrate)  
    nll_product = (1/(2 * sigma_product^2)) * sum(abs2, product - solution[2,:]) + 2*(length(timepoints)/2) * log(sigma_product)  
  
    return nll_product + nll_substrate  
end
```

1: Simulate ODE

3: Sum negative log likelihoods

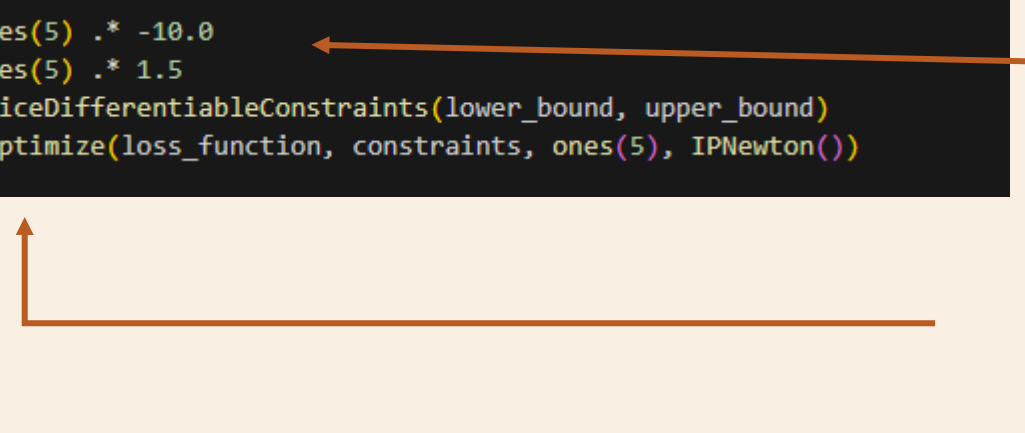
2: Calculate the negative log likelihood.

difference between the simulation and the data.

DATA ASSIMILATION

Step 2: Minimize the error/the negative log likelihood

```
lower_bound = ones(5) .* -10.0
upper_bound = ones(5) .* 1.5
constraints = TwiceDifferentiableConstraints(lower_bound, upper_bound)
opt_solution = optimize(loss_function, constraints, ones(5), IPNewton())
```



1: Specify upper and lower bounds for estimated parameters

2: direct the optimizer function to our defined loss function, specifying any constraints (i.e upper and lower bounds)

Note: While any optimizer could be used, we tend to favour use of a local solver with a multi-start approach, sampling initial parameter guesses using latin-hypercube sampling of the possible solution space rather than more common global solvers.

ODE Models in Julia

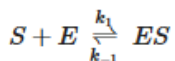


```
In [1]: using OrdinaryDiffEq, CairoMakie
using Random
using Optim
using ForwardDiff
using ADTypes: AutoForwardDiff
using StableRNGs, QuasiMonteCarlo
using Statistics: mean
using ProgressMeter
```

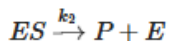
Example model: an Enzyme-catalyzed reaction

In this example, we will model a simple enzyme-catalyzed reaction using ordinary differential equations (ODEs). We have a substrate (S), an enzyme (E), and a product (P). The reactions are as follows:

1. The substrate binds to the enzyme to form a complex (ES).



2. The complex then converts to the product and releases the enzyme.



Using mass-action kinetics, we can represent the dynamics of the system with the following ODEs:

$$\begin{aligned}\frac{d[S]}{dt} &= -k_1[S][E] + k_{-1}[ES] \\ \frac{d[E]}{dt} &= -k_1[S][E] + (k_{-1} + k_2)[ES] \\ \frac{d[ES]}{dt} &= k_1[S][E] - (k_{-1} + k_2)[ES] \\ \frac{d[P]}{dt} &= k_2[ES]\end{aligned}$$

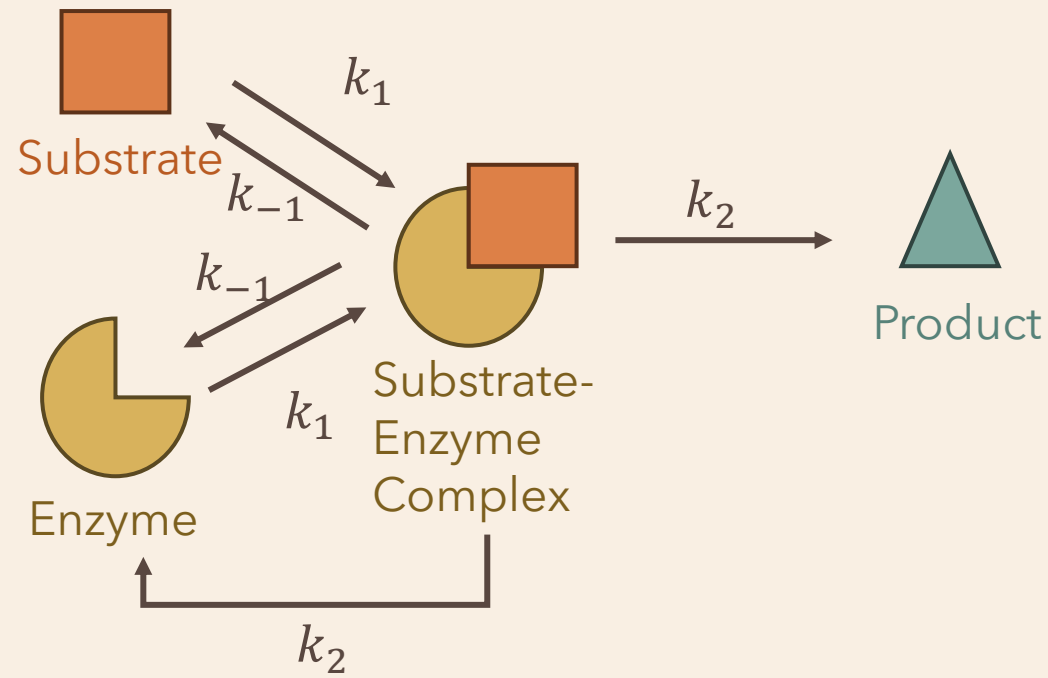
We can implement this model in Julia using the `OrdinaryDiffEq` package

We first implement the set of equations as a function with the signature `f(du, u, p, t)`, where `u` is the vector of state variables, `p` is the vector of parameters, and `t` is time. The function will compute the derivatives of the state variables and store them in `du`.

```
In [2]: function enzyme_reaction!(du, u, p, t)
        k1, kn1, k2 = p
```

MODEL SIMULATION

- Model of enzymatic catalysis of a reaction



Model Enzyme Catalysis

$$\frac{d[S]}{dt} = -k_1[S][E] + k_{-1}[ES]$$

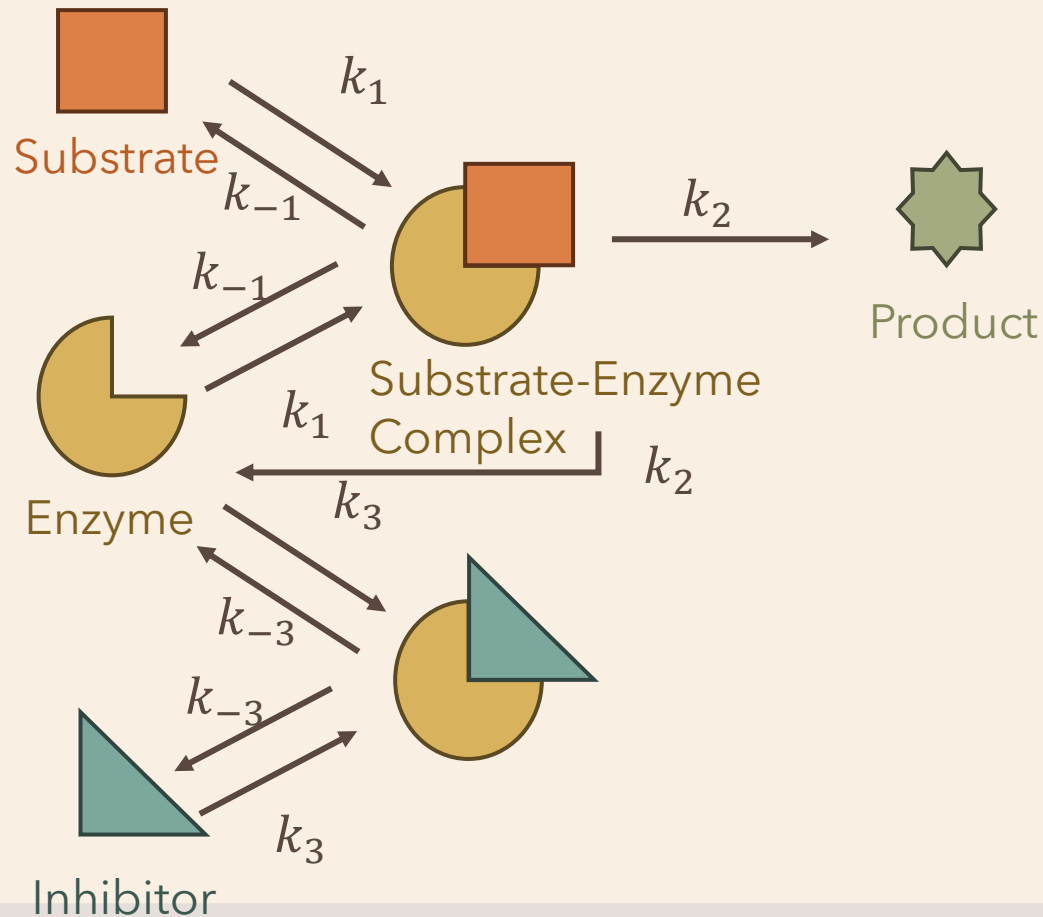
$$\frac{d[E]}{dt} = -k_1[S][E] + (k_{-1} + k_2)[ES]$$

$$\frac{d[ES]}{dt} = k_1[S][E] - (k_{-1} + k_2)[ES]$$

$$\frac{d[P]}{dt} = k_2[ES]$$

MODEL SIMULATION

- Model of enzymatic catalysis of a reaction with competitive inhibition



Model Enzyme Catalysis

$$\frac{d[S]}{dt} = -k_1[S][E] + k_{-1}[ES]$$

$$\frac{d[E]}{dt} = -k_1[S][E] + (k_{-1} + k_2)[ES] - k_3[I][E] + k_{-3}[EI]$$

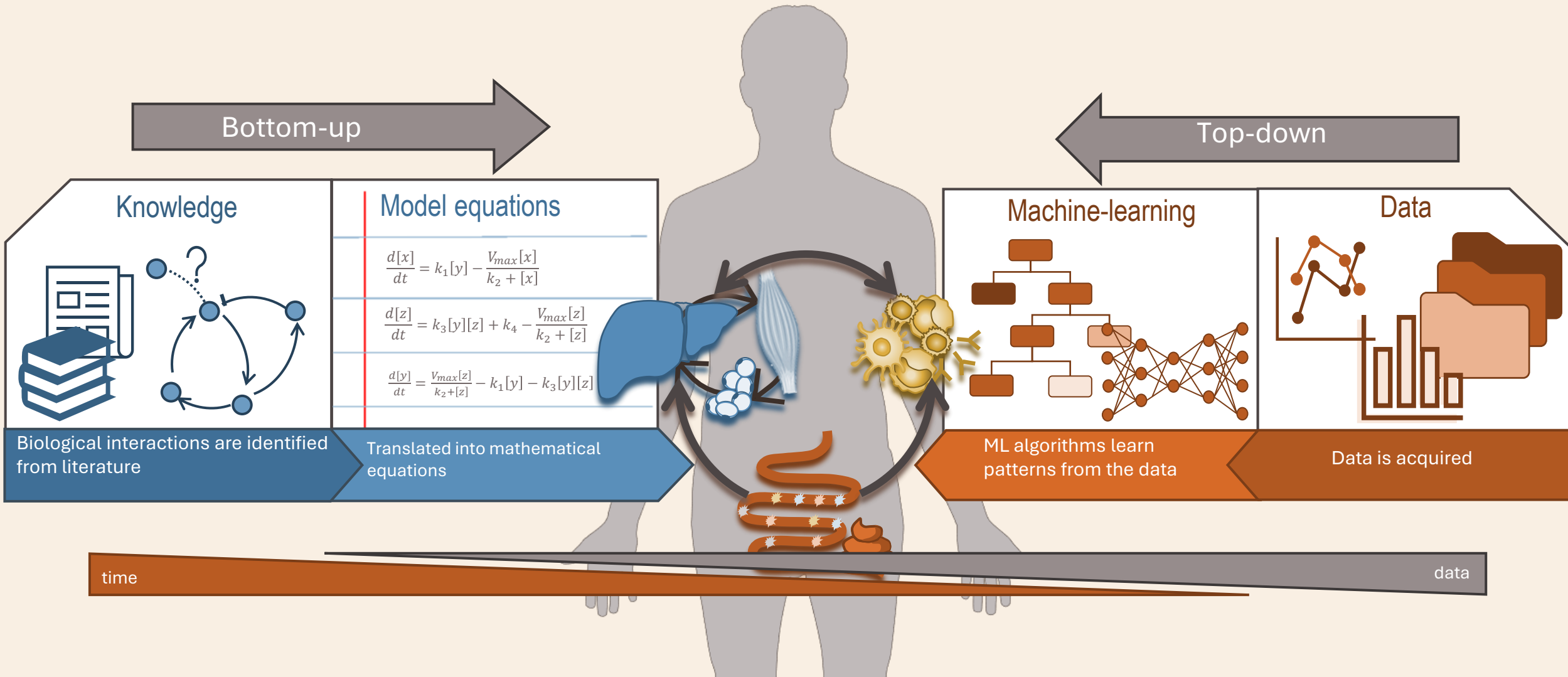
$$\frac{d[ES]}{dt} = k_1[S][E] - (k_{-1} + k_2)[ES]$$

$$\frac{d[P]}{dt} = k_2[ES]$$

$$\frac{d[I]}{dt} = -k_3[I][E] + k_{-3}[EI]$$

$$\frac{d[EI]}{dt} = k_3[I][E] - k_{-3}[EI]$$

TOP-DOWN V BOTTOM-UP



KNOWLEDGE versus DATA

Bottom-up

Encode existing knowledge into a system.

- ✓ Parameters are interpretable.
- ✓ Can work with little data.
- ✓ Can be used for hypothesis testing
- ✗ Construction and validation can be labor intensive.
- ✗ Scope is often limited.

Top-down

Identify patterns from the data itself.

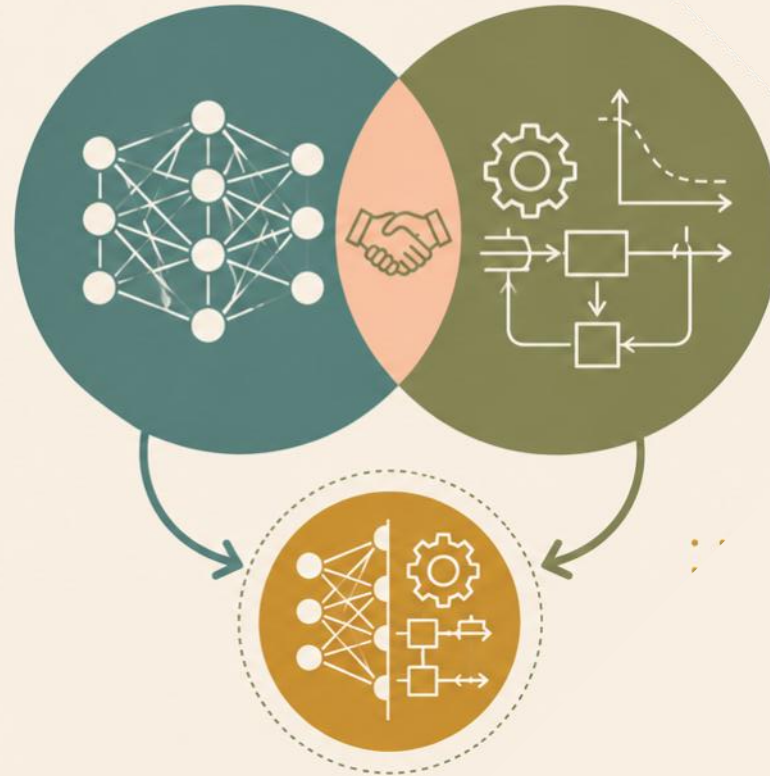
- ✓ Identify novel interactions.
- ✓ Can be quite fast.
- ✗ Black box - uninterpretable parameters.
- ✗ Requires large number of learning examples.
- ✗ Cannot be reliably predict outside of distribution.

Hybrid-modelling approaches



SCIENTIFIC MACHINE LEARNING

Blend data driven AI with physics-based mathematical models.



HYBRID-MODELS



Speed up model development using data



Allow model to be trained using fewer learning examples



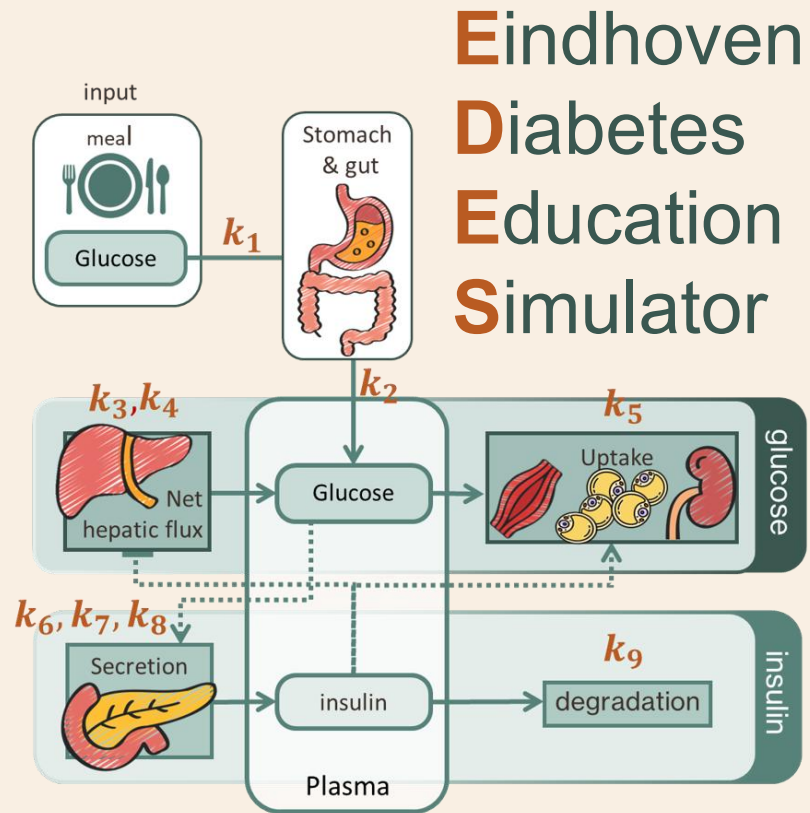
Facilitate multi-modal data integrations



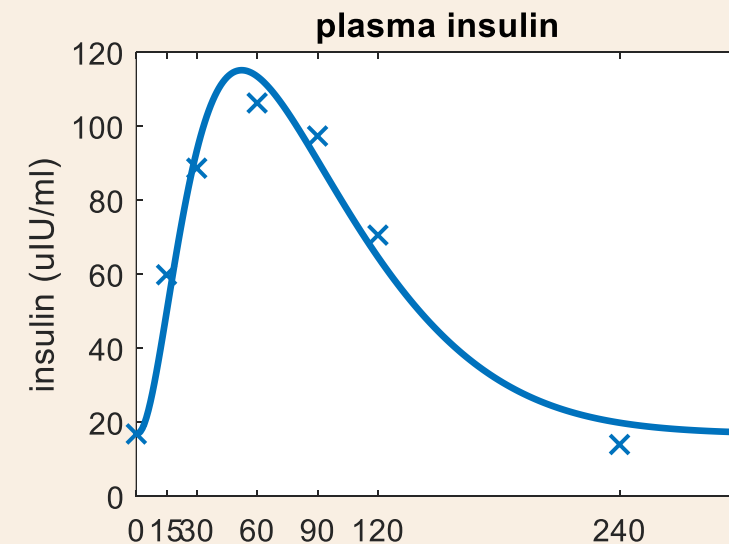
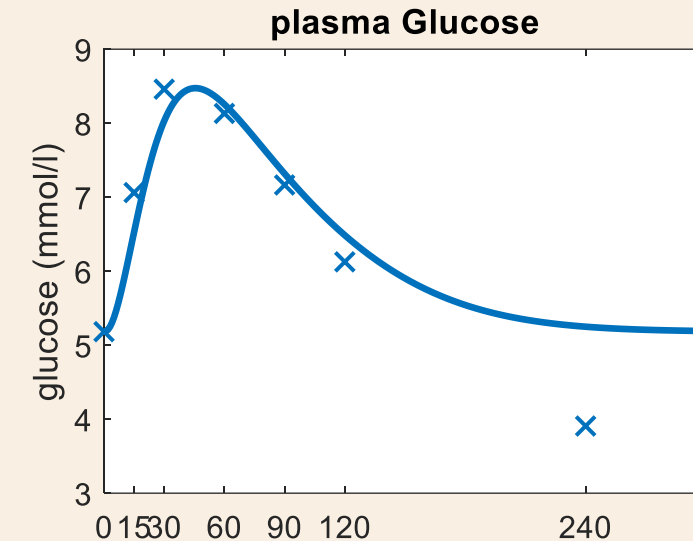
Greater model interpretability – grey box



IMPROVE/EXTEND MODELS



Meal 1
75g glucose



IMPROVE/EXTEND MODELS

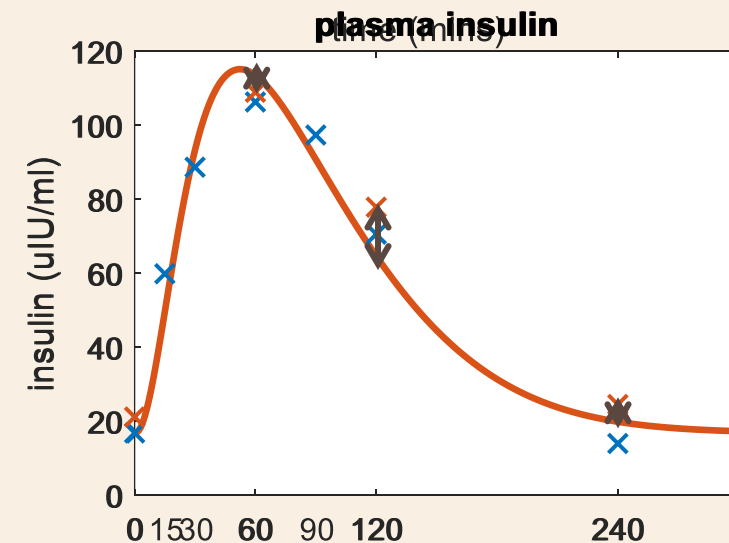
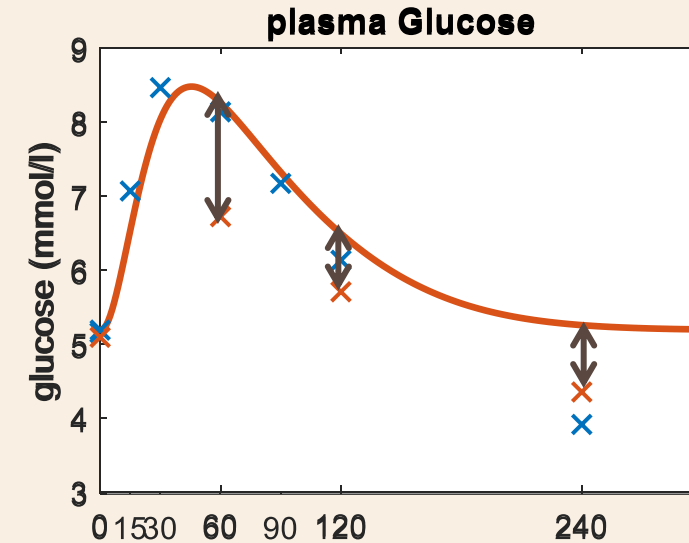
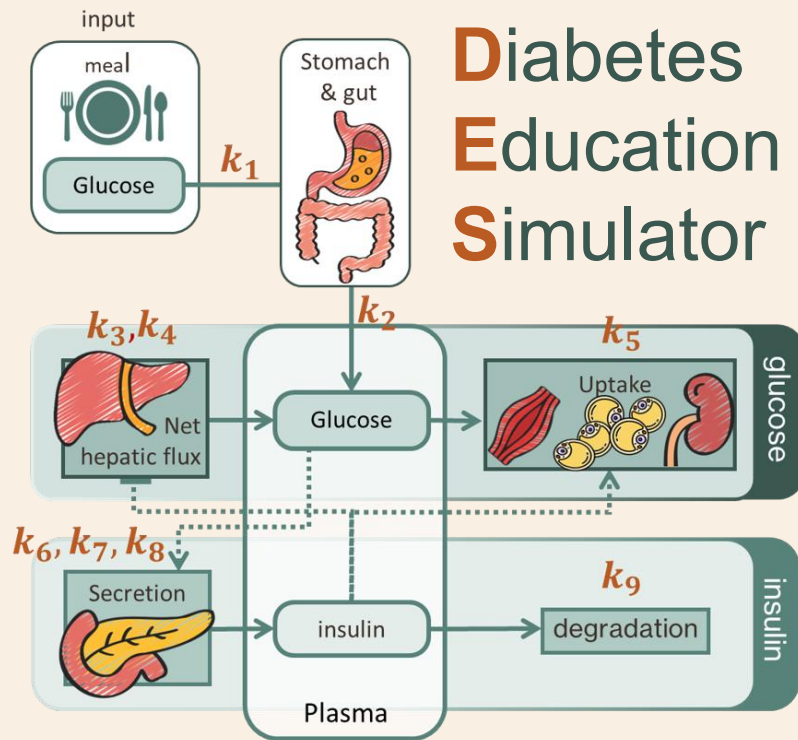
Eindhoven Diabetes Education Simulator



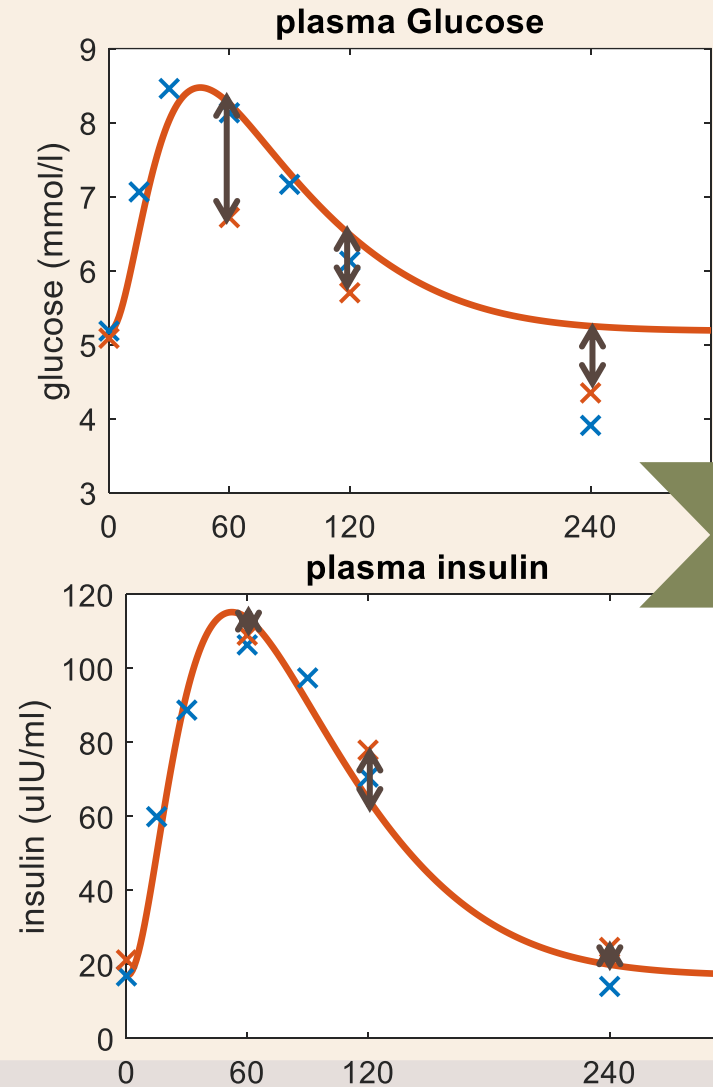
Meal 1
75g glucose



Meal 2
75g glucose
60g fat
20g protein



HYBRID MODELS IN SERIES

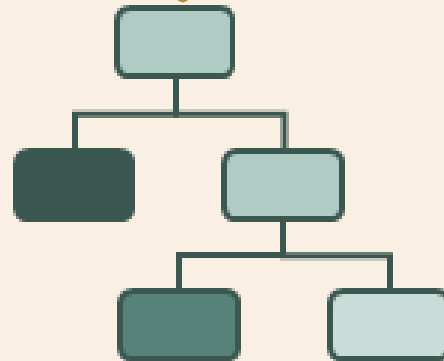


XGBoost/
other ML Model



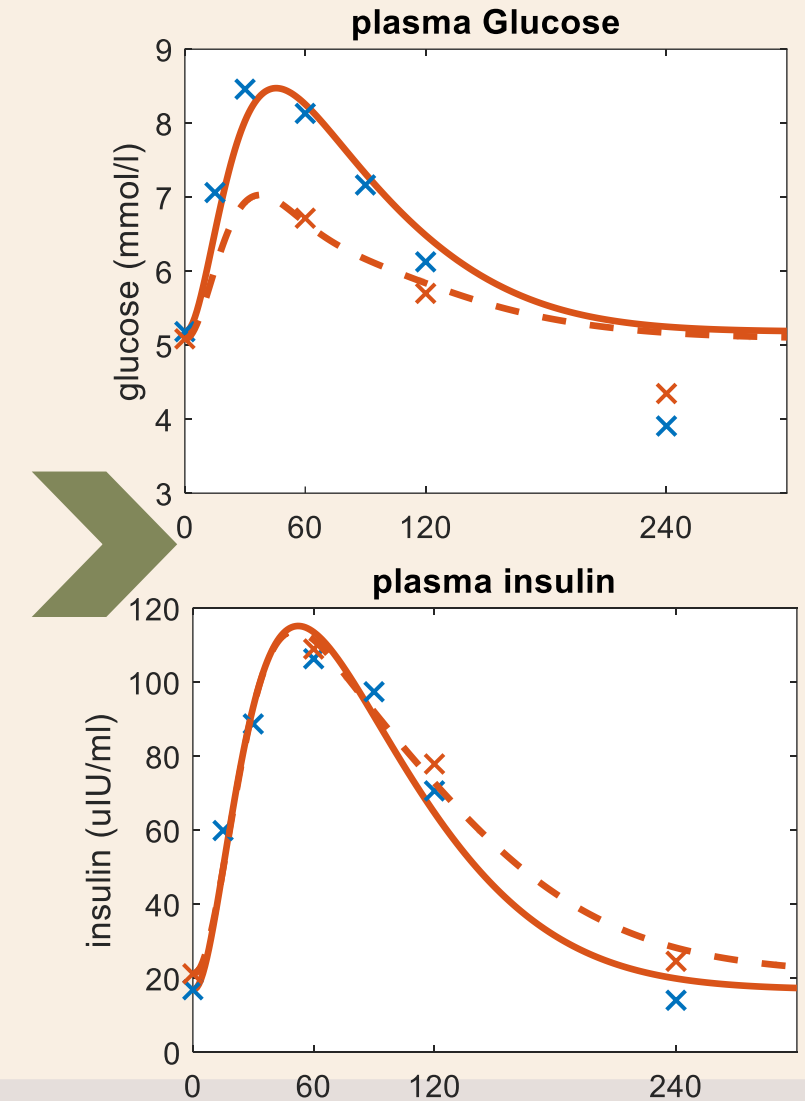
Input Data

- Meal Composition
- Age
- Sex
- Time of day....



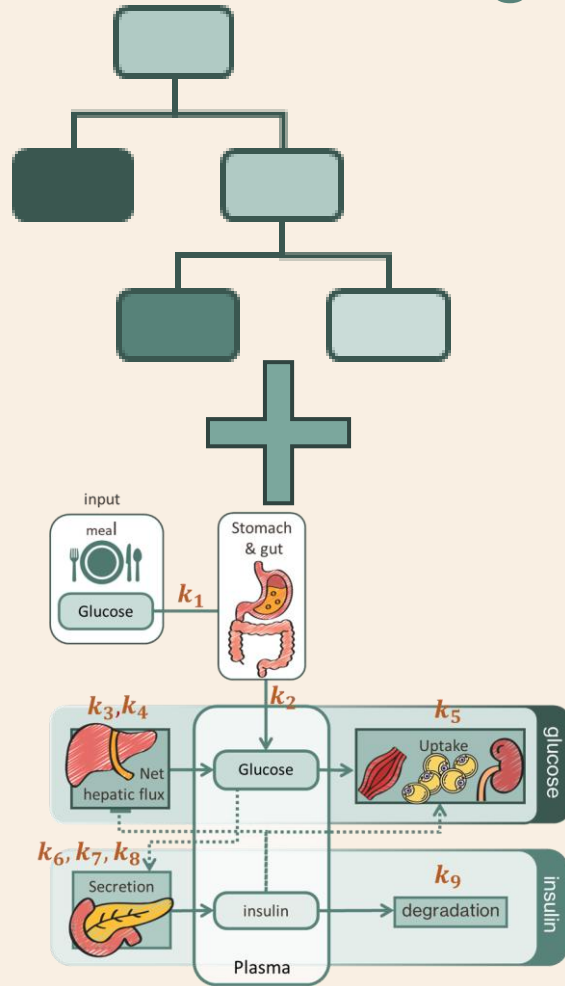
Target variable

- Residual at single time point.
- Sum of residuals

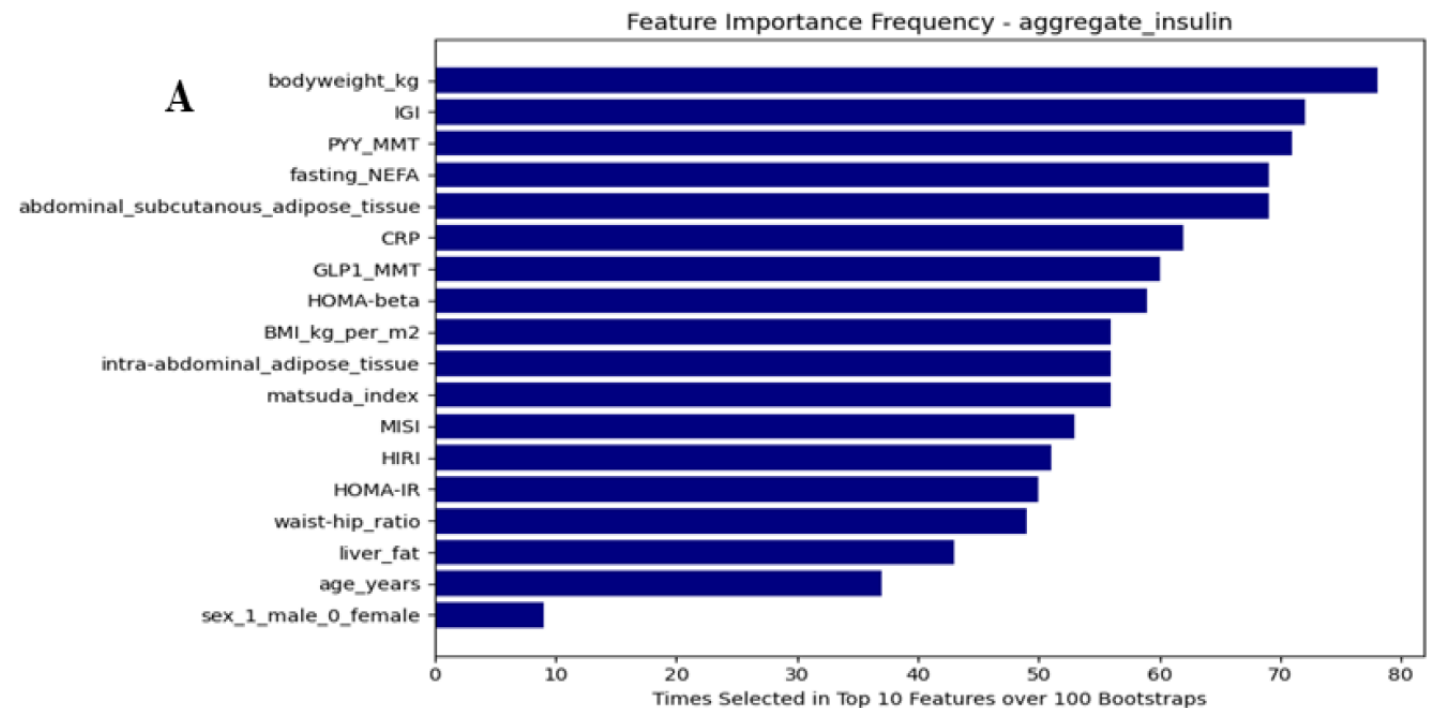


HYBRID MODELS IN SERIES

Machine learning



Feature Importance



Hybrid Models in Series

In this example, we'll explore **serial hybrid modeling**, where we combine a mechanistic model with a machine learning model to correct for missing mechanisms.

The Idea

Sometimes our mechanistic models are incomplete - we may know some of the mechanisms, but not all of it. In such cases, we can:

1. Build a mechanistic model with the physics we *do* know
2. Use it to make predictions and observe systematic errors (residuals)
3. Train a machine learning model to predict those residuals from features
4. Combine both: **$y_{\text{hybrid}} = y_{\text{mechanistic}} + y_{\text{ML_correction}}$**

This approach lets us leverage both physical knowledge and data-driven learning.

Our Example: The Missing Inhibitor

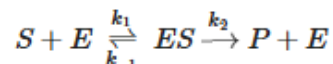
- **True system:** Has inhibitor binding ($I + E \rightleftharpoons EI$) that slows down the reaction
- **Mechanistic model:** Ignores the inhibitor (simpler, incomplete model)
- **ML correction:** Learn residuals as a function of inhibitor concentration and system state
- **Test generalization:** Train on some inhibitor concentrations, test on others

In [5]:

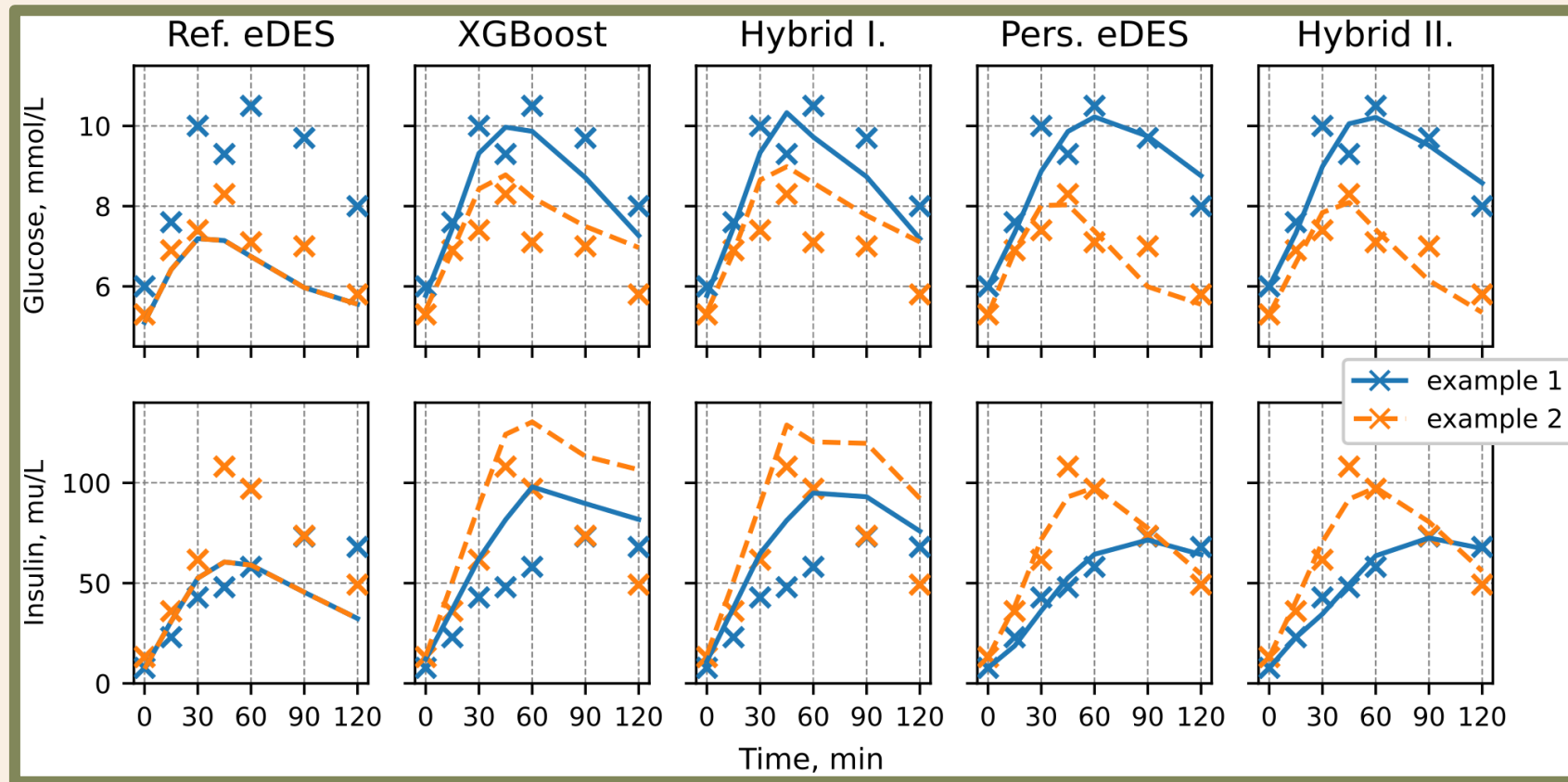
```
using OrdinaryDiffEq, CairoMakie
using Random
using Statistics: mean
using DataFrames
using XGBoost
```

The "True" System: Enzyme Kinetics with Inhibitor

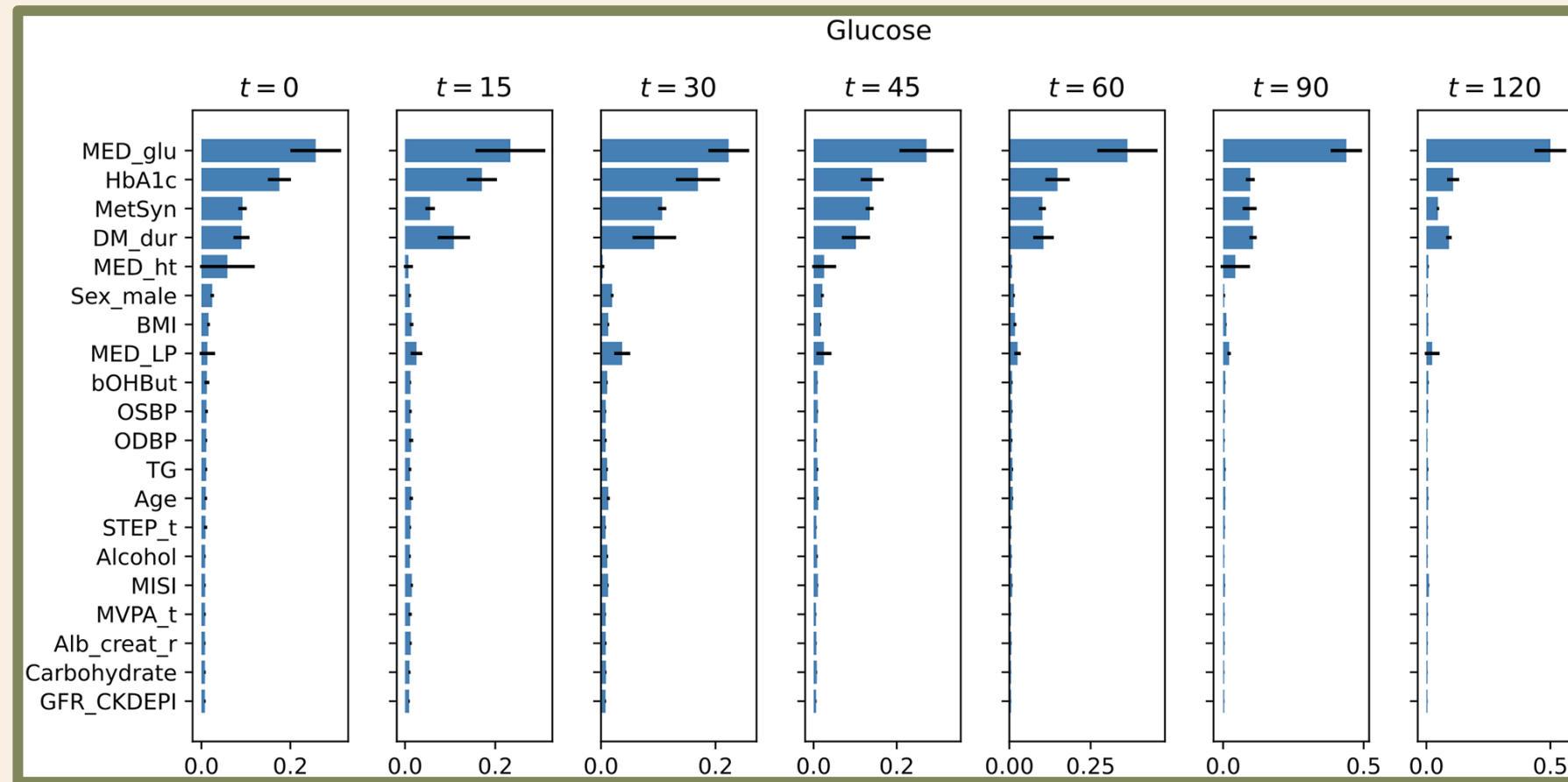
Recall from Tutorial 1 the basic enzyme-substrate reaction:



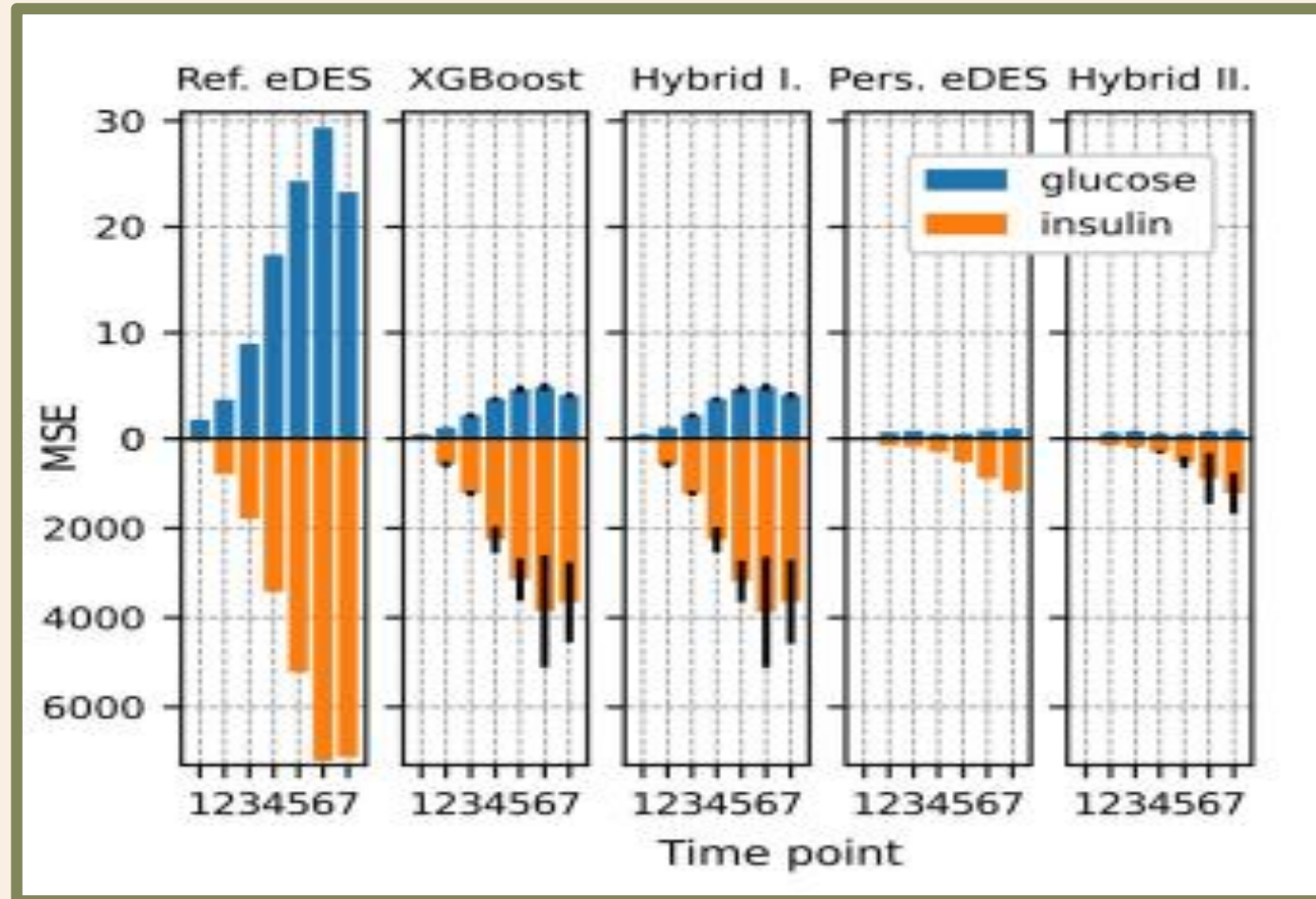
HYBRID MODELS IN SERIES



HYBRID MODELS IN SERIES



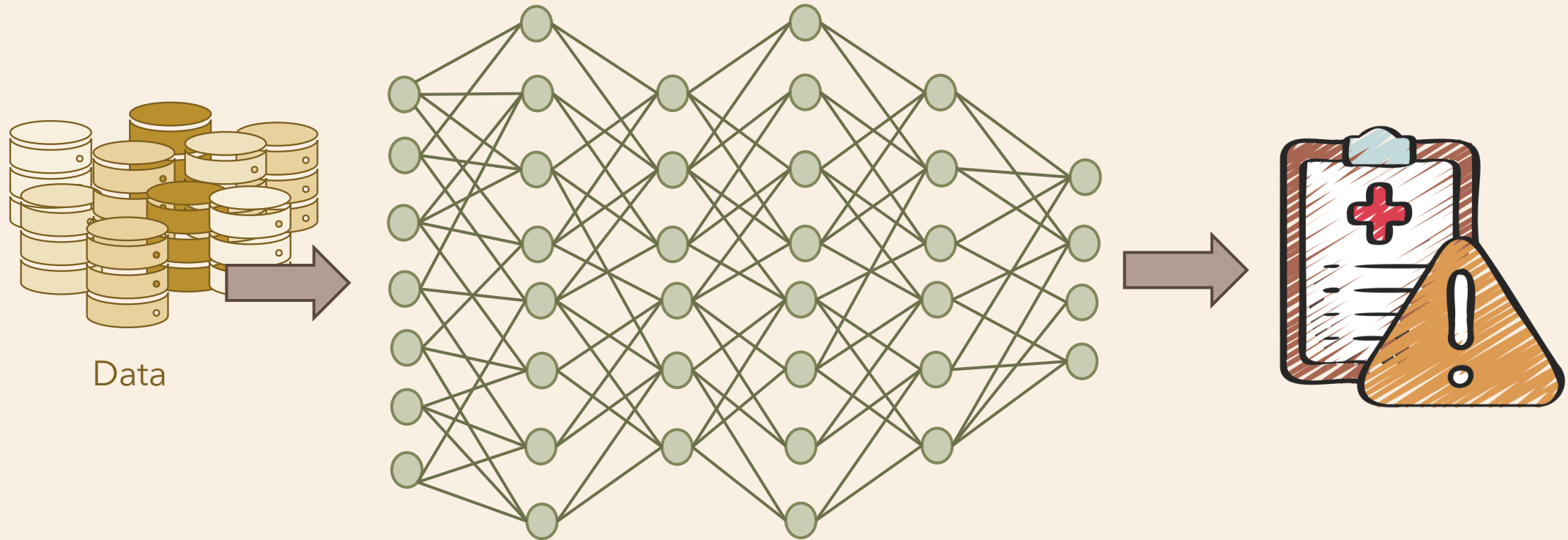
HYBRID MODELS IN SERIES



HYBRID MODELS IN SERIES

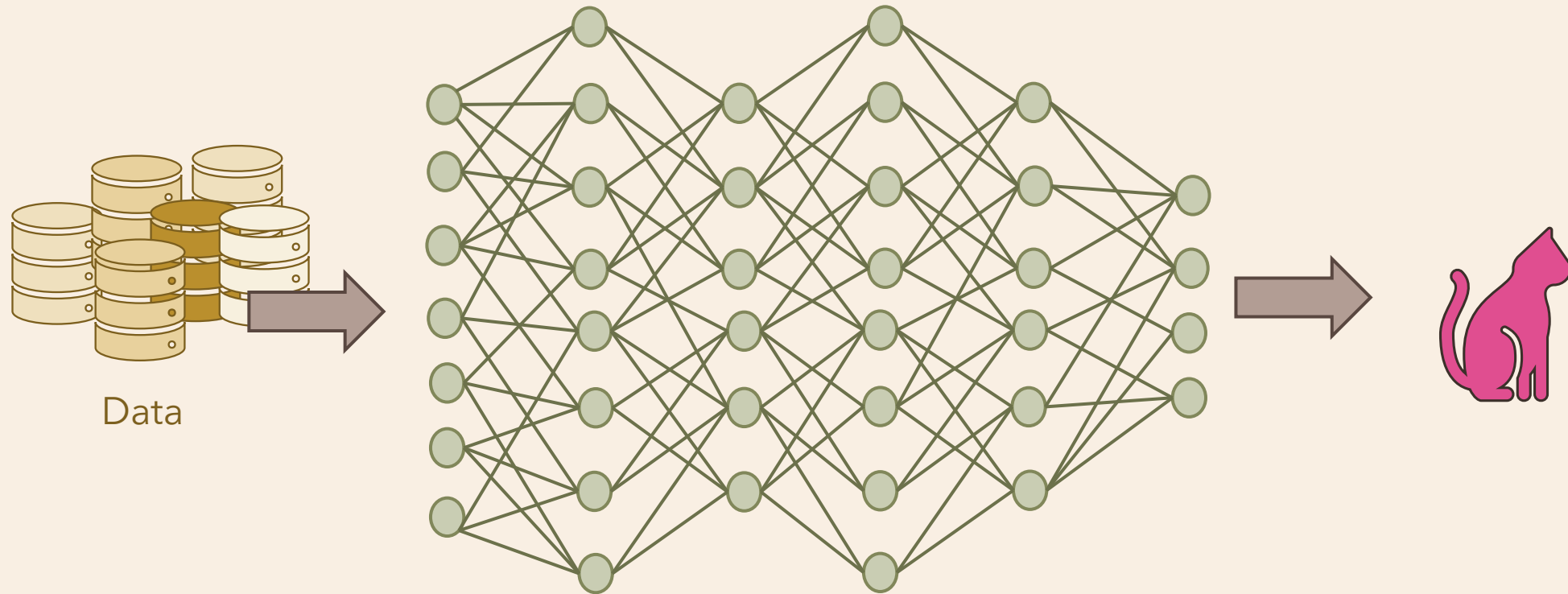
- ✓ Improve model fit using data.
- ✓ Integrate different data types/features not in model.
- ✓ Identify missing/influential features (depending on ML model)
- ✗ Assumes mechanistic models is correct.
- ✗ Still requires sufficient data to train ML model.

ARTIFICIAL NEURAL NETWORKS



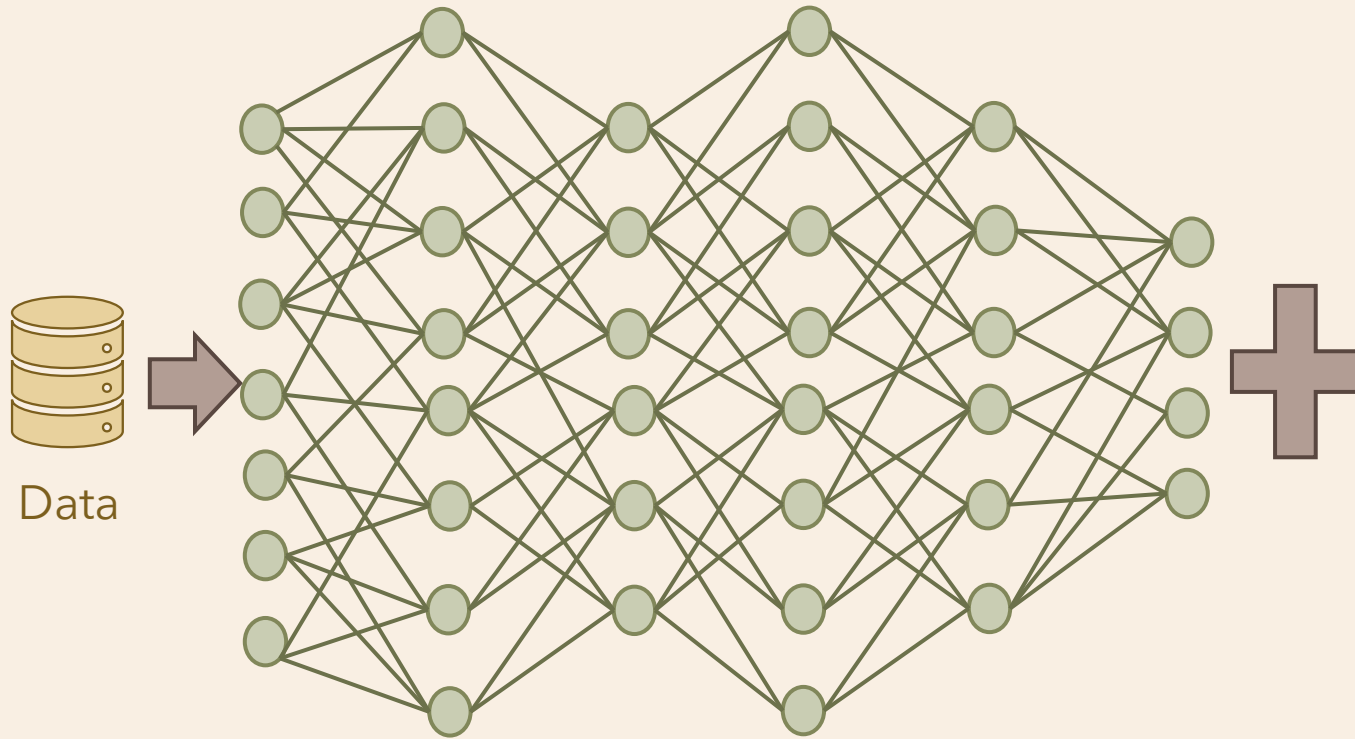
Insufficient data to reliably train the network

ARTIFICIAL NEURAL NETWORKS

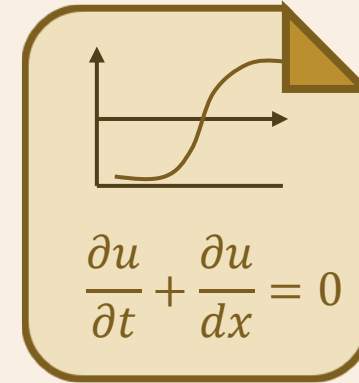


Model makes physically impossible predictions

PHYSIC INFORMED NEURAL NETWORKS



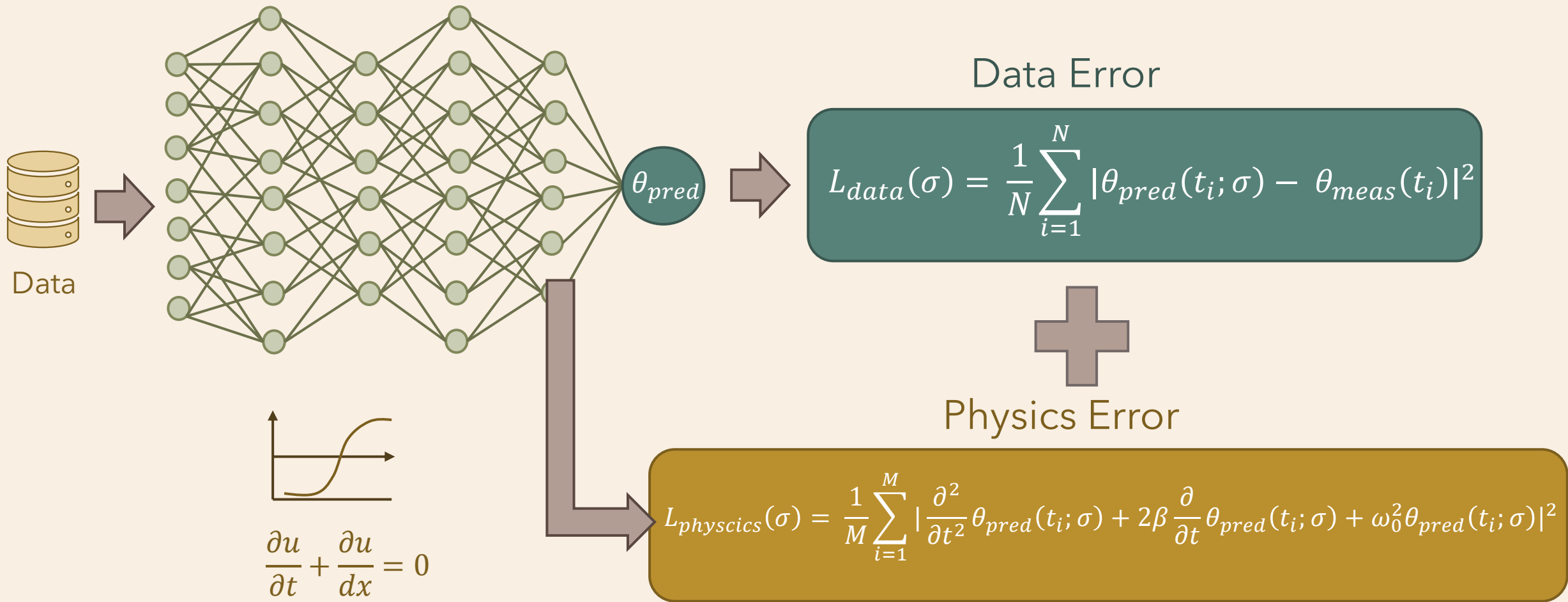
Neural Network/
data driven model



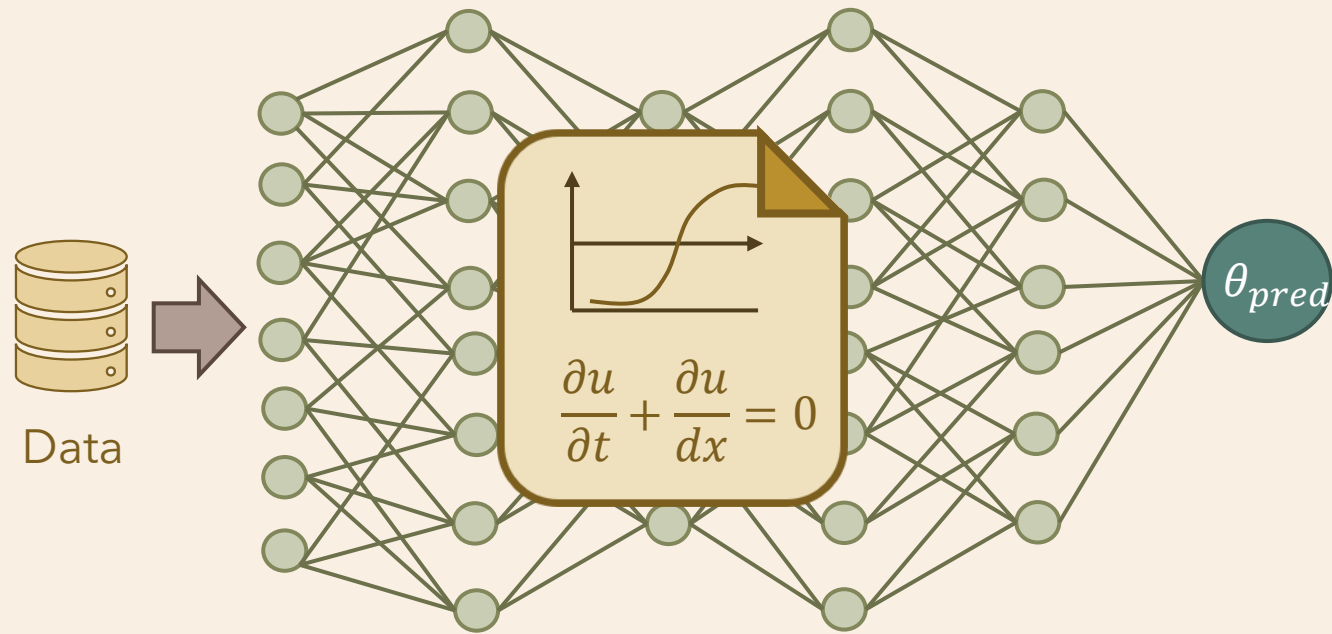
$$\begin{aligned}\theta''(t) + 2\beta\theta'(t) + \omega_0^2\theta(t) &= 0 \\ \theta(t_0) &= 0 \\ \theta'(t_0) &= 0\end{aligned}$$

Physics/
Biological knowledge

PHYSIC INFORMED NEURAL NETWORKS

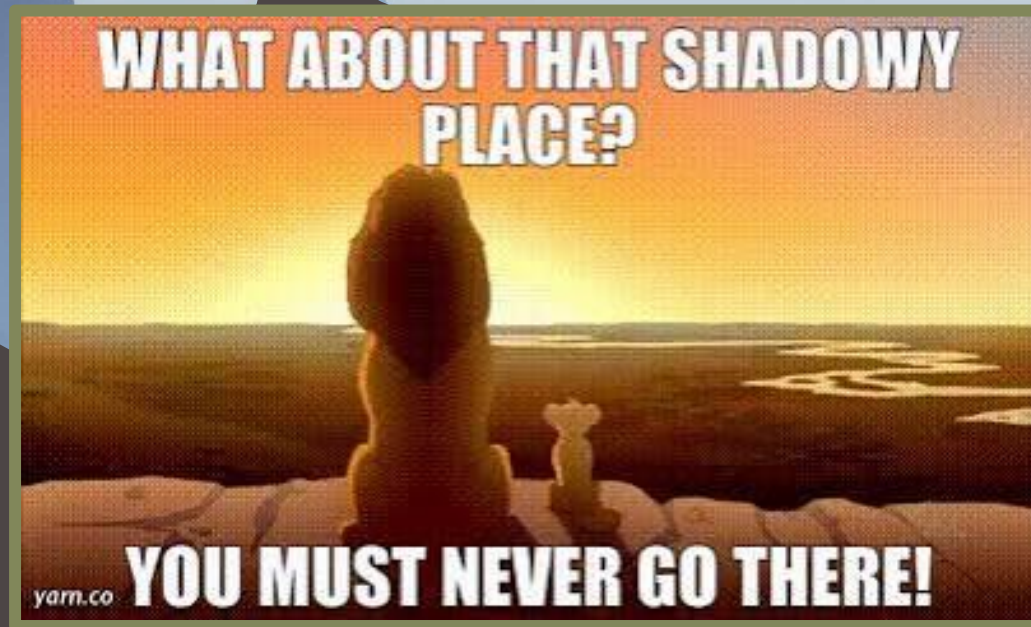


PHYSIC INFORMED NEURAL NETWORKS



PINN Error

$$L_{PINN} = L_{data} + L_{physics}$$

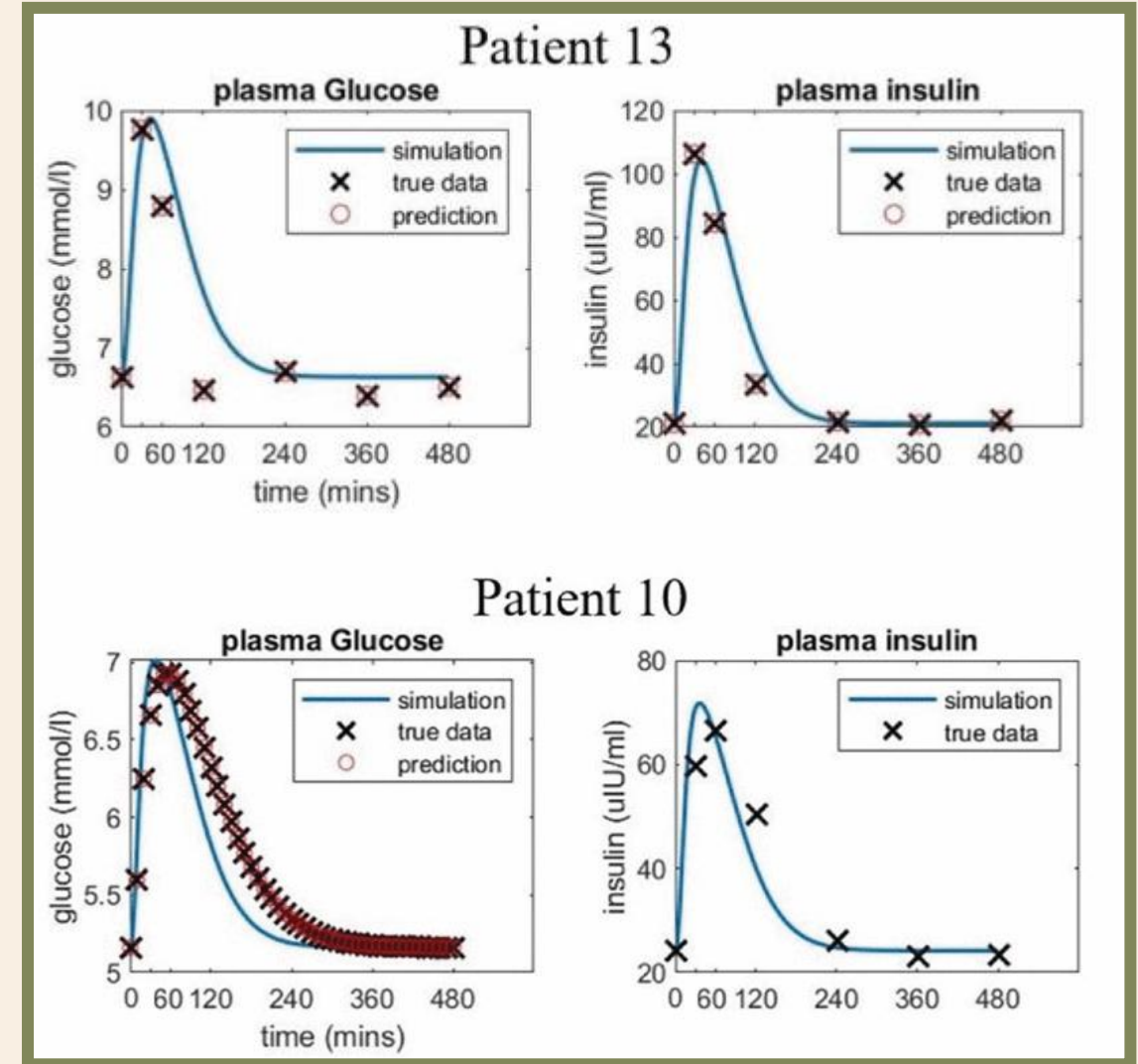
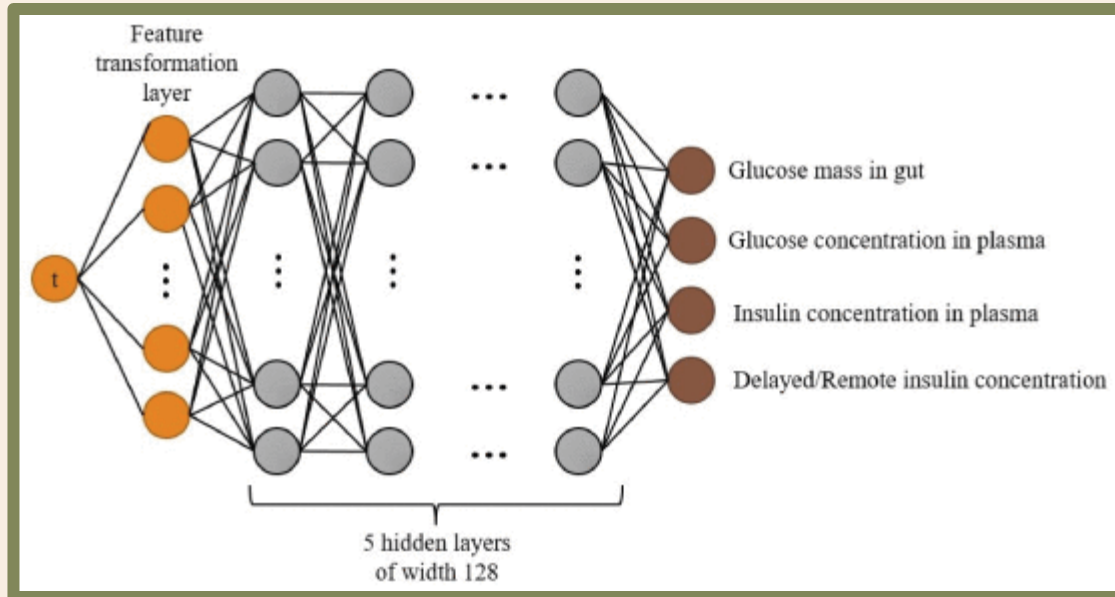


$$L_{PINN} = L_{data} + L_{physics}$$

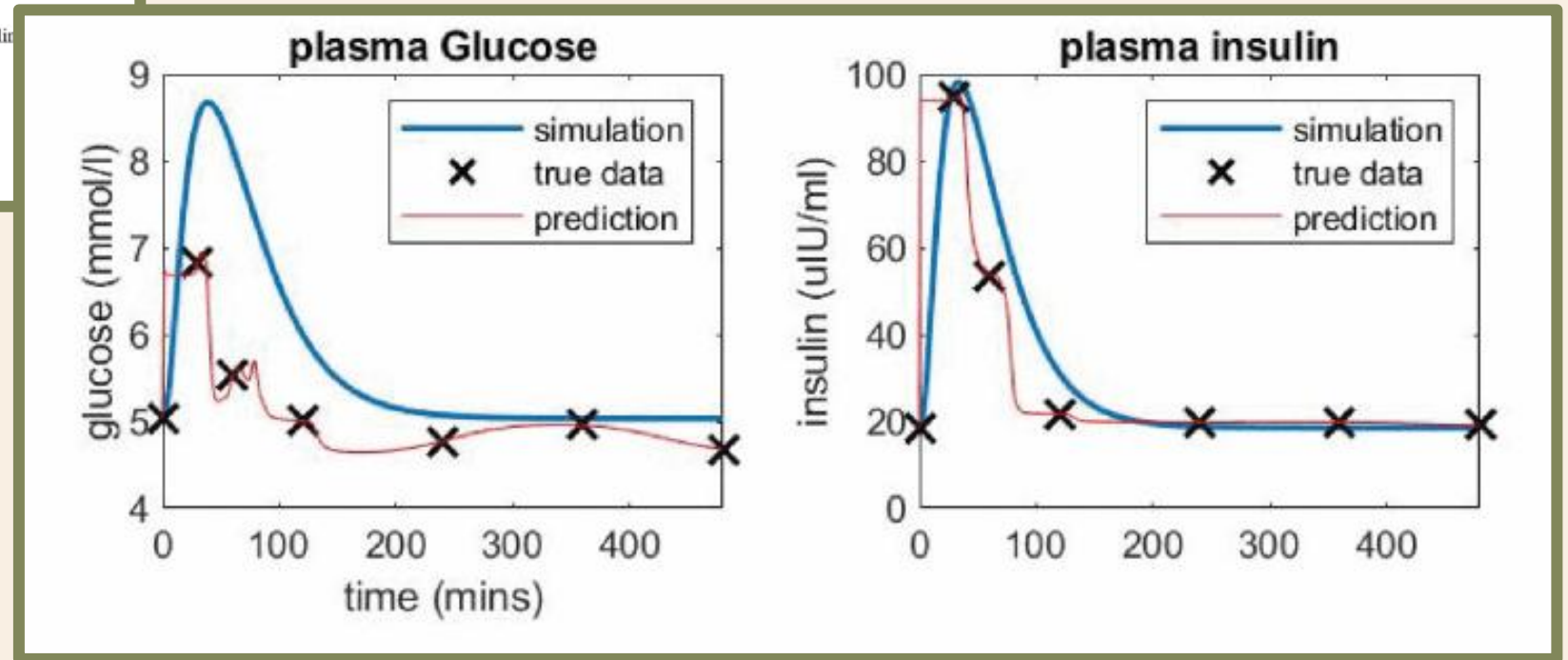
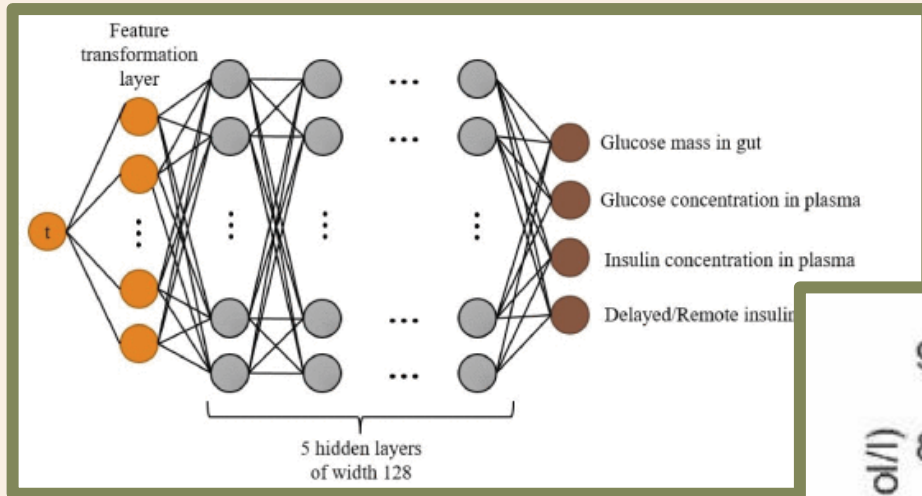
PHYSIC INFORMED NEURAL NETWORKS

- Prevent the NN model learning **physically (biologically) impossible behaviour**.
- Typically, can be trained with **fewer learning examples**.
- Surrogate modelling – **Speeding up computation**
 - Solving complex systems of ODEs or PDEs
 - Estimating unknown parameters, material properties, constant from sparse data.

SYSTEMS BIOLOGY INFORMED NEURAL NETWORKS



SYSTEMS BIOLOGY INFORMED NEURAL NETWORKS



Physics-informed neural networks

Another variant of hybrid models are physics-informed neural networks (PINNs). In a PINN, neural networks are used to learn both the physical parameters of a system, as well as the solution to the system itself.

A common example for PINNs, is the damped harmonic oscillator system:

$$\frac{d^2x}{dt^2} = -\frac{\mu}{m} \frac{dx}{dt} - \frac{k}{m}x$$

Two identifiable quantities in this system are $\delta = \frac{\mu}{2m}$ and the natural frequency $\omega_n = \sqrt{\frac{k}{m}}$.

We will focus here on the underdamped case, where $\delta < \omega_n$.

Setting $\frac{dx}{dt} = v$, we can implement the system of ODEs:

In [34]:

```
function damped_harmonic_oscillator!(du, u, p, t)
    x, v = u
    μ, m, k = p

    du[1] = v
    du[2] = - (μ / m) * v - (k / m) * x
end
```

damped_harmonic_oscillator! (generic function with 1 method)

Let's simulate some data

In [35]:

```
parameters = [2.0, 0.5, 20*20*0.5] # μ, m, k

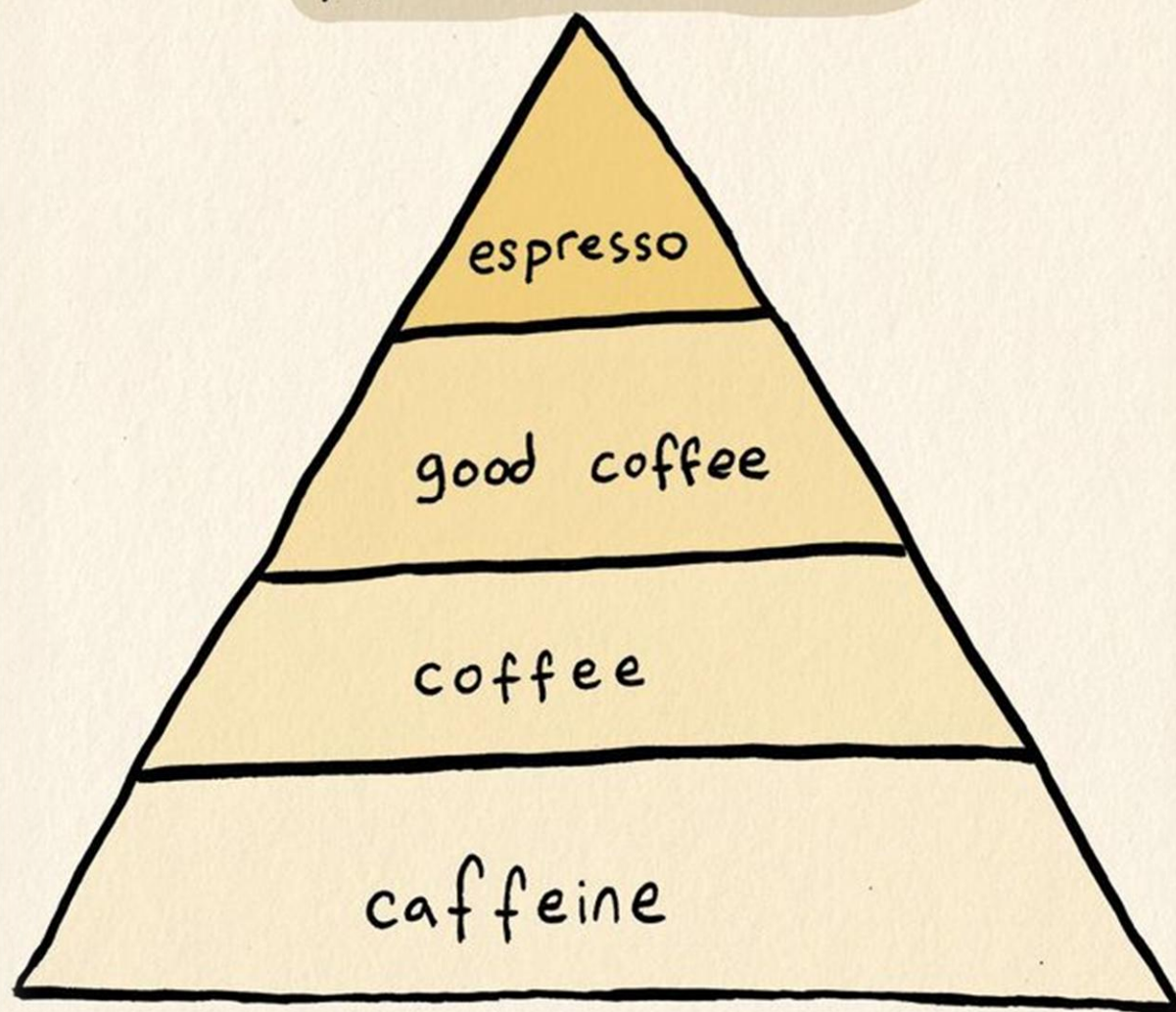
# Make sure we are in the underdamped regime (μ/(2m) < sqrt(k/m))
@assert parameters[1]/(2 * parameters[2]) < sqrt(parameters[3]/parameters[2]) "The system is overdamped. Please choose different parameters."
initial_conditions = [1.0, 0.0] # x₀, v₀

tspan = (0.0, 1.0)

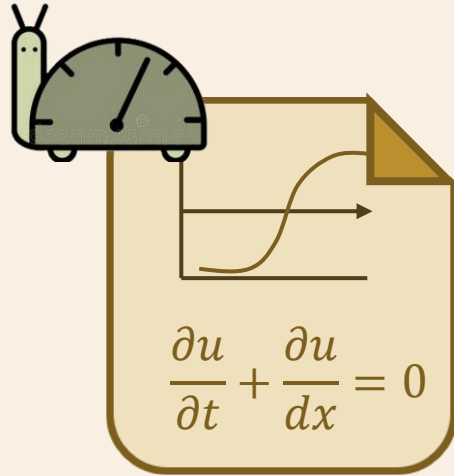
prob = ODEProblem(damped_harmonic_oscillator!, initial_conditions, tspan, parameters)
sol = solve(prob, Tsit5(), saveat=range(0,1,500))

x_data = sol.t[1:20:200]
y_data = sol[1, 1:20:200]
```


MASLOW'S HIERARCHY OF
NEEDS COFFEE



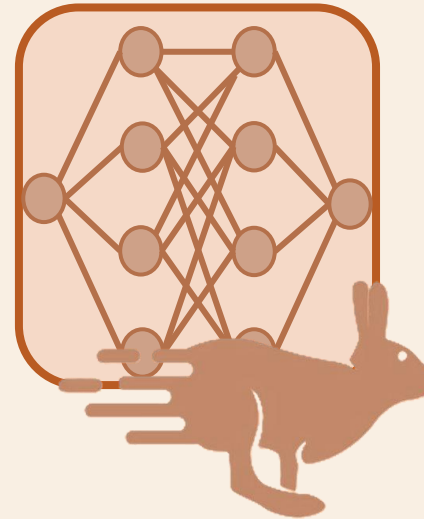
RECAP



Mechanistic Models

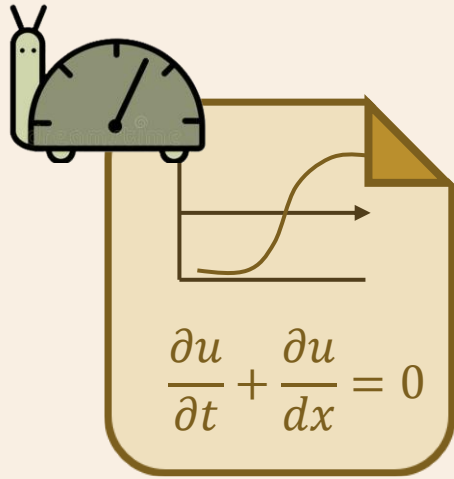
- Slow to build
- Rigid
- Little to no data

Machine Learning



- Relatively fast
- Highly plastic
- Data hungry

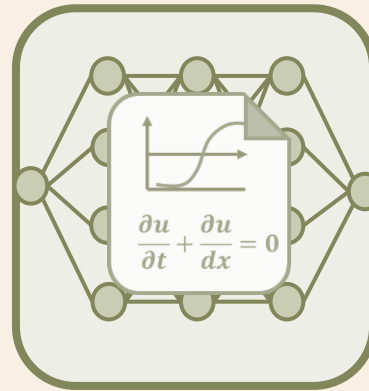
RECAP



Mechanistic Models

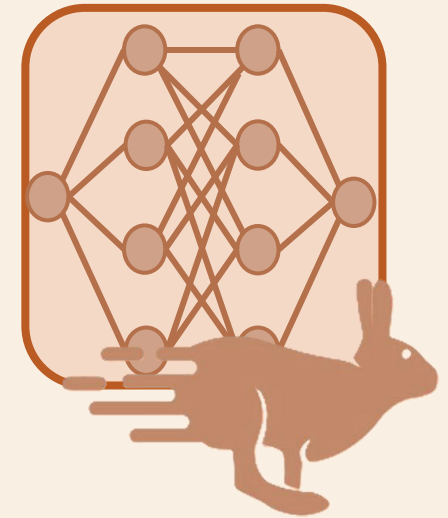
- Slow to build
- Rigid
- Little to no data

PINN



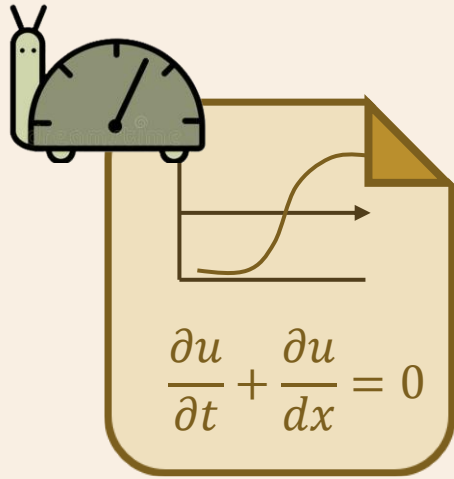
- Speeds up computation
- plastic
- Requires (a lot) of high temporal data

Machine Learning



- Relatively fast
- Highly plastic
- Data hungry

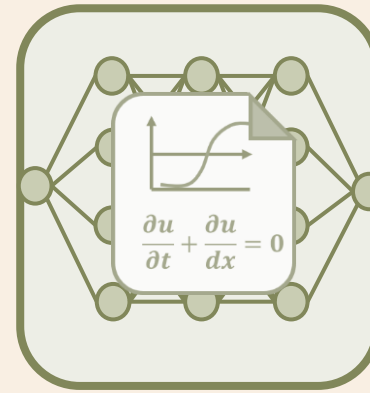
RECAP



Mechanistic Models

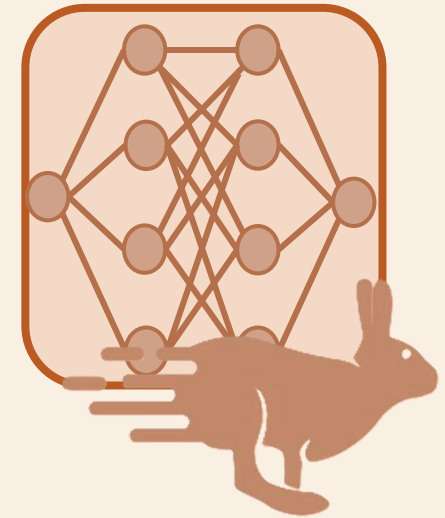
- Slow to build
- Rigid
- Little to no data

PINN



- Speeds up computation
- plastic
- Requires (a lot) of high temporal data

Machine Learning



- Relatively fast
- Highly plastic
- Data hungry

PREDATOR-PREY MODEL

PREDATOR: 

Fox

PREY :



Rabbit

How do these populations change over time?

PREDATOR-PREY MODEL

How do these populations change over time?

Model changes in the rabbit population in isolation

PREY : $\frac{d \text{ }{dt} = ?$

PREDATOR-PREY MODEL

How do these populations change over time?

Model changes in the rabbit population in isolation

$$\text{PREY: } \frac{d \text{ }{dt} = B_R \text{ } - D_R \text{ $$

PREDATOR-PREY MODEL

How do these populations change over time?

What effect do the foxes have on the rabbit population?

$$\text{PREY: } \frac{d \text{ }{dt} = B_R \text{ } - D_R \left(\text{} - H \text{  } \right)$$

assume rabbits only die from hunting

PREDATOR-PREY MODEL

How do these populations change over time?

Model changes in the rabbit population in isolation

$$\text{PREY: } \frac{d \text{ }{dt} = B_R \text{ } - D_R \text{ $$

$$\text{PREDATOR: } \frac{d \text{ }{dt} = ?$$

PREDATOR-PREY MODEL

How do these populations change over time?

Model changes in the rabbit population in isolation

$$\text{PREY: } \frac{d \text{  } }{dt} = B_R \text{  } - H_R \text{   }$$

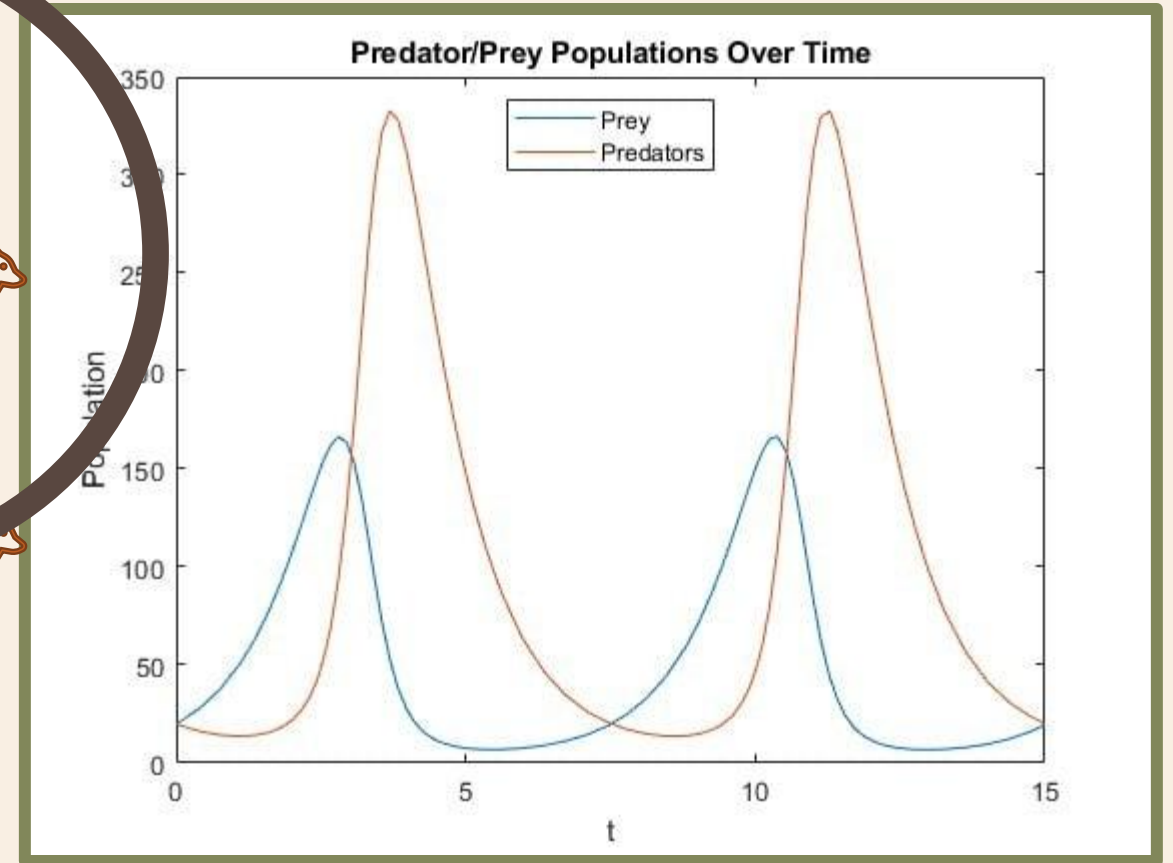
$$\text{PREDATOR: } \frac{d \text{  } }{dt} = H_F \text{   } - D_F \text{  }$$

PREDATOR-PREY MODEL

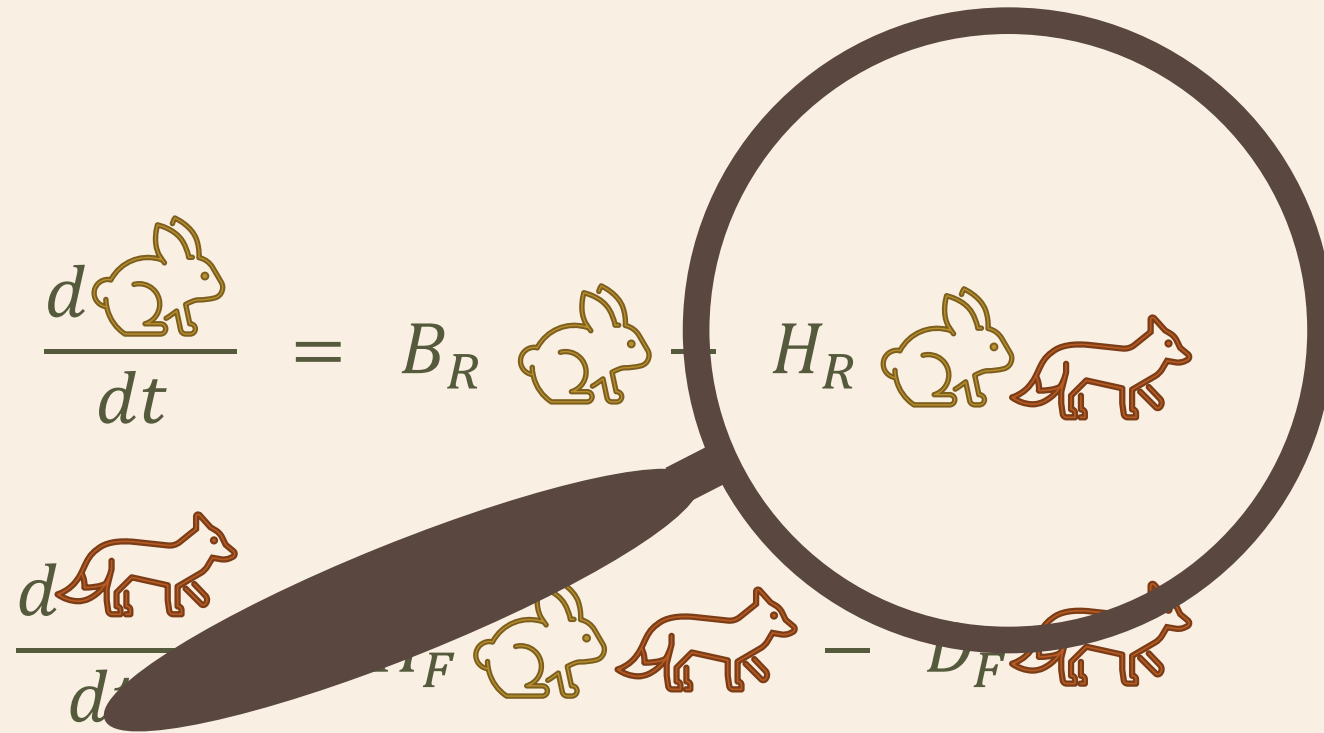
$$\frac{dR}{dt} = B_R R - H_R R F$$

$$\frac{dF}{dt} = H_F R F - D_F F$$

A magnifying glass is positioned over the term $H_R R F$ in the first equation, highlighting the interaction between rabbits and foxes.



PREDATOR-PREY MODEL



$$\frac{dR}{dt} = B_R R - H_R R F$$

$$\frac{dF}{dt} = A_F R F - D_F F$$

Many factor may impact the success of hunting

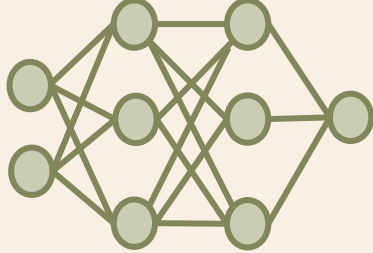
- Available space
- Tree coverage
- Health of the foxes/rabbits
- Hawks

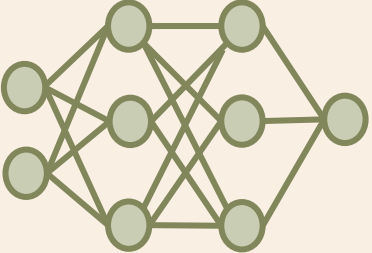
How can we account for these factors?

PREDITOR-PREY MODEL

How do these populations change over time?

Model changes in the rabbit population in isolation

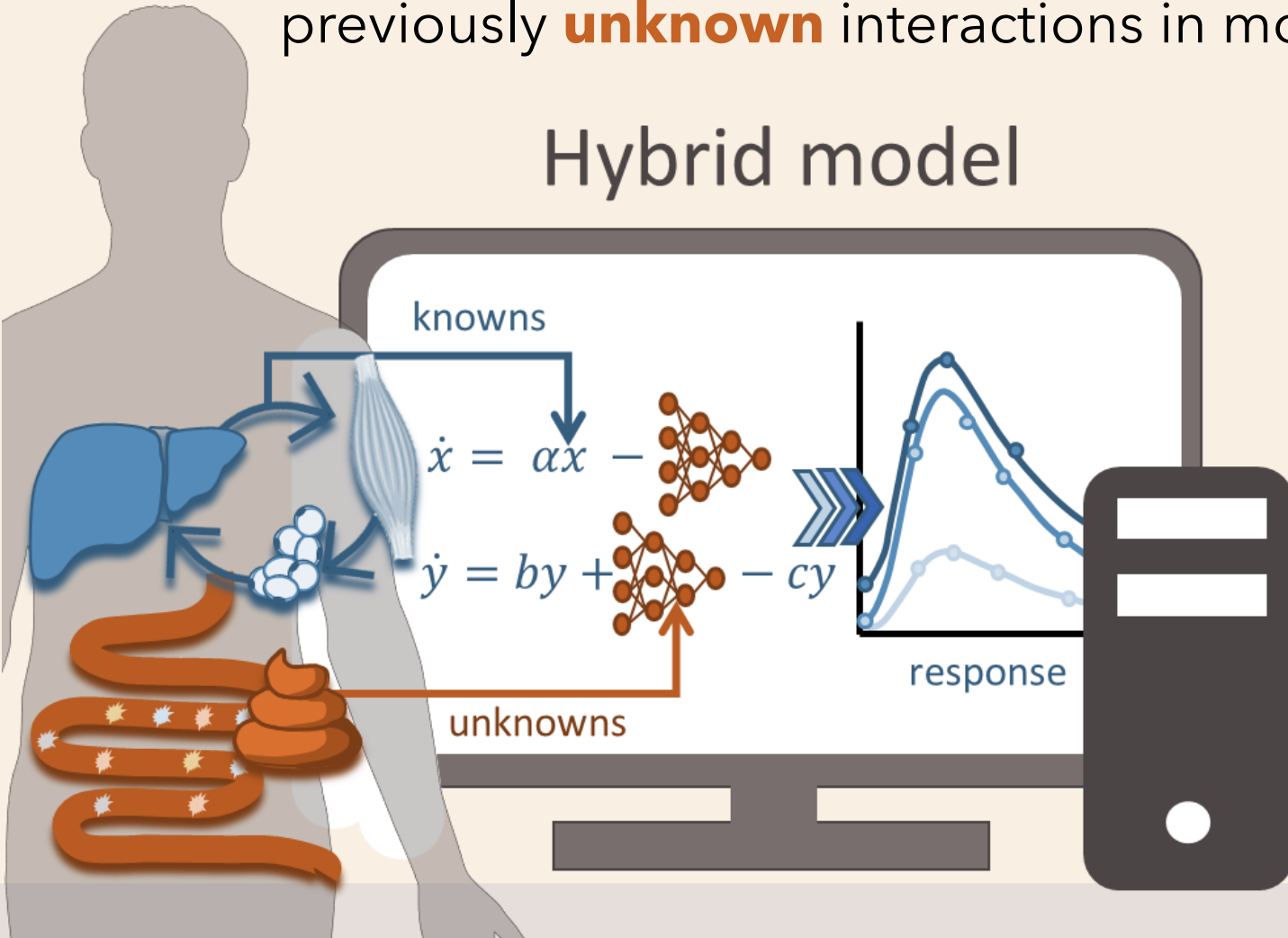
PREY : $\frac{d \text{ }{dt} = B_R \text{ } - \text{ $

PREDITOR : $\frac{d \text{ }{dt} = \text{ } - D_F \text{ $

UNIVERSAL DIFFERENTIAL EQUATIONS

Combining **existing knowledge** with **artificial intelligence** to learn previously **unknown** interactions in model system directly from data

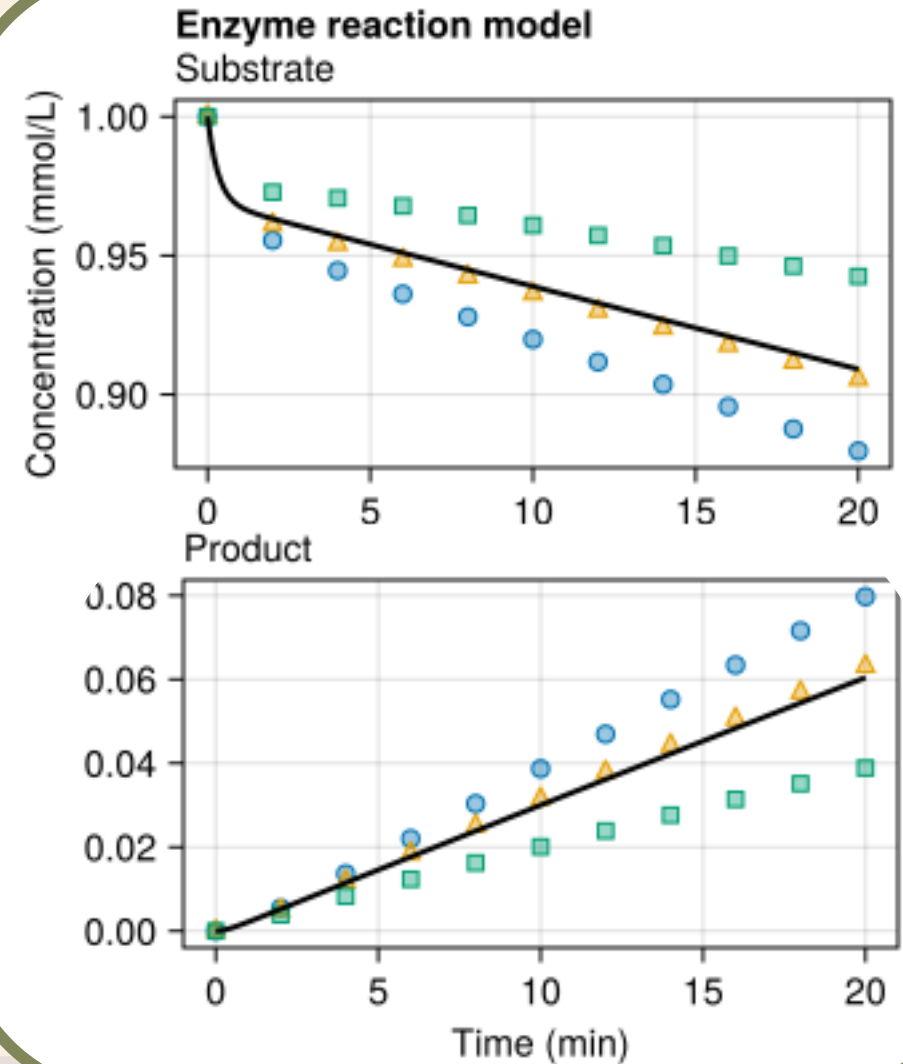
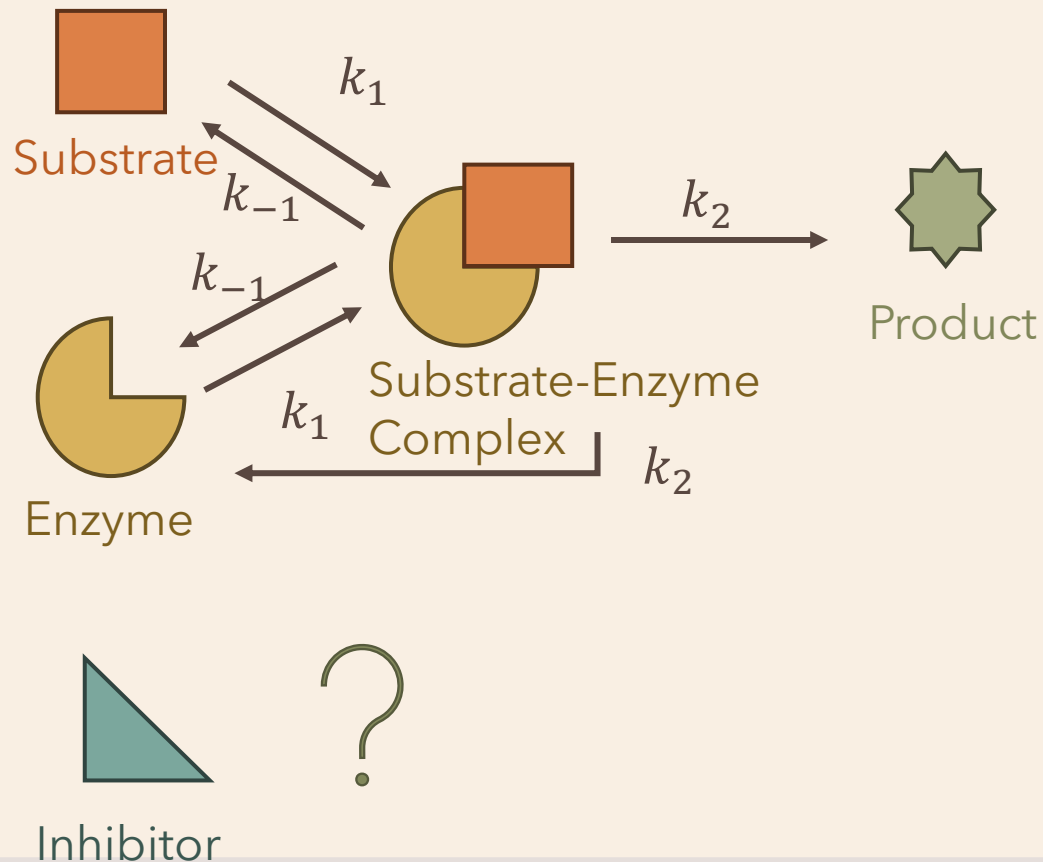
Hybrid model



- Facilitate **rapid model extension**.
- Model can be trained with **fewer learning examples**.
- Facilitate multi-modal **data integration**.

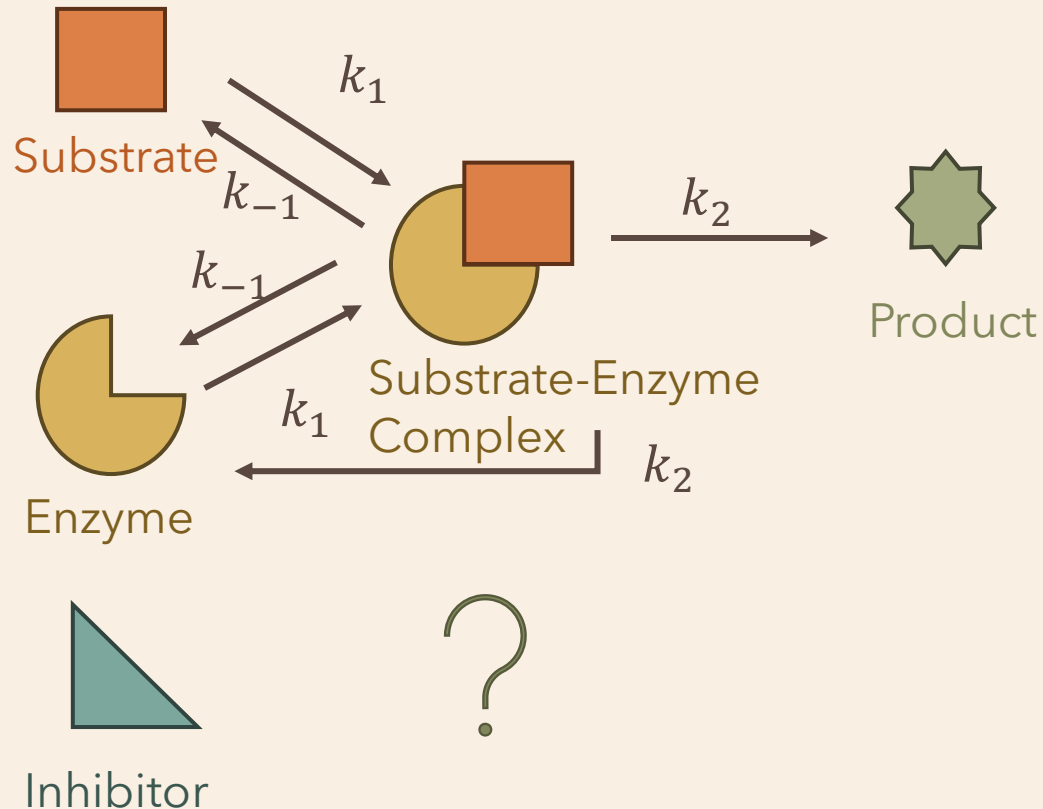
UNIVERSAL DIFFERENTIAL EQUATIONS

- **Step 1:** Identify possible modulators



UNIVERSAL DIFFERENTIAL EQUATIONS

- **Step 2:** Hypothesis where the missing interaction occurs.



Model Enzyme Catalysis

$$\frac{d[S]}{dt} = -k_1[S][E] + k_{-1}[ES]$$

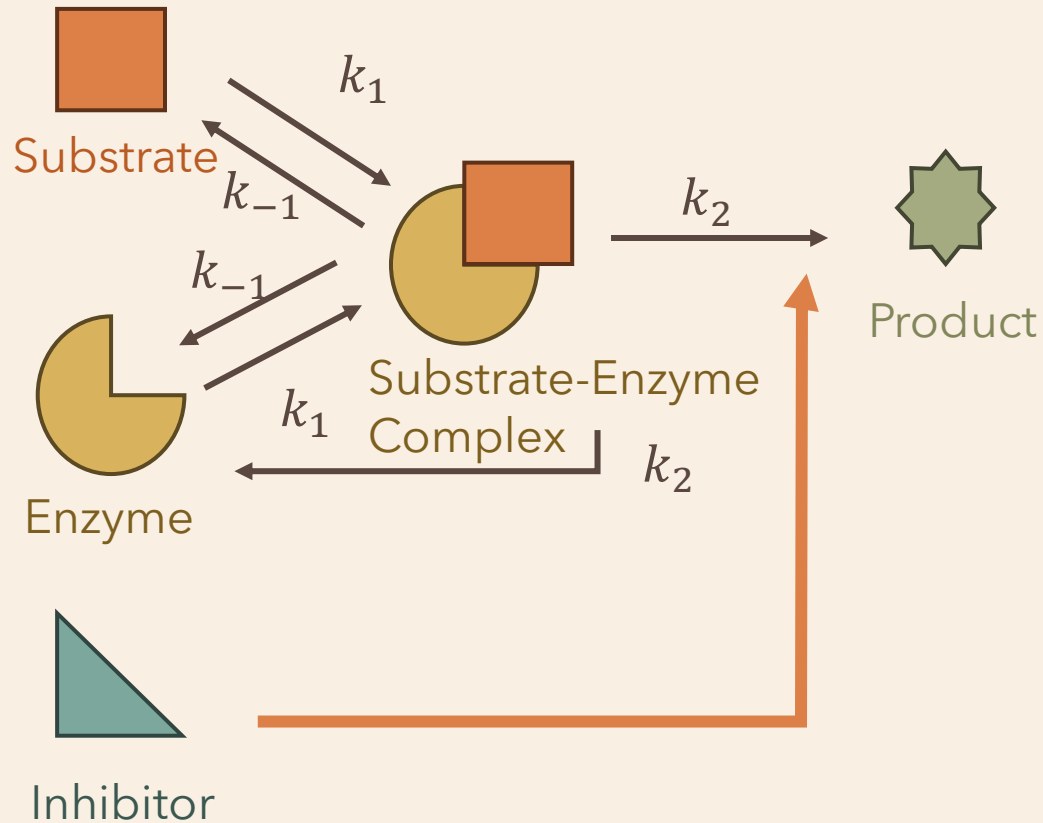
$$\frac{d[E]}{dt} = -k_1[S][E] + (k_{-1} + k_2)[ES]$$

$$\frac{d[ES]}{dt} = k_1[S][E] - (k_{-1} + k_2)[ES]$$

$$\frac{d[P]}{dt} = k_2[ES]$$

UNIVERSAL DIFFERENTIAL EQUATIONS

- **Step 2:** Hypothesis that the [I] inhibits product formation



Model Enzyme Catalysis

$$\frac{d[S]}{dt} = -k_1[S][E] + k_{-1}[ES]$$

$$\frac{d[E]}{dt} = -k_1[S][E] + (k_{-1} + k_2)[ES]$$

$$\frac{d[ES]}{dt} = k_1[S][E] - (k_{-1} + k_2 \cdot N(ES, I))[ES]$$

$$\frac{d[P]}{dt} = k_2[ES] \cdot N(ES, I)$$

UNIVERSAL DIFFERENTIAL EQUATIONS

- **Step 3:** Define neural network

```
softplus(x) = log(1 + exp(x))
rng = StableRNG(1234)

# we initialize weights close to zero to avoid large corrections at the beginning of training
function build_weight_initializer(;scale=1e-3)
    return (rng, out_dims, in_dims) -> scale * randn(rng, out_dims, in_dims)
end

chain = Chain(
    Dense(2, 4, softplus, use_bias=false, init_weight=build_weight_initializer()),
    Dense(4, 1, softplus, use_bias=false, init_weight=build_weight_initializer())
)

p_init, st = Lux.setup(rng, chain)

function network(u, p)
    return chain(u, p, st)[1][1]
end

network([1.0, 0.0], p_init) # test the network with some input
```


UNIVERSAL DIFFERENTIAL EQUATIONS

- **Step 4:** Place NN into ODE definition

Function input:

- Change in states
- Initial values
- Parameters (Both ODE and NN)
- Time
- Neural network
- Forcing function of I

```
function proposition_1!(du, u, p, t, network, force)
    S, P, E, ES = u
    k1, kn1, k2 = p.known

    correction = network([ES, force(t)], p.network)

    du[1] = -k1 * S * E + kn1 * (ES) # dS/dt
    du[2] = k2 * (ES) * correction # dP/dt
    du[3] = -k1 * S * E + (kn1 + k2 * correction) * (ES) # dE/dt
    du[4] = k1 * S * E - (kn1 + k2 * correction) * (ES) # dES/dt
end
```

Run NN

Define ODEs with
NN correction

UNIVERSAL DIFFERENTIAL EQUATIONS

Define ODE problem including inputs for NN



```
# define ODE problem
prob_1 = ODEProblem((du, u, p, t) -> proposition_1!(du, u, p, t, network, force_exp_1), u0_known, tspan, ComponentArray{known=exp.(ones(3)), network=p_init}))

#define Error function/cost function
function sum_of_squares_ude(p, (prob_1, data, u0))
    loss = eltype(p.known)(0.0)
    params = ComponentArray(
        #ODE parameters are log transformed while NN parameters are not
        known = exp.(p.known),
        network = p.network
    )

    sol = solve(prob_1, Rodas5P(), saveat=timepoints, p=params, u0=eltype(p.known).(u0))
    if size(sol, 2) != length(timepoints)
        return eltype(p.known)(Inf)
    end
    #calculate the some of square errors between simulation and measured data
    loss_S = sum(sum(abs2, sol[1, :] - data[1, :]))
    loss_P = sum(sum(abs2, sol[2, :] - data[2, :]))
    loss += loss_S + loss_P
end
return loss
end
```

Define error functions/cost function to be optimised

UNIVERSAL DIFFERENTIAL EQUATIONS

- **Step 5:** Optimise UDE system

```
#run parameter optimisation (note first we use ADAM and then LBFGS)
res_1 = optimize(
    loss_ude_1,
    ComponentArray(known=exp.(ones(3)), network=p_init),
    Adam(;alpha=0.01),
    Optim.Options(iterations=1000,callback=get_progress_callback(1000))
)
res_1 = optimize(
    loss_ude_1,
    res_1.minimizer,
    LBFGS(),
    Optim.Options(iterations=1000,callback=get_progress_callback(1000))
)
```

First run ADAM -
global optimiser

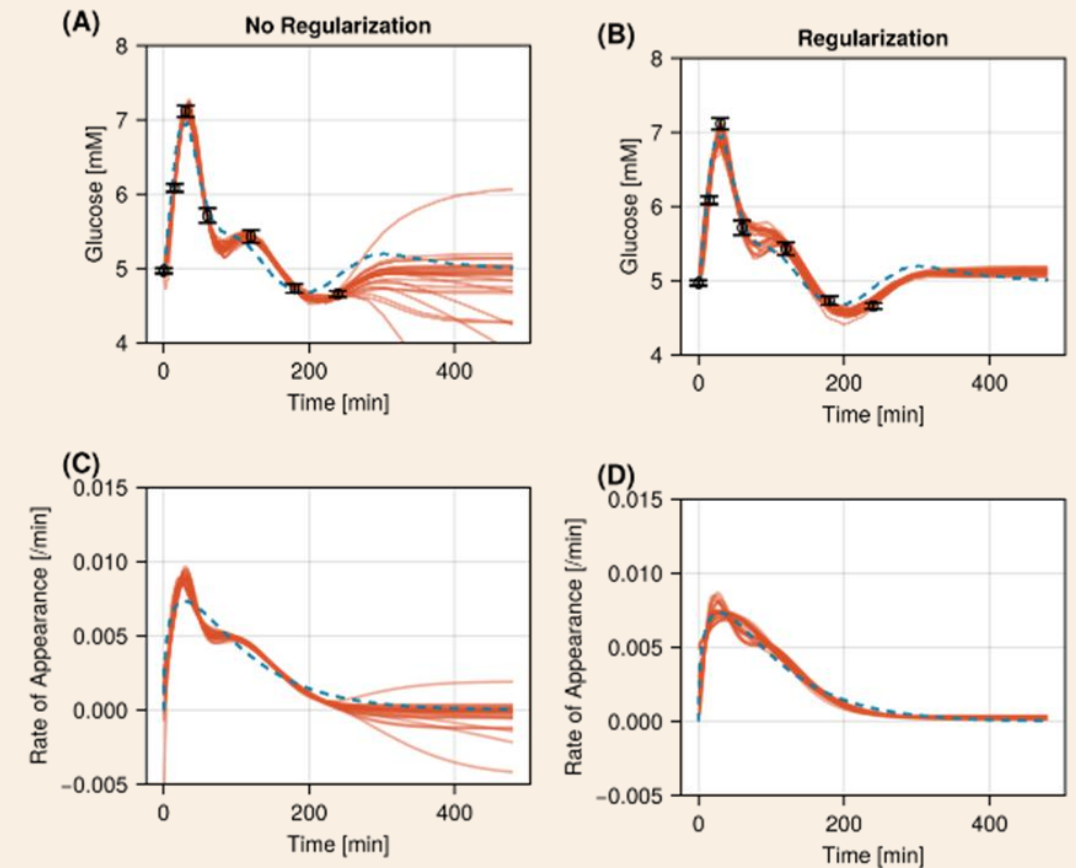
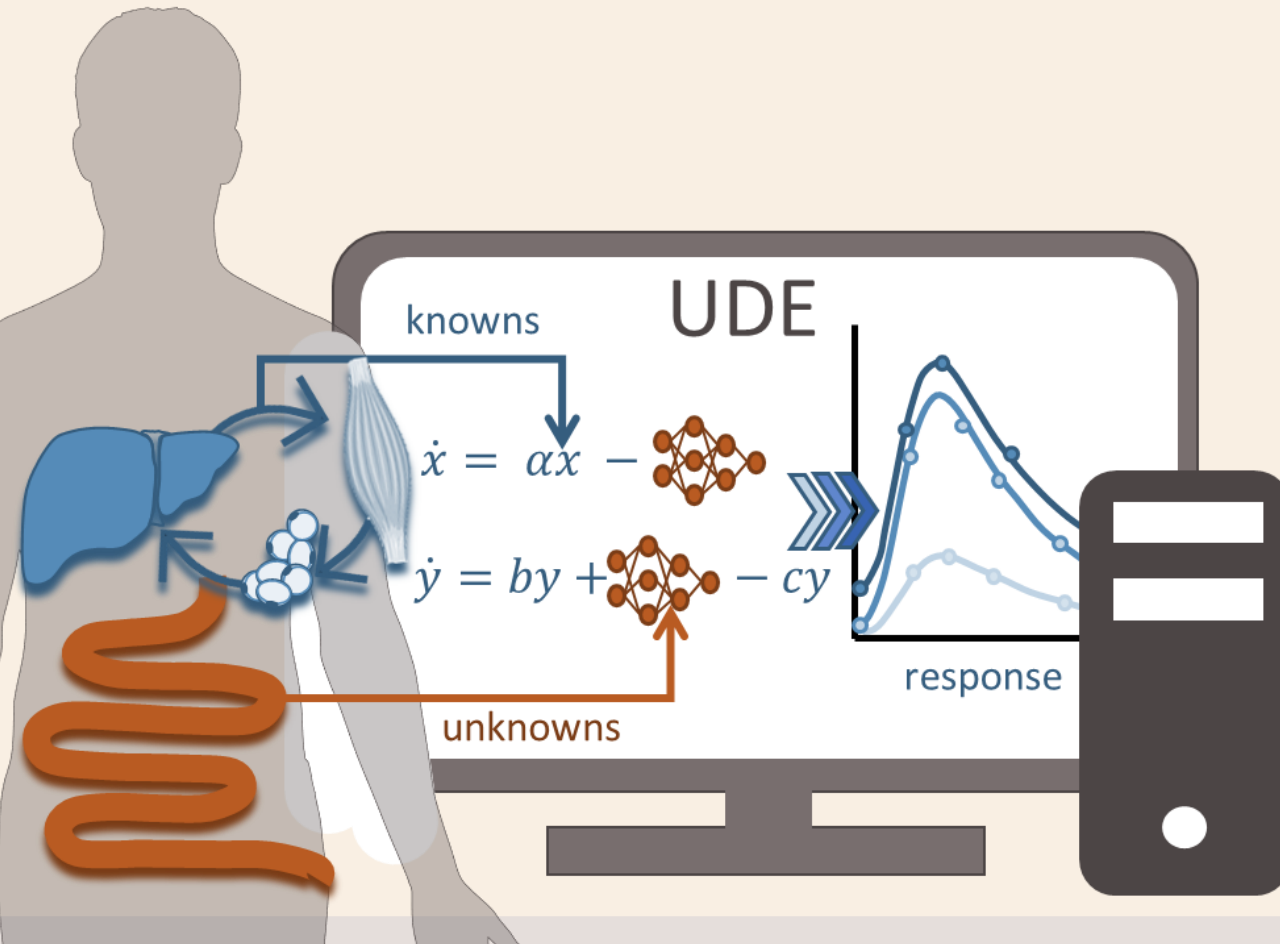
Then run LBFGS - local
optimizer - using the
global minimum as the
initial value

UDEs IN BIOLOGY

Physiology-informed regularization allows a UDE to learn the rate of glucose appearance from the gut on **sparse data**.



Max de Rooij



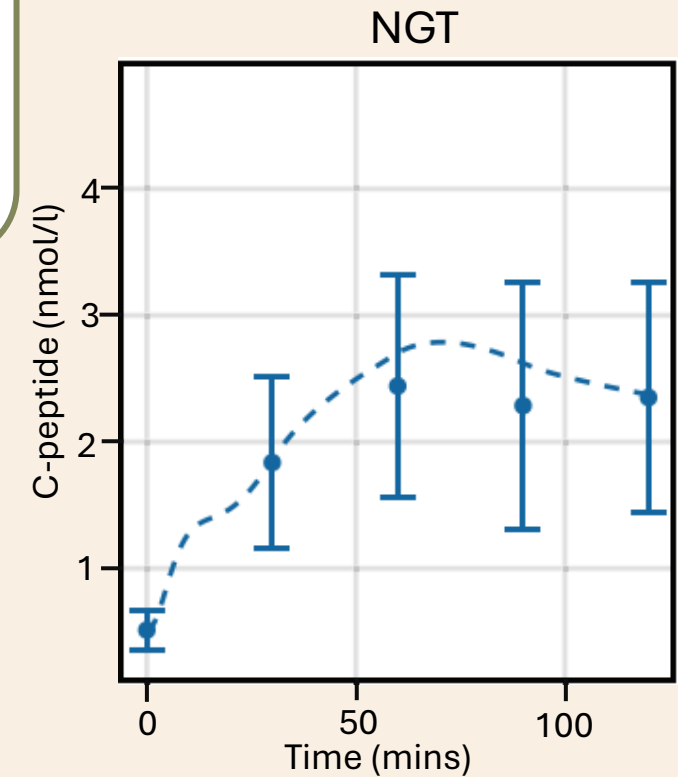
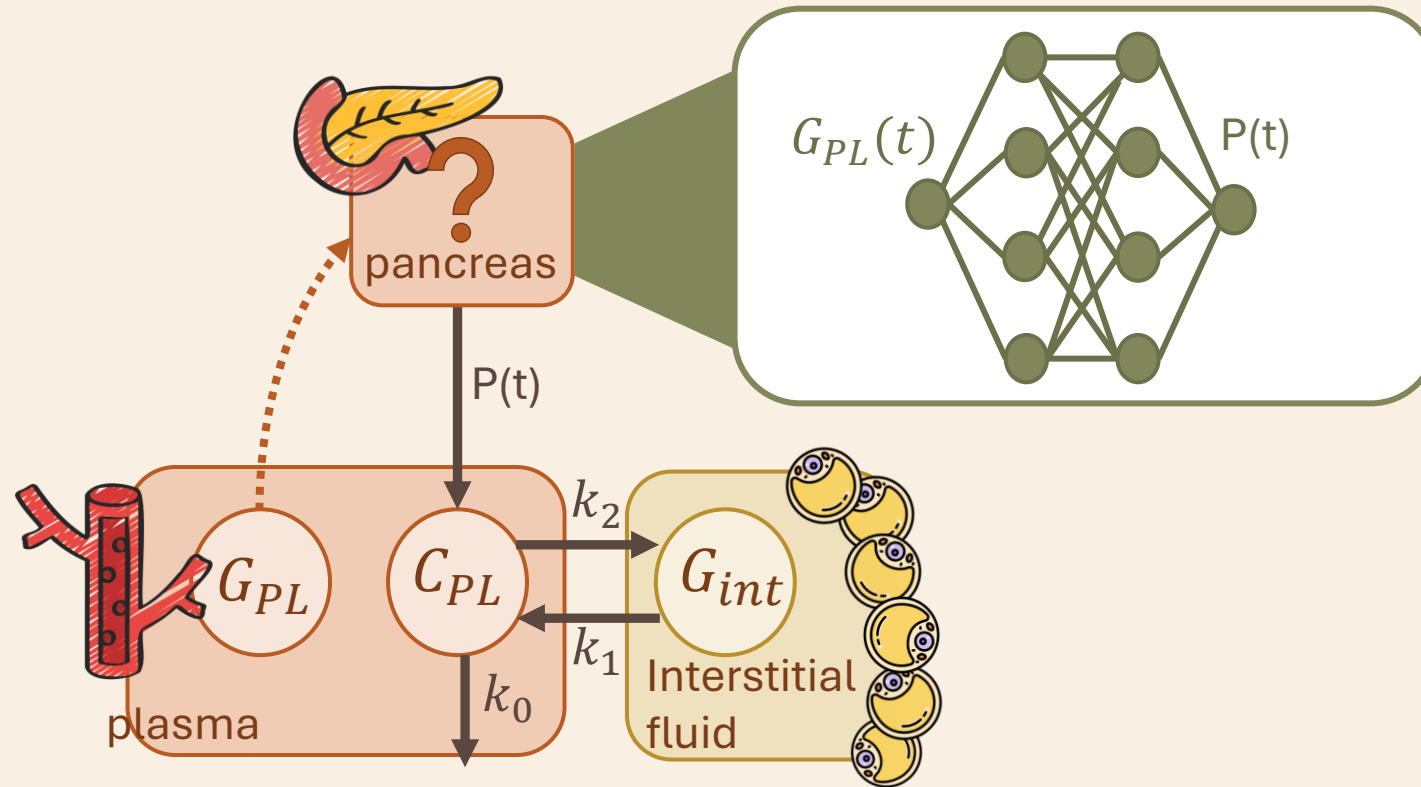
UNIVERSAL DIFFERENTIAL EQUATIONS

- ✓ Can learn missing interactions from data - greatly speeding up model development.
- ✓ Can incorporate external features.
- ✓ Can be trained with much fewer learning examples than NNs or PINNs/still needs some data.
- ✗ Requires domain knowledge.
- ✗ User must specify location of unknown interactions.
- ✗ NN is still a black box

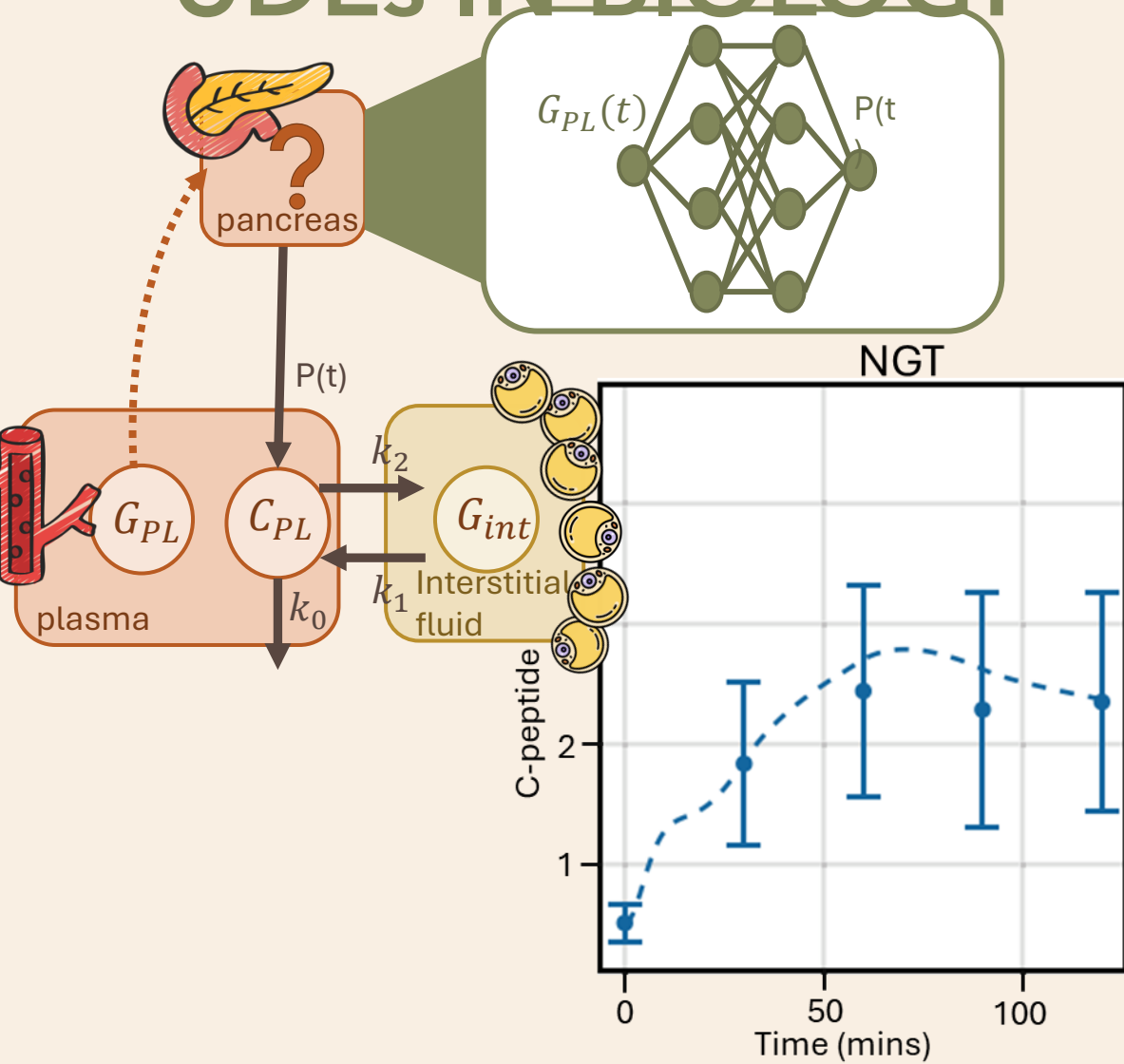
UDEs IN BIOLOGY



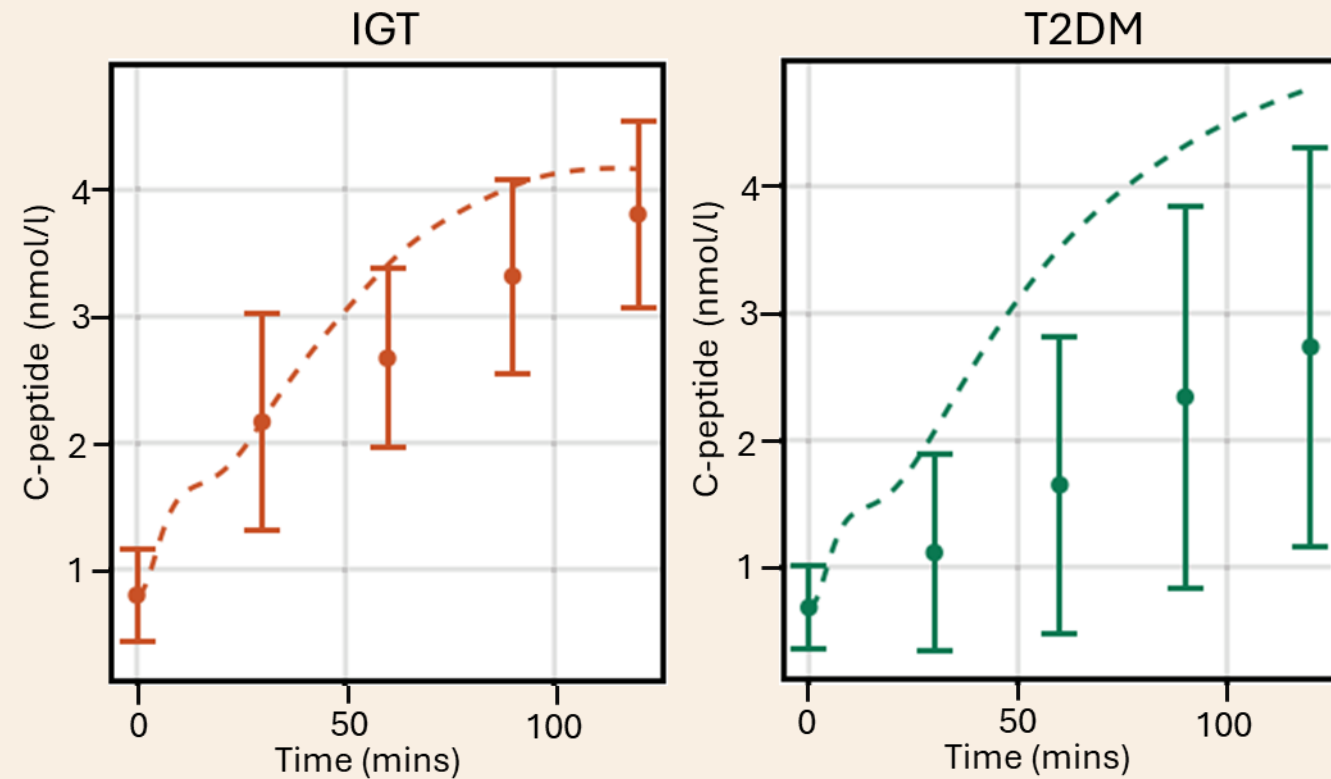
Max de Rooij



UDEs IN BIOLOGY



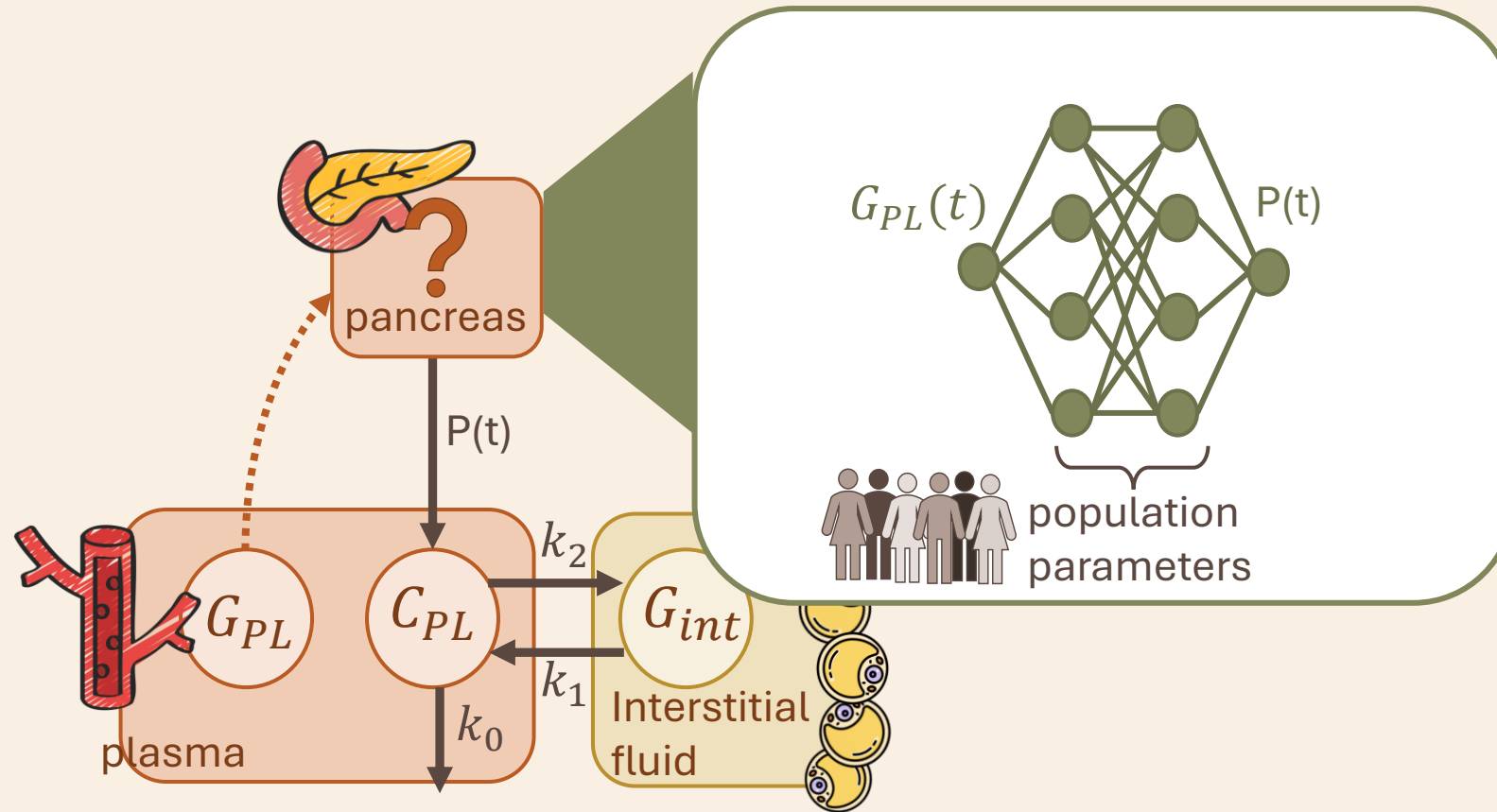
Max de Rooij



UDEs IN BIOLOGY



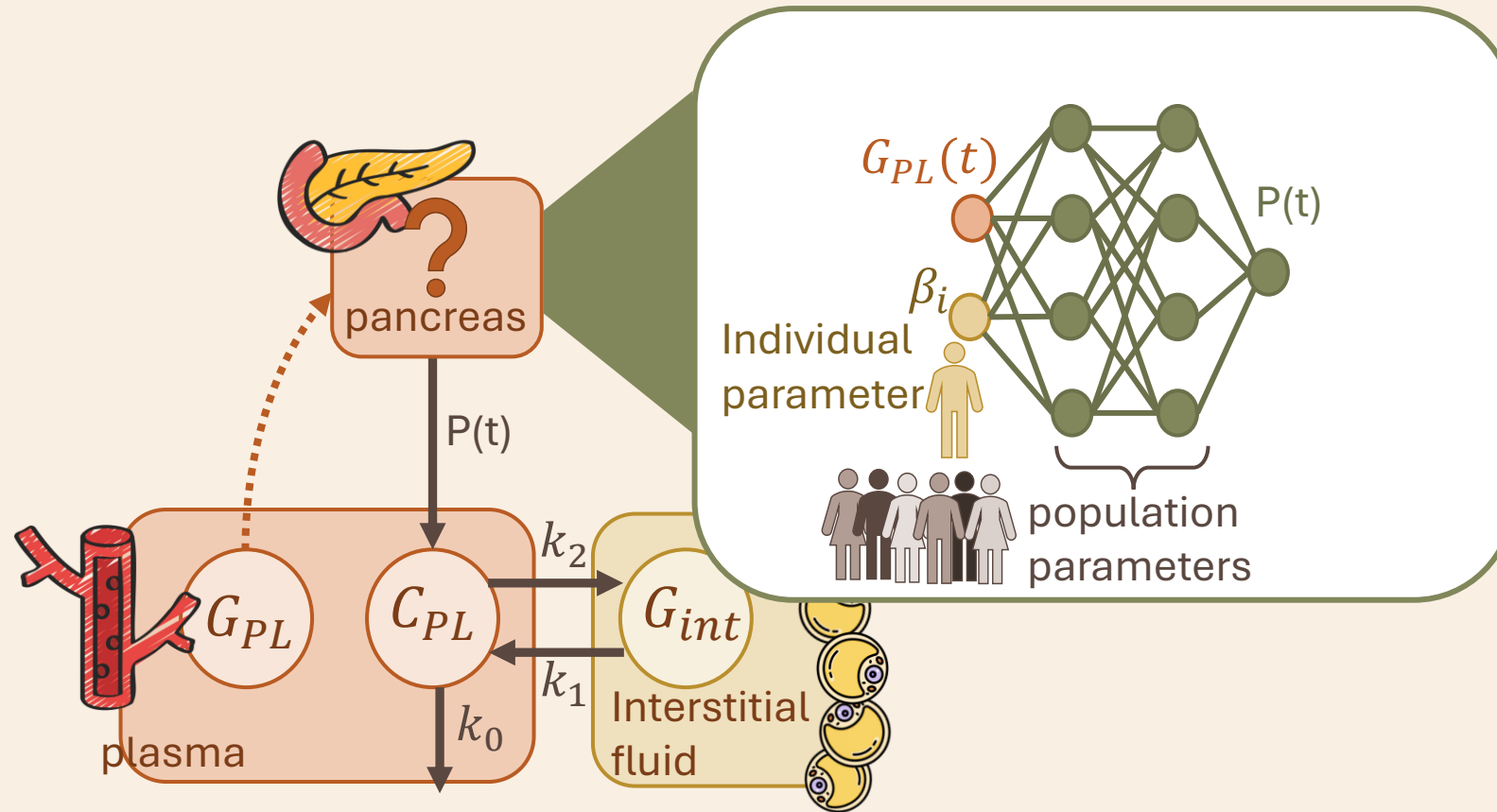
Max de Rooij



UDEs IN BIOLOGY



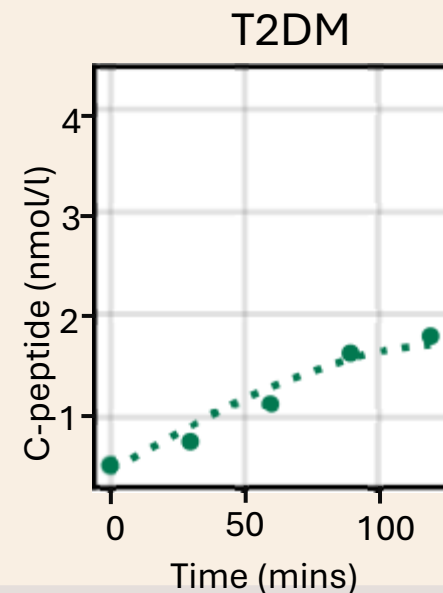
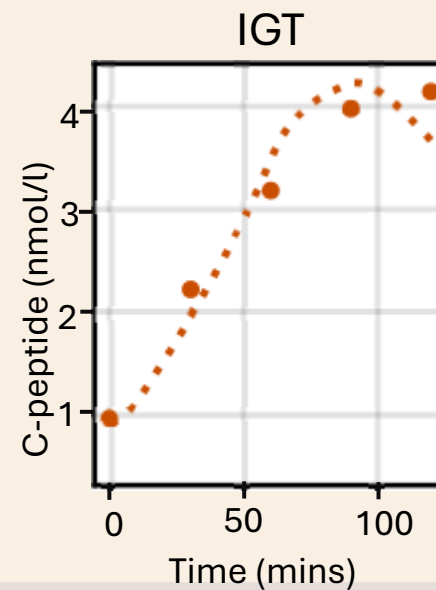
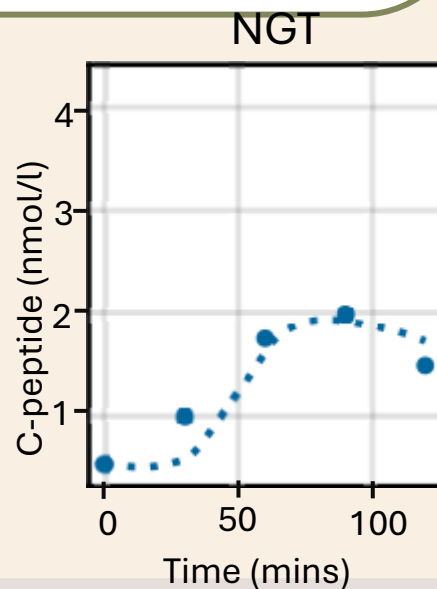
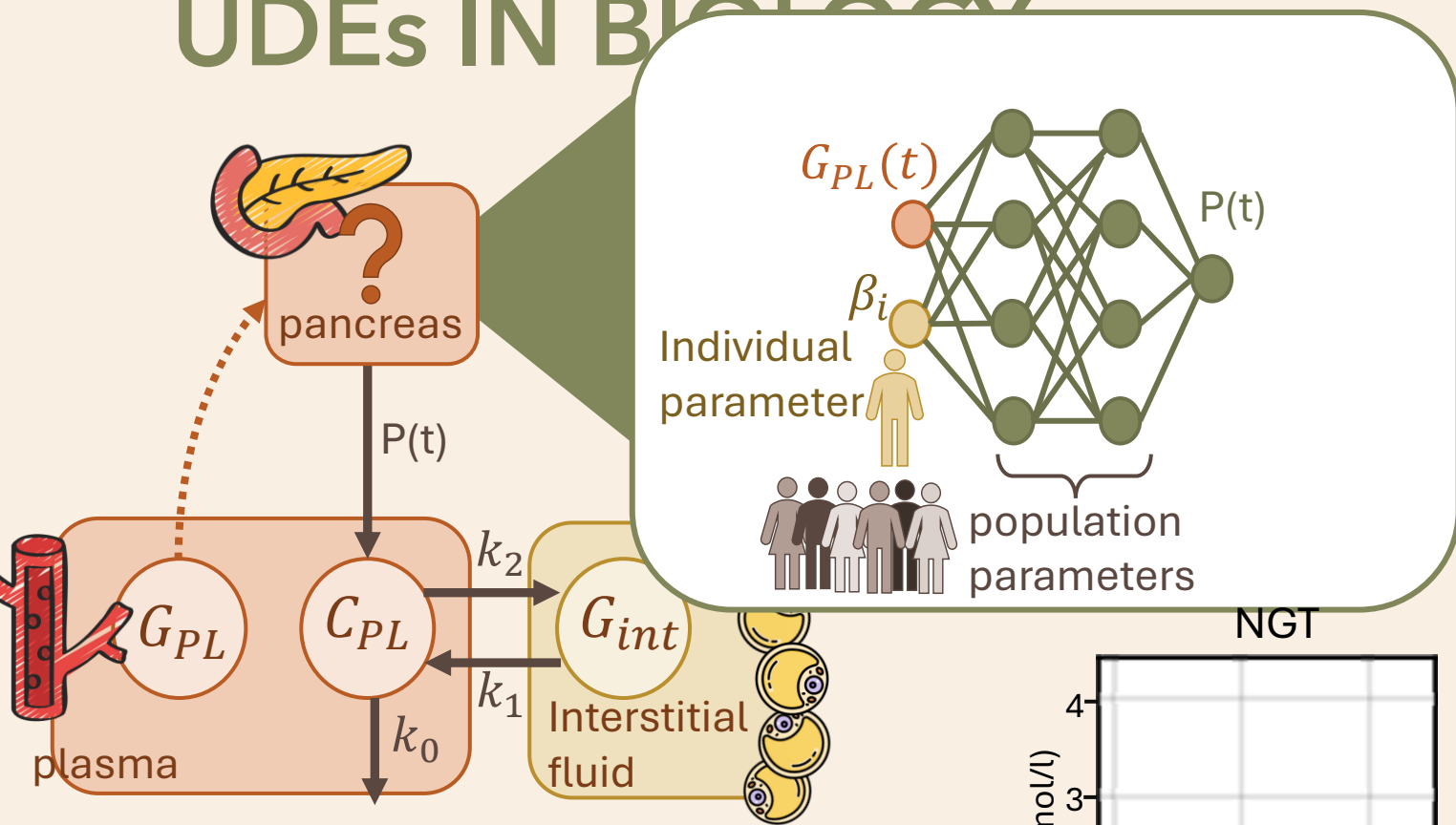
Max de Rooij



UDEs IN BIOLOGY



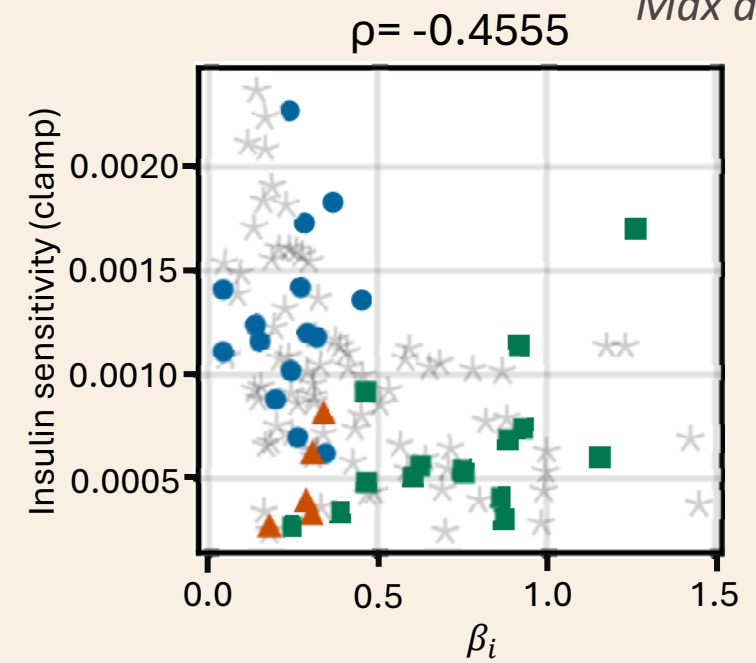
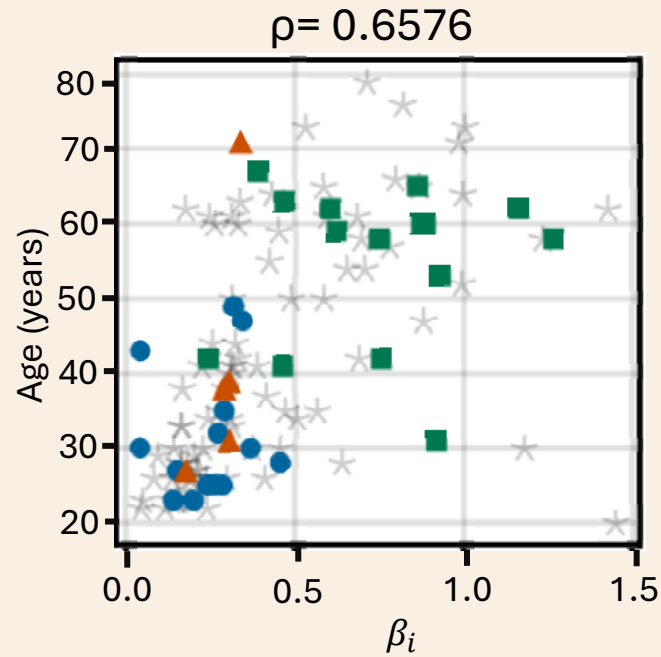
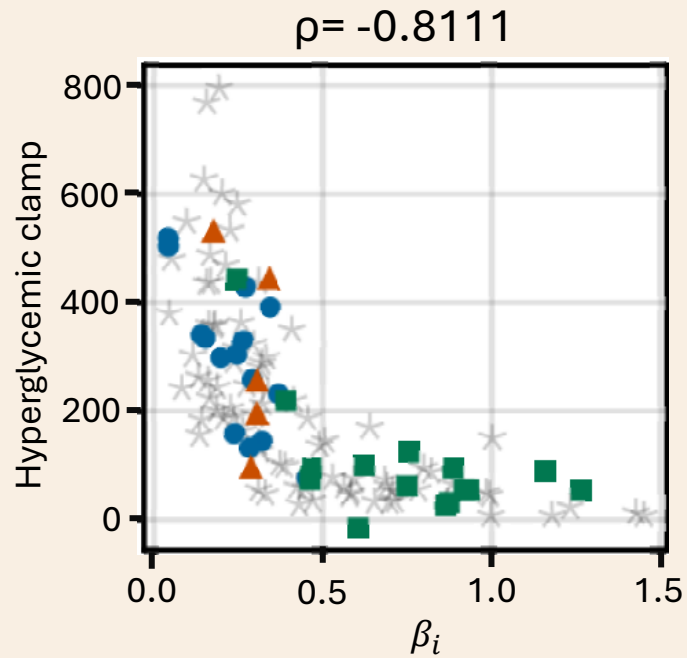
Max de Rooij



UDEs IN BIOLOGY



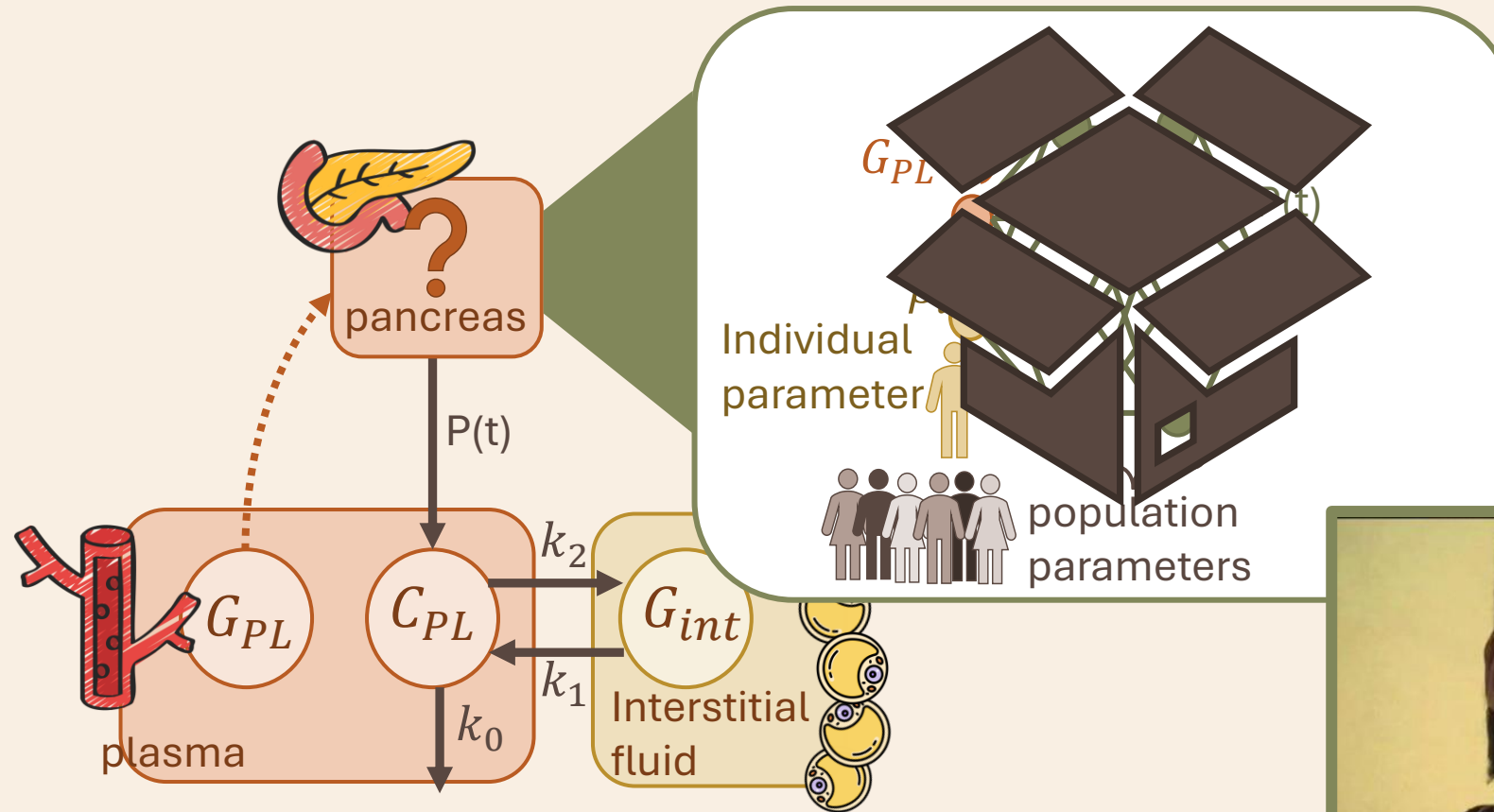
Max de Rooij



UDEs IN BIOLOGY

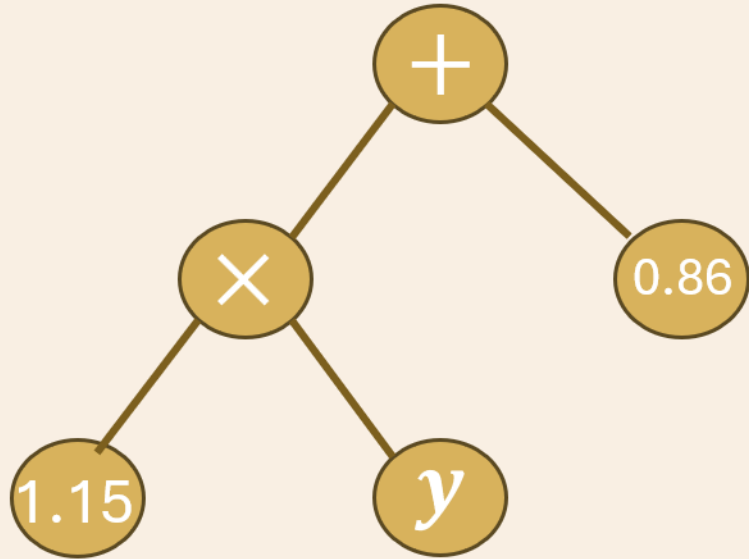


Max de Rooij

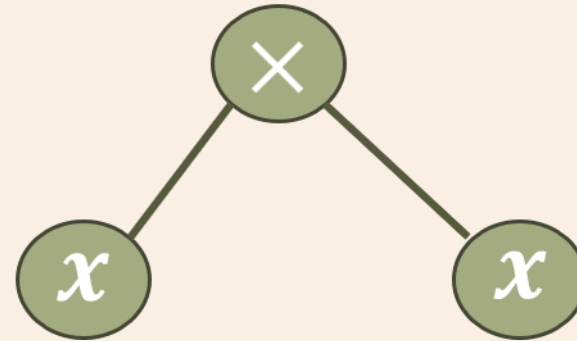


FUNCTIONS AS TREES

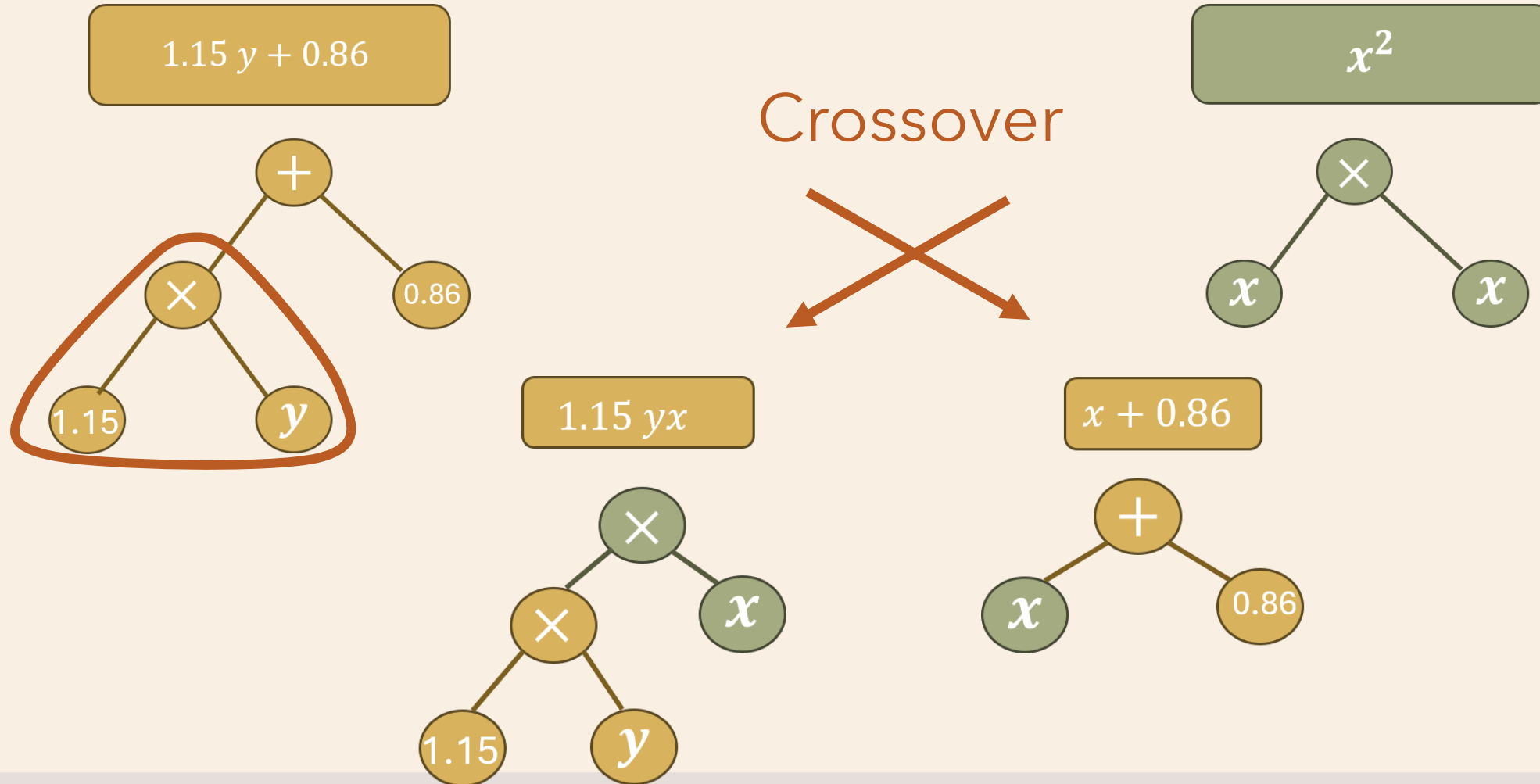
$$1.15y + 0.86$$



$$x^2$$



FUNCTIONS AS TREES



FUNCTIONS AS TREES

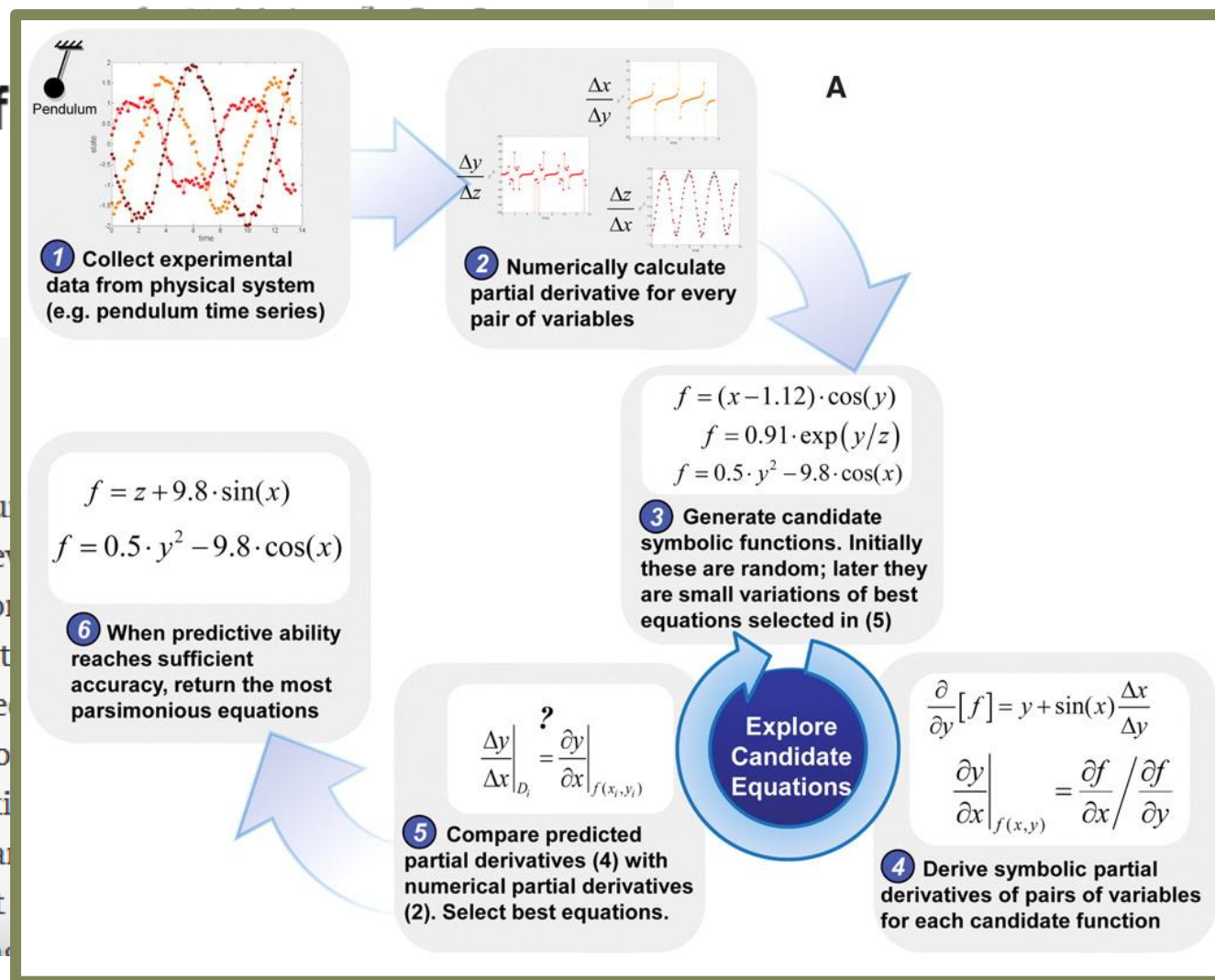


Distilling Free-Form Natural Laws from Data

MICHAEL SCHMIDT AND HOD LIPSON [Authors Info & Affiliations](#)

Abstract

For centuries, scientists have attempted to identify and document the natural laws that underlie physical phenomena in nature. Despite the power of modern computing, the process of finding natural laws and their corresponding mathematical representations has resisted automation. A key challenge to finding analytic relationships is defining algorithmically what makes a correlation in observed data insightful. We propose a principle for the identification of natural laws and demonstrate this approach by automatically searching for natural laws in data gathered from various physical systems, ranging from simple harmonic motion to chaotic double-pendula. Without any prior knowledge about the underlying physics or geometry, the algorithm discovered Hamiltonians, Lagrangians,

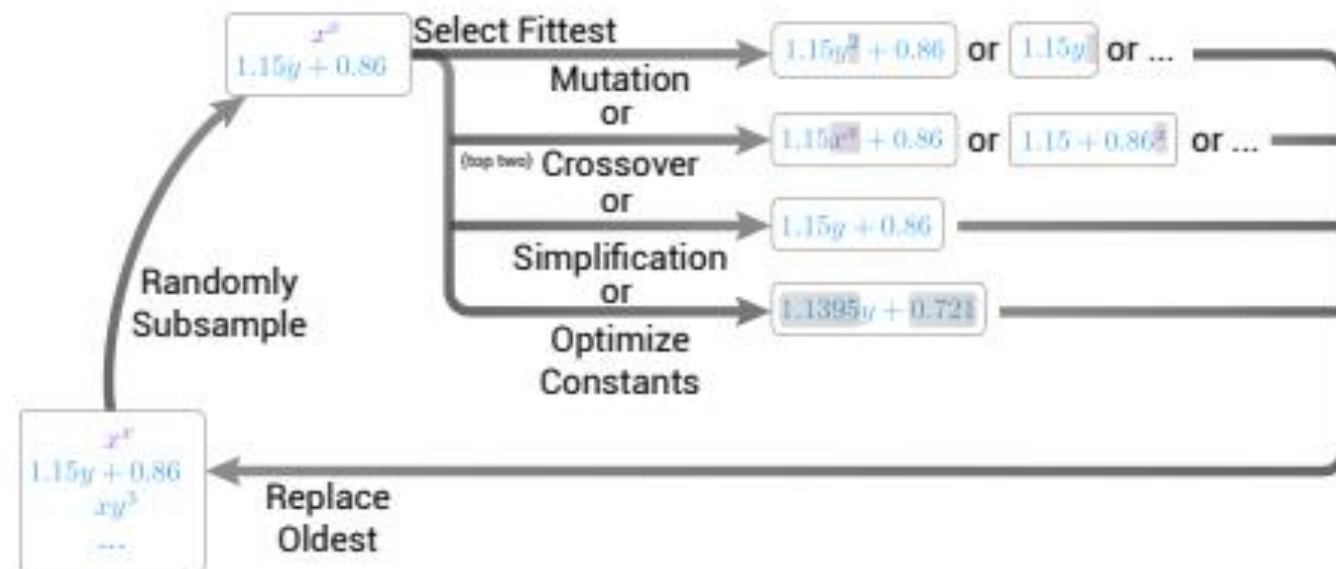


Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl

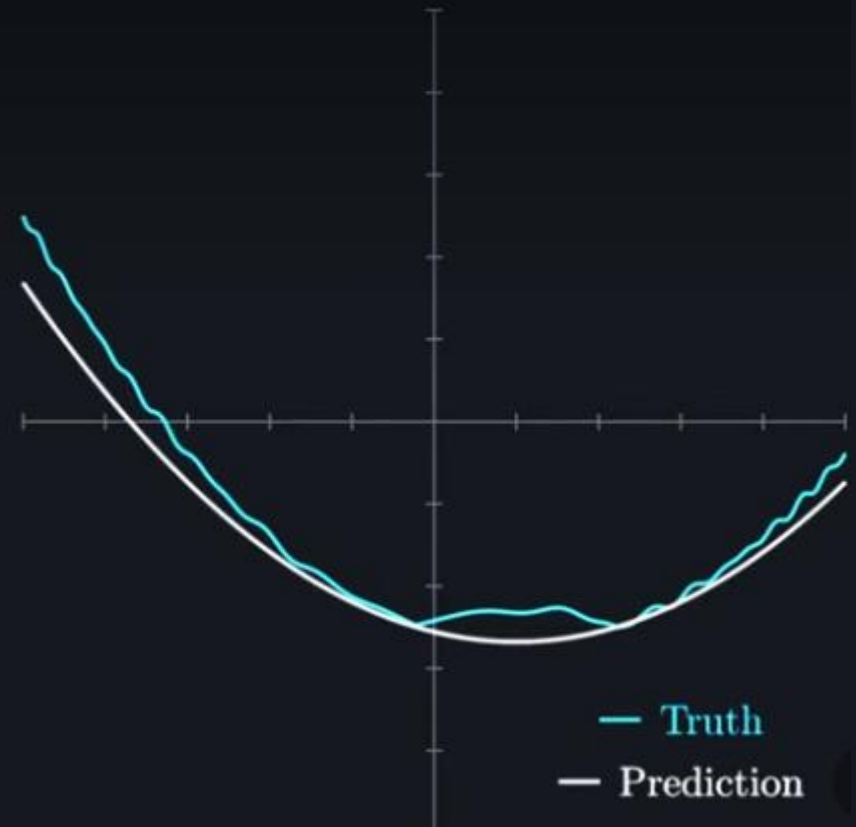
Miles Cranmer^{1,2}

¹Princeton University, Princeton, NJ, USA

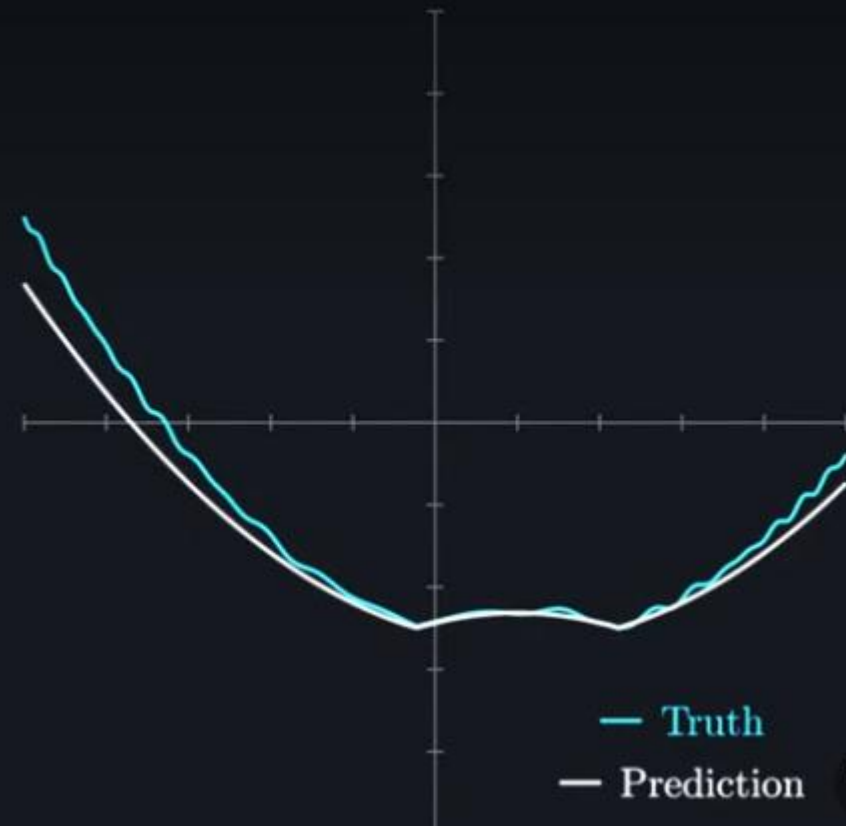
²Flatiron Institute, New York, NY, USA



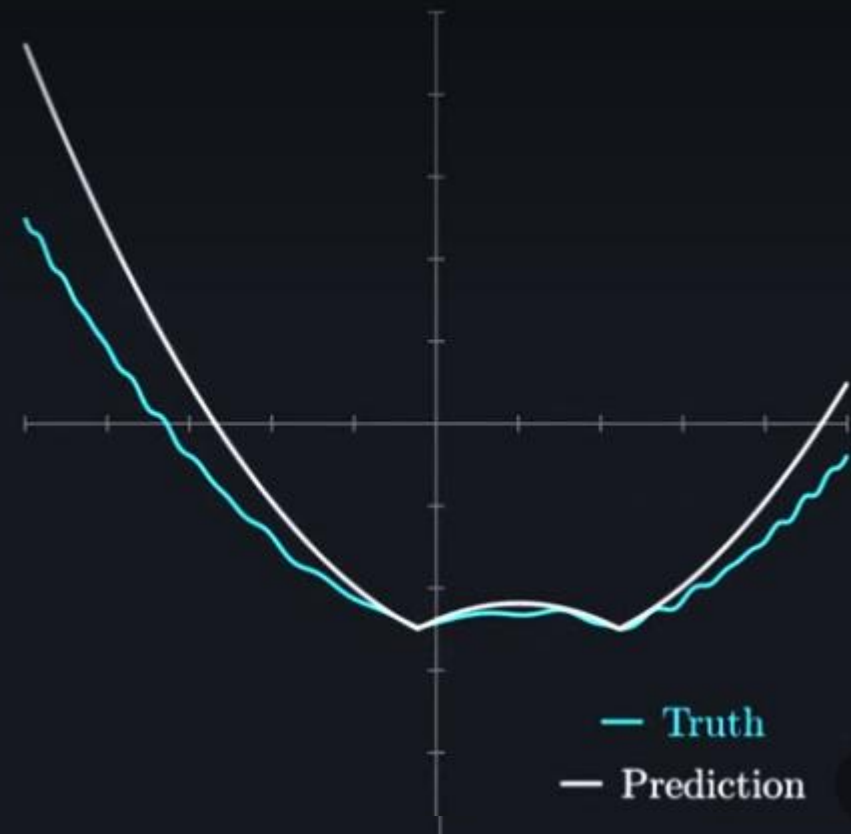
PySR - SYMBOLIC REGRESSION



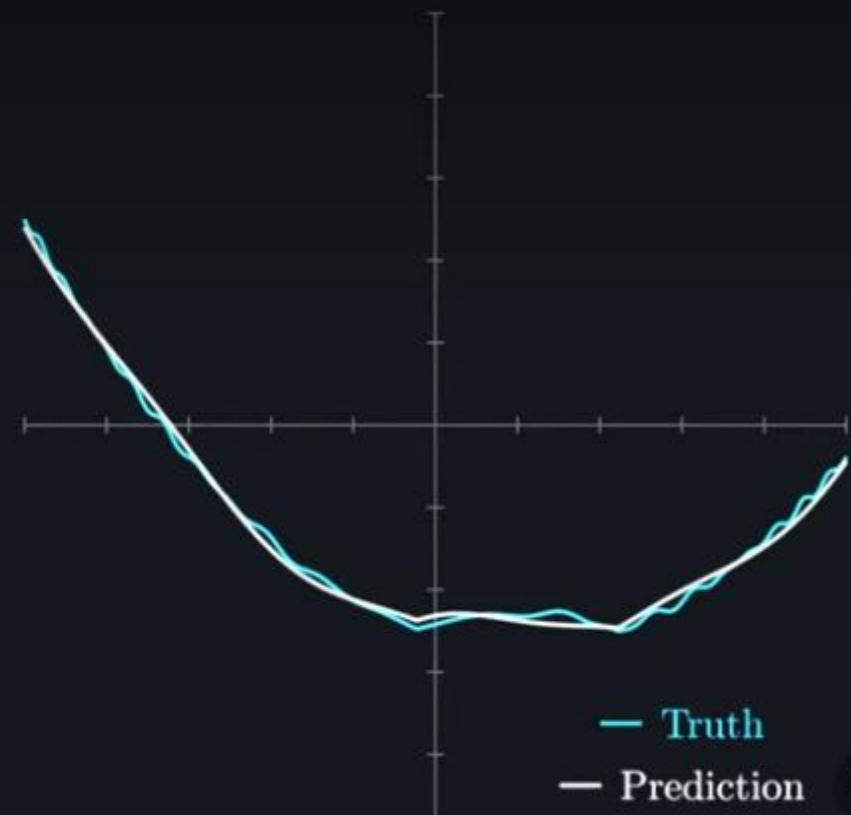
PySR - SYMBOLIC REGRESSION



PySR - SYMBOLIC REGRESSION



PySR - SYMBOLIC REGRESSION

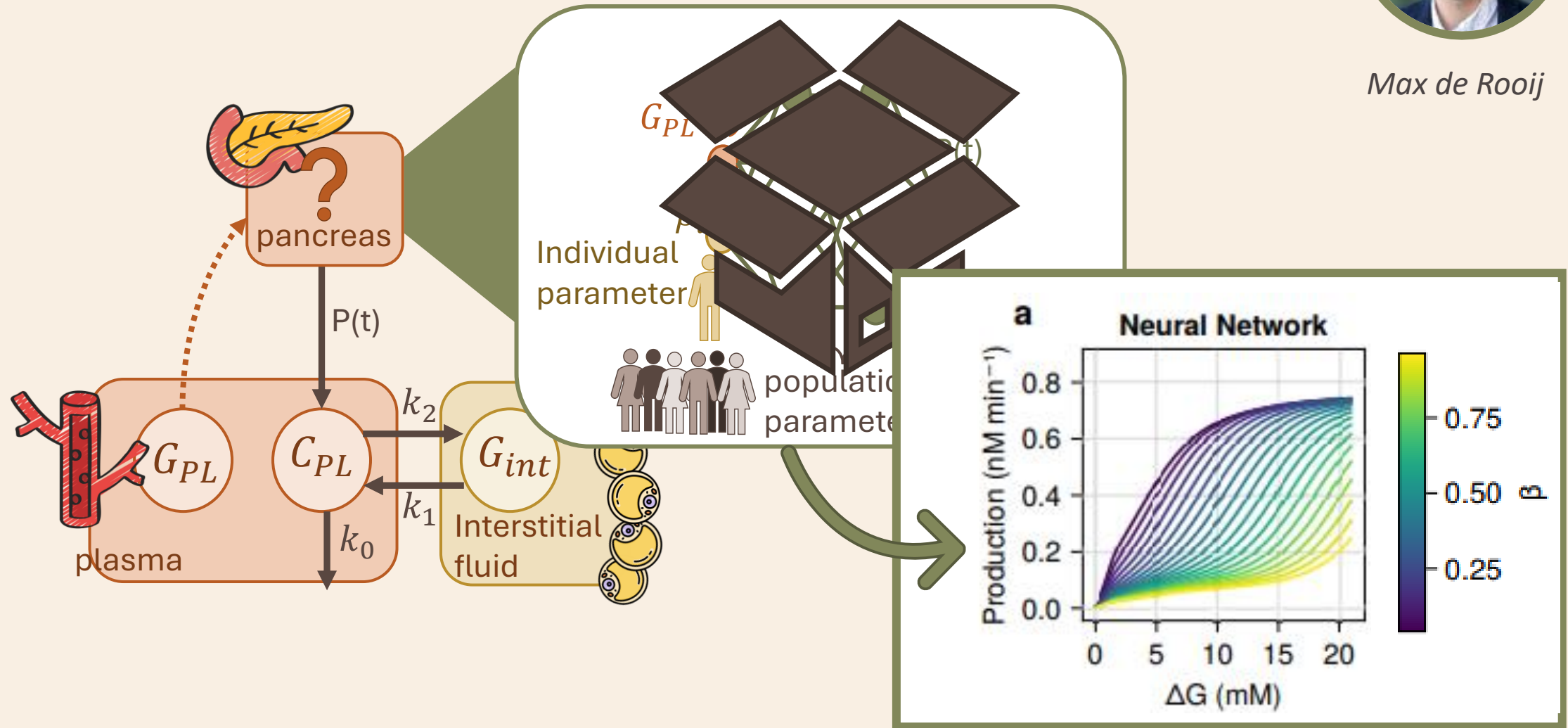


Cranmer "PySR and SymbolicRegression.jl paper trailer" YouTube

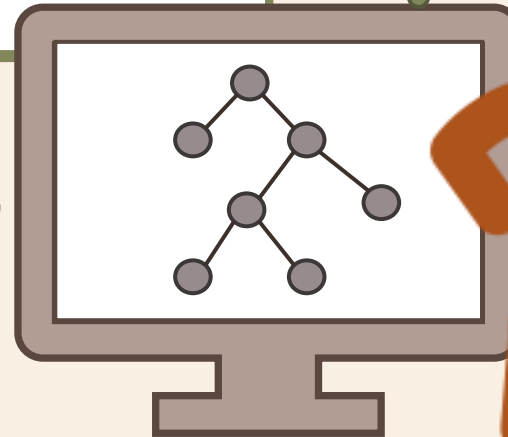
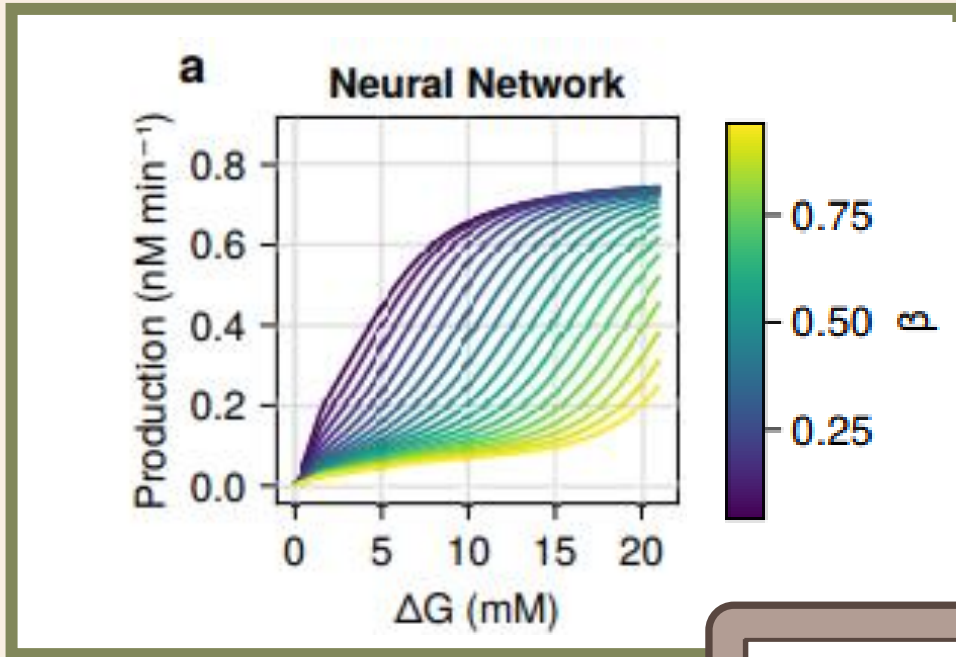
UDEs IN BIOLOGY



Max de Rooij



SYMBOLIC REGRESSION



Expressions evaluated per second: 1.300e+04
Progress: 823 / 900 total iterations (91.444%)

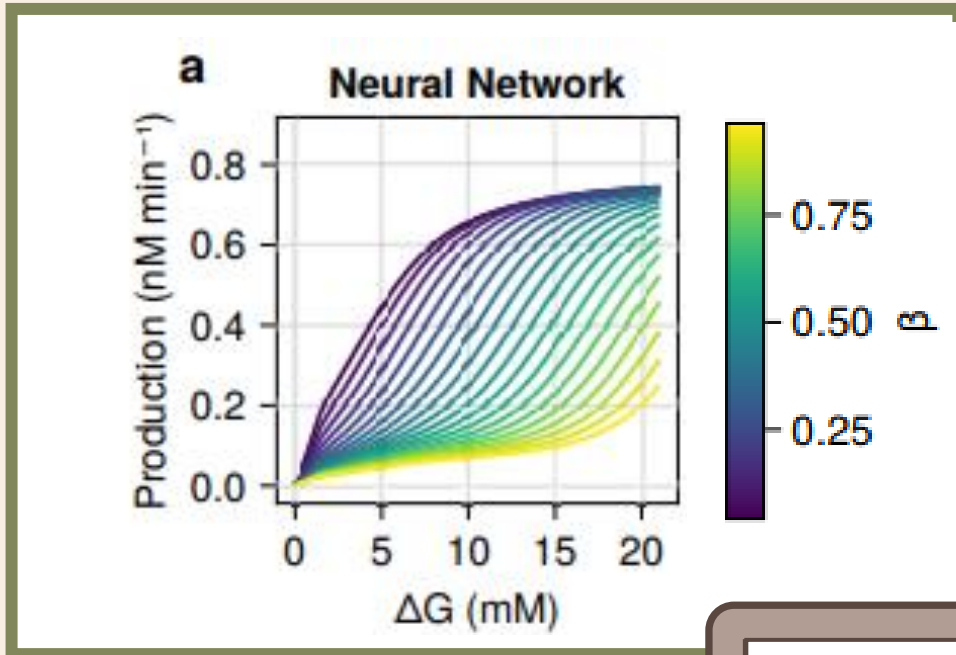
Complexity	Loss	Score	Equation
1	1.035e+09	0.000e+00	y = 85008
3	1.035e+09	-0.000e+00	y = 85010 - h
4	3.551e+08	-0.000e+00	y = cube(44.037 - h)
5	2.186e+08	-0.000e+00	y = (h * -9798.6) + 84998
7	1.127e+08	-0.000e+00	y = (h - (16.257 - v)) * -7505
8	9.300e+07	-0.000e+00	y = 1.096e+05 - (9825.2 * (h + sqrt(v)))
9	1.376e+07	-0.000e+00	y = (1.0962e+05 - (v * 4980.3)) - (h * 9825.3)
10	1.086e+05	4.154e+00	y = 1.0131e+05 - ((square(v) * 499.07) - (h * -9795.5))
12	1.086e+05	-0.000e+00	v = ((h * (-9795.6 - h)) - -1.0132e+05) - (square(v) * 49...

$$p(G^{PL}(t), \beta_i) = \frac{11.78 \cdot (G^{PL}(t) - G^{PL}(0))}{21.8 + 167(\beta_i)^3 + (G^{PL}(t) - G^{PL}(0))}$$

```

19 1.085e+05 -0.000e+00 y = ((-8770.1 - (h * (h - -9795.7))) + 1.1009e+05) - (squ...
are(v * -22.34) + (sin(v) * -17.094))
20 1.084e+05 -0.000e+00 y = (1.0132e+05 - square(v * -22.339)) - (((h * (1.0606 - ...
sin(square(-1.1325 - v)))) - -9795.4) * h)
21 1.083e+05 -0.000e+00 y = (1.0132e+05 - square(-22.339 * v)) - (((h * (1.0606 - ...
sin(square(square(h) - v)))) - -9795.4) * h)
23 1.083e+05 -0.000e+00 y = ((1.0132e+05 - square(-22.339 * v)) - (((h * (1.0606 - ...
- sin(square(square(h) - v)))) - -9795.4) * h)) + -0.3232...
8
27 1.083e+05 -0.000e+00 y = (1.0132e+05 - (h - (h * -9795.4))) - (square(v * -22....
342) + (sin(square((v * v) + -22.311) + square(v * -16.79...
9)) * -18.384))
28 1.083e+05 -0.000e+00 y = ((44807 - square((v * -22.327) + 76.35)) + ((v * -341...
5.6) + (62349 - (h * 9795.2)))) + (((v * 0.061271) / (-1....
2618 - h)) * v)
30 1.083e+05 -0.000e+00 y = ((44807 - square((v * -22.327) + 76.35)) + (((62349 - ...
(h * 9795.2)) + (v * -3415.6)) + -0.12409)) + (((v * 0.0...
61271) / (-1.2618 - h)) * v)
    
```


SYMBOLIC REGRESSION



Expressions evaluated per second: 1.300e+04
Progress: 823 / 900 total iterations (91.444%)

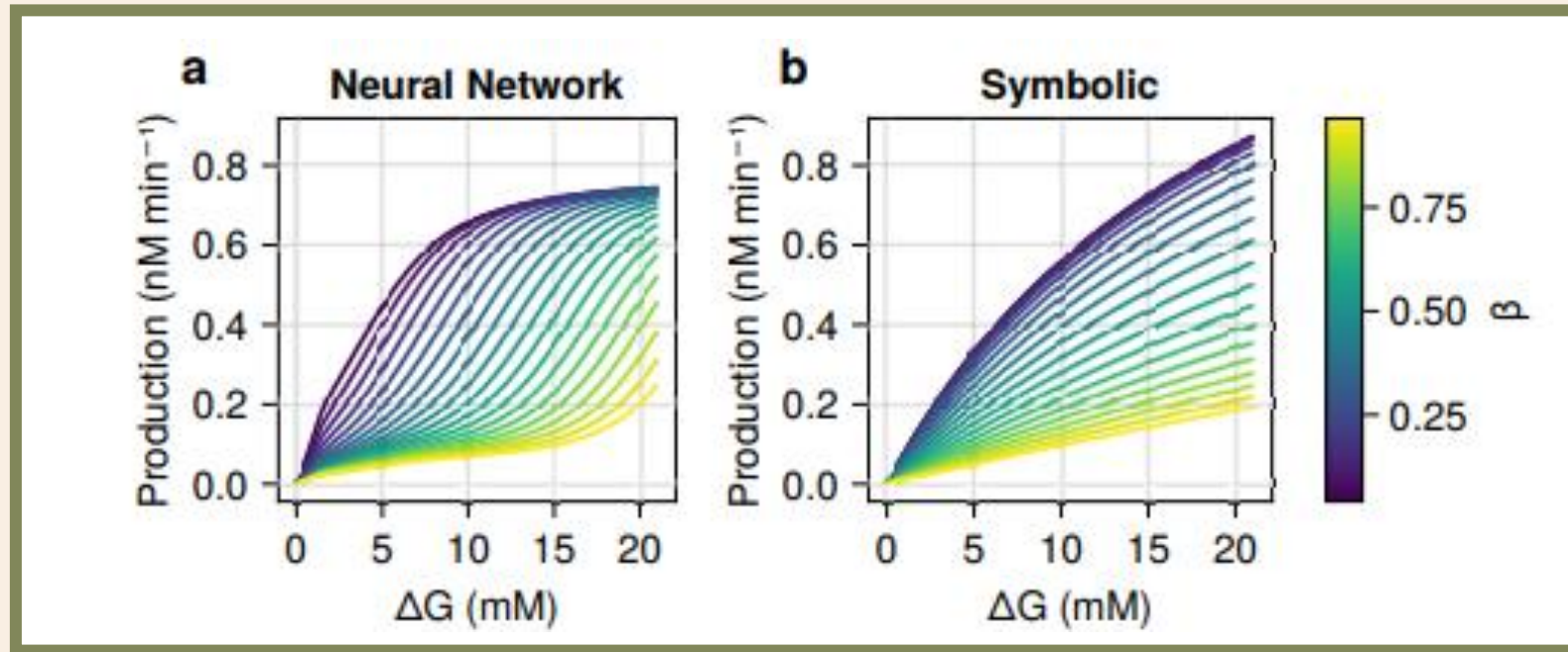
Complexity	Loss	Score	Equation
1	1.035e+09	0.000e+00	y = 85008
3	1.035e+09	-0.000e+00	y = 85010 - h
4	3.551e+08	-0.000e+00	y = cube(44.037 - h)
5	2.186e+08	-0.000e+00	y = (h * -9798.6) + 84998
7	1.127e+08	-0.000e+00	y = (h - (16.257 - v)) * -7505
8	9.300e+07	-0.000e+00	y = 1.096e+05 - (9825.2 * (h + sqrt(v)))
9	1.376e+07	-0.000e+00	y = (1.0962e+05 - (v * 4980.3)) - (h * 9825.3)
10	1.086e+05	4.154e+00	y = 1.0131e+05 - ((square(v) * 499.07) - (h * -9795.5))
12	1.086e+05	-0.000e+00	v = ((h * (-9795.6 - h)) - -1.0132e+05) - (square(v) * 49...

$$p(G^{PL}(t), \beta_i) = \frac{11.78 \cdot (G^{PL}(t) - G^{PL}(0))}{k_i + (G^{PL}(t) - G^{PL}(0))}$$

```

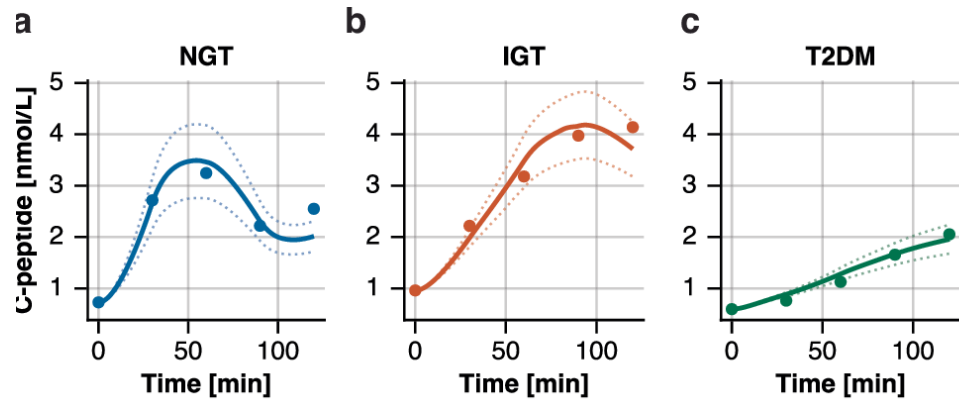
1 1.085e+05 -0.000e+00 y = ((-8770.1 - (h * (h - -9795.7))) + 1.1009e+05) - (squ...
are(v * -22.34) + (sin(v) * -17.094))
20 1.084e+05 -0.000e+00 y = (1.0132e+05 - square(v * -22.339)) - (((h * (1.0606 - ...
sin(square(-1.1325 - v)))) - -9795.4) * h)
1.083e+05 -0.000e+00 y = (1.0132e+05 - square(-22.339 * v)) - (((h * (1.0606 - ...
sin(square(square(h) - v)))) - -9795.4) * h)
23 1.083e+05 -0.000e+00 y = ((1.0132e+05 - square(-22.339 * v)) - (((h * (1.0606 ...
- sin(square(square(h) - v)))) - -9795.4) * h)) + -0.3232...
8
27 1.083e+05 -0.000e+00 y = (1.0132e+05 - (h - (h * -9795.4))) - (square(v * -22....
342) + (sin(square((v * v) + -22.311) + square(v * -16.79...
9)) * -18.384))
28 1.083e+05 -0.000e+00 y = ((44807 - square((v * -22.327) + 76.35)) + ((v * -341...
5.6) + (62349 - (h * 9795.2)))) + (((v * 0.061271) / (-1....
2618 - h)) * v)
30 1.083e+05 -0.000e+00 y = ((44807 - square((v * -22.327) + 76.35)) + (((62349 - ...
(h * 9795.2)) + (v * -3415.6)) + -0.12409)) + (((v * 0.0...
61271) / (-1.2618 - h)) * v)
    
```


SYMBOLIC REGRESSION

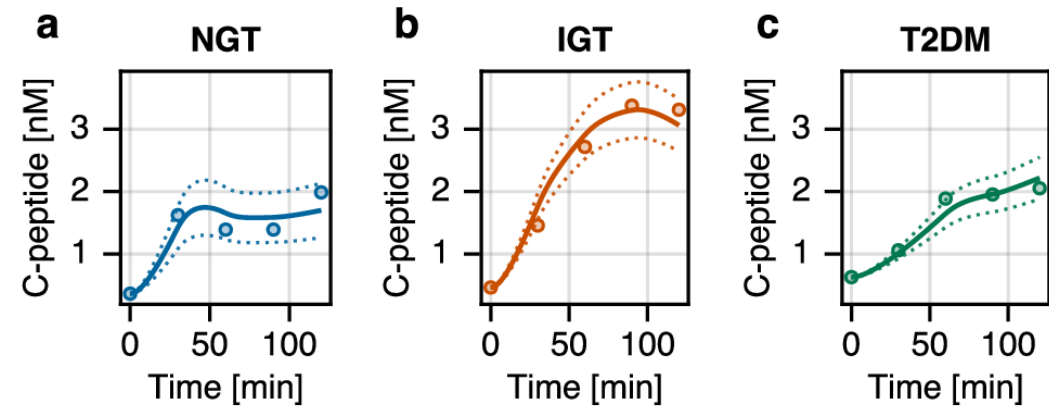


SYMBOLIC REGRESSION

Neural Network

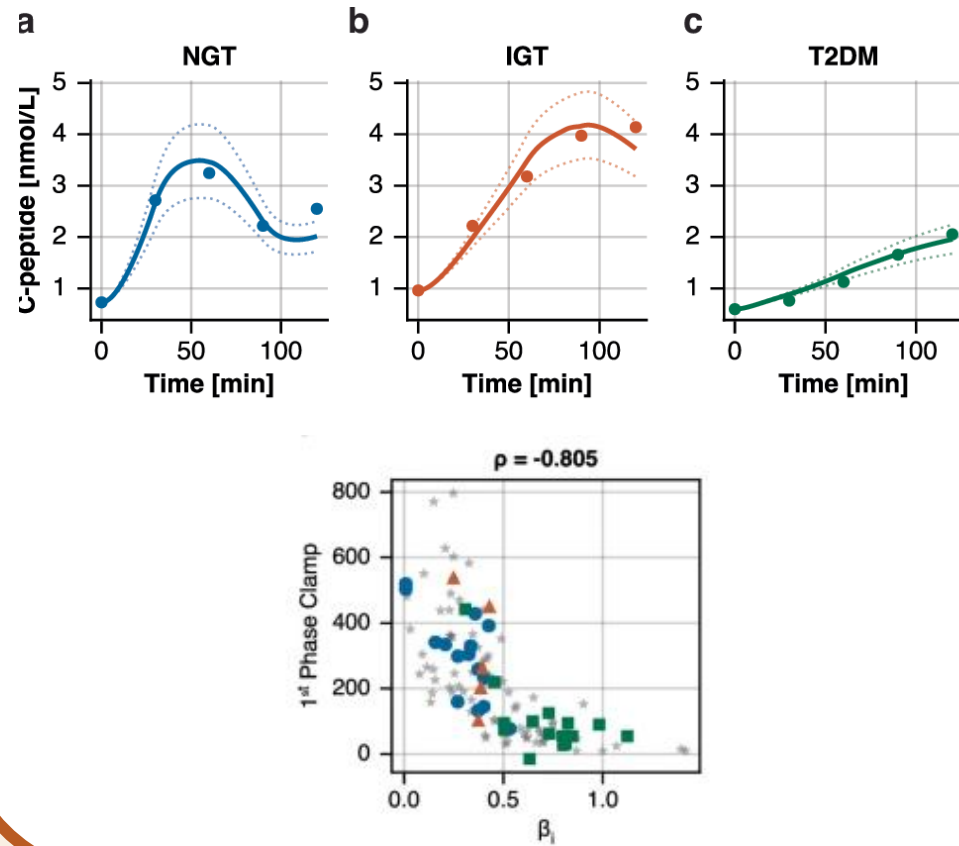


Analytical Expression

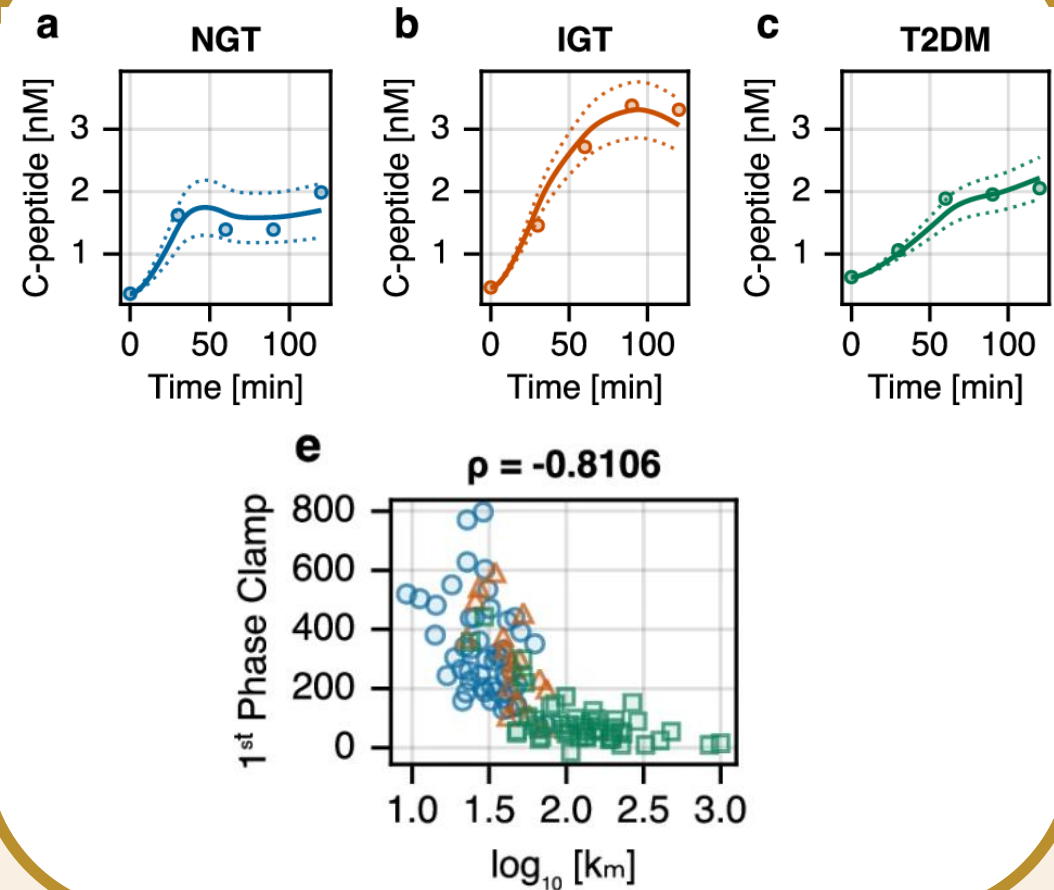


SYMBOLIC REGRESSION

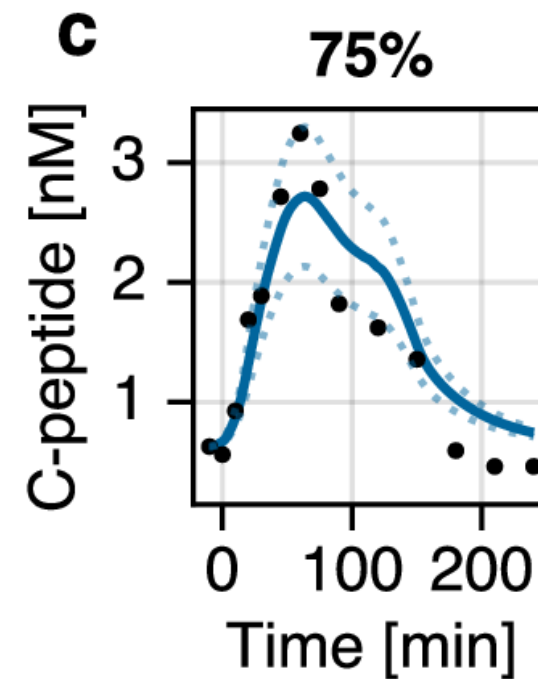
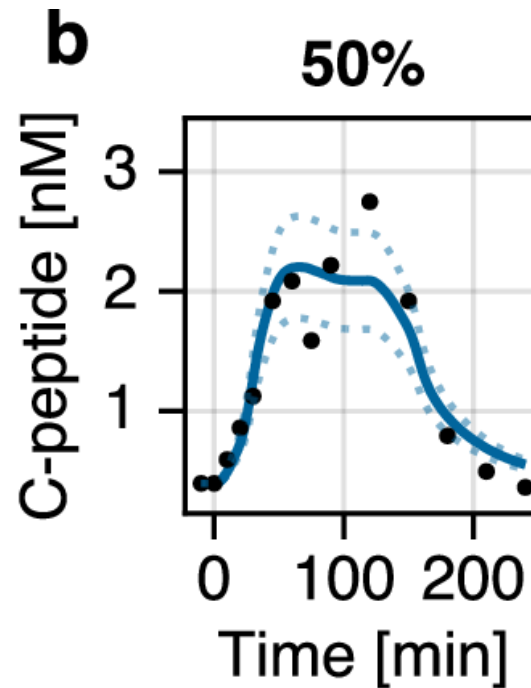
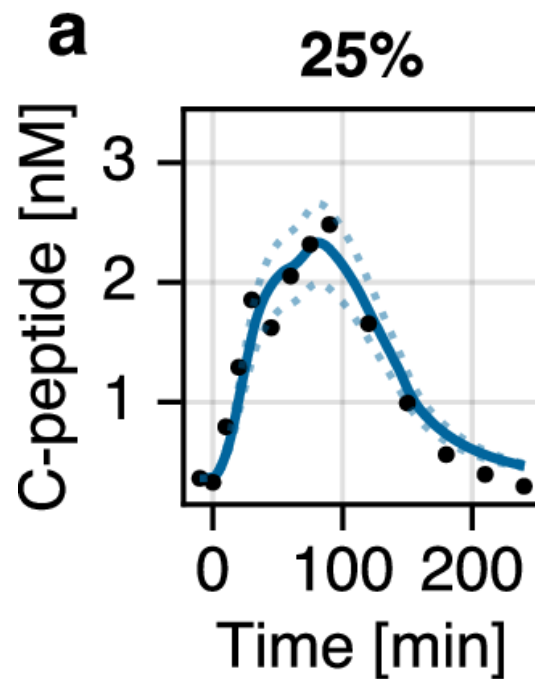
Neural Network



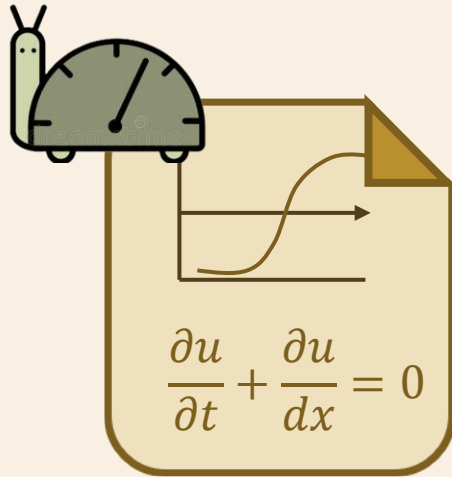
Analytical Expression



SYMBOLIC REGRESSION



RECAP



Mechanistic
Models

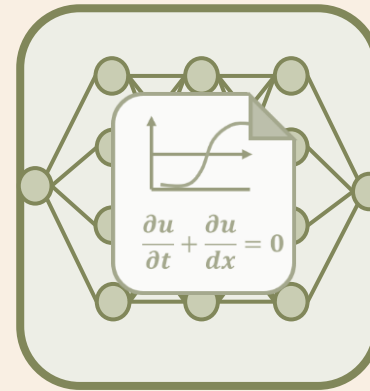
- Slow to build
- Rigid
- Little to no data

UDE

$$\begin{aligned}\frac{d[u]}{dt} &= k_1[u] + \text{NN} \\ \frac{d[v]}{dt} &= \text{NN} - k_2[v]\end{aligned}$$

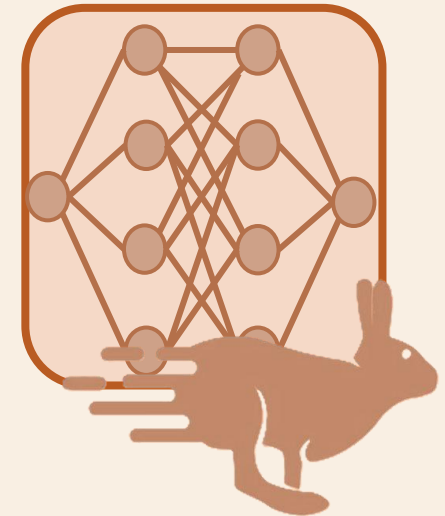
- Speeds up computation
- Flexible in user specified places
- Requires some data

PINN



- Speeds up computation
- plastic
- Requires (a lot) of high temporal data

Machine
Learning



- Relatively fast
- Highly plastic
- Data hungry

Thank you



Max de Rooij



Petrus Bult



Joshua Müller

✉ s.d.odonovan@tue.nl